

Reliability Analysis with R: A Companion Guide

Paul Govan

2026-05-22

Table of contents

Preface	7
How to Use This Book and the Tutorials Together	7
The ReliaLearnR Ecosystem	7
Installation	8
Chapter Overview	8
About the Author	8
1 Reliability, Availability, and Maintainability (RAM)	9
1.1 Introduction	9
1.2 Learning Objectives	9
1.3 What is RAM?	9
1.4 Reliability	10
1.5 Availability	11
1.6 Mean Time to Repair (MTTR)	11
1.7 Mean Time to Failure (MTTF)	12
1.8 Mean Time Between Failures (MTBF)	12
1.9 Failure Rate	13
1.10 Probability of Failure	14
1.10.1 The Reliability Curve	14
1.11 B_n or L_n Life	16
1.12 From Exponential to Weibull	17
1.13 The Bathtub Curve	17
1.14 System Reliability	20
1.14.1 Series Systems	20
1.14.2 Parallel Systems	21
1.15 Summary	23
2 Reliability Block Diagrams	24
2.1 Introduction	24
2.2 Learning Objectives	24
2.3 What is a Reliability Block Diagram?	24
2.4 Series Systems	25
2.4.1 Example: Water Pumping System	25
2.5 Parallel Systems	27
2.5.1 Example: Backup Power System	28

2.6	Mixed Systems	31
2.6.1	Example: Safety System	32
2.7	k-out-of-n Systems	34
2.7.1	Example: Flight Control Computers	35
2.8	System MTTF	37
2.8.1	Example: Two Pumps in Parallel	38
2.9	Common Cause Failure	38
2.9.1	The Beta-Factor Model	39
2.10	Introduction to Fault Tree Analysis	40
2.10.1	Gates	40
2.10.2	RBD—Fault Tree Duality	40
2.11	Case Study: Industrial Cooling System	43
2.12	Summary	44
3	Life Data Analysis	45
3.1	Introduction	45
3.2	Learning Objectives	45
3.3	Life Distributions	45
3.3.1	The Weibull Distribution	45
3.3.2	The Cumulative Distribution Function	46
3.4	WeibullR	47
3.4.1	Getting Started	47
3.4.2	Fitting a Weibull Model	48
3.4.3	Reading the Probability Plot	48
3.5	Data Censoring	49
3.5.1	Right-Censored Data (Suspensions)	50
3.5.2	Interval-Censored Data	51
3.5.3	Grouped Data	52
3.6	Parameter Estimation Methods	53
3.6.1	MRR vs MLE	53
3.7	Goodness of Fit	54
3.8	Choosing a Distribution	55
3.9	Sample Size Considerations	57
3.10	Other Weibull Models	59
3.10.1	The Weibayes Model	59
3.10.2	The 3-Parameter Weibull	60
3.11	Multi-Plots	61
3.12	Competing Failure Modes	61
3.13	Contour Plots	63
3.13.1	Comparing Contour Plots	64
3.14	Summary	65

4	Reliability Growth Analysis	66
4.1	Introduction	66
4.2	Learning Objectives	66
4.3	Reliability Growth Analysis	66
4.4	Test Planning	66
4.4.1	Reliability Demonstration Tests — <code>rdt()</code>	67
4.4.2	Forecasting Growth — <code>predict_rga()</code>	69
4.5	The Duane Model	69
4.5.1	Fitting with ReliaGrowR	71
4.6	The Crow-AMSAA Model	73
4.6.1	Fitting with ReliaGrowR	74
4.6.2	The Piecewise NHPP Model	76
4.6.3	Change Point Detection	77
4.7	Summary	78
5	Accelerated Life Testing	79
5.1	Introduction	79
5.2	Learning Objectives	79
5.3	The Acceleration Factor	79
5.4	The Arrhenius Model	80
5.4.1	WeibullR.ALT: Arrhenius-Lognormal Example	81
5.5	The Power Law Model	82
5.5.1	WeibullR.ALT: Power Law Example	82
5.6	Multiple Stress Factors	83
5.7	Parallelization	84
5.8	Relationship Plots	85
5.9	Summary	87
6	Repairable Systems Analysis	88
6.1	Introduction	88
6.2	Learning Objectives	88
6.3	Repairable vs Non-Repairable Systems	88
6.4	Failure Processes: HPP and NHPP	89
6.4.1	Homogeneous Poisson Process (HPP)	89
6.4.2	Non-Homogeneous Poisson Process (NHPP)	89
6.5	The Power Law Process	91
6.6	NHPP Analysis with <code>nhpp()</code>	92
6.7	Piecewise NHPP	94
6.8	Exposure Analysis	95
6.9	Mean Cumulative Function	97
6.10	Interpreting Results and Predicting Future Failures	99
6.10.1	Reading $\hat{\beta}$	99
6.10.2	Cumulative vs Instantaneous MTBF	99

6.10.3	Predicting Future Failures	100
6.11	Summary	101
7	Interactive Plots with ReliaPlotR	102
7.1	Introduction	102
7.2	Installation	102
7.3	Life Data Analysis Plots	102
7.3.1	Weibull Probability Plot — <code>plotly_wblr()</code>	103
7.3.2	Contour Plot — <code>plotly_contour()</code>	109
7.4	Reliability Growth Plots	113
7.4.1	Duane Plot — <code>plotly_duane()</code>	113
7.4.2	Crow-AMSAA Plot — <code>plotly_rga()</code>	115
7.5	Accelerated Life Testing Plots	117
7.5.1	ALT Probability Plot — <code>plotly_alt()</code>	117
7.5.2	Life-Stress Relationship Plot — <code>plotly_rel()</code>	119
7.6	Repairable Systems Plots	121
7.6.1	NHPP Plot — <code>plotly_nhpp()</code>	121
7.6.2	MCF Plot — <code>plotly_mcf()</code>	123
7.6.3	Exposure Plot — <code>plotly_exposure()</code>	125
7.7	Embedding in Reports and Dashboards	127
7.8	When to Use ReliaPlotR	127
7.9	Getting Help	128
7.10	Summary	128
8	Web-Based Analysis with ReliaShiny	129
8.1	Introduction	129
8.2	Installation and Launch	129
8.3	App Overview	129
8.4	Module Walkthroughs	131
8.4.1	RAM Module	131
8.4.2	Life Data Analysis Module	131
8.4.3	Reliability Growth Module	132
8.4.4	Accelerated Life Testing Module	133
8.4.5	Repairable Systems Module	134
8.4.6	Reliability Block Diagrams Module	136
8.5	Data Upload Tips	136
8.6	Downloading Results	136
8.7	When to Use ReliaShiny vs Scripted R	136
8.8	Getting Help	137
8.9	Summary	137
Appendix: Helper Functions Reference		138
	<code>rel()</code> — Reliability	138

avail() — Availability	139
mttf() — Mean Time to Failure	139
mtbf() — Mean Time Between Failures	140
fr() — Failure Rate	141
Validation Behaviour	141
Glossary and Formula Reference	143
Key Symbols	143
Key Formulas	145
RAM — Chapter 1	145
Weibull Distribution — Chapter 3	145
Reliability Block Diagrams — Chapter 2	146
Accelerated Life Testing — Chapter 5	146
Reliability Growth Analysis — Chapter 4	147
Repairable Systems — Chapter 6	147

Preface

This book is a companion reference guide to the **ReliaLearnR** interactive learning modules. Where the tutorials offer hands-on practice with immediate feedback and interactive widgets, this book provides readable, searchable, printable chapters that cover the same material in a narrative format — plus two additional chapters on the wider ReliaLearnR ecosystem.

How to Use This Book and the Tutorials Together

The interactive tutorials and this book cover the same core content but serve different purposes:

Feature	Interactive Tutorials	This Book
Instant code feedback	Yes	No
Quiz questions	Yes	Review questions
Interactive sliders	Yes	Static equivalents
Searchable / printable	No	Yes
Copy-paste code	Limited	Full
Ecosystem chapters (ReliaPlotR, ReliaShiny)	No	Yes

Recommended workflow: Read a chapter here first to understand the concepts and see the code, then open the corresponding tutorial to practice. The chapter order mirrors the tutorial progression.

The ReliaLearnR Ecosystem

ReliaLearnR is one part of a broader suite of R packages for reliability analysis:

Package	Role
ReliaLearnR	Interactive learning modules
WeibullR	Weibull life data analysis
ReliaGrowR	Reliability growth and repairable systems

Package	Role
WeibullR.ALT	Accelerated life testing
ReliaPlotR	Interactive reliability plots
ReliaShiny	Web-based reliability analysis app

Installation

Install the core packages needed to run the code in this book:

```
install.packages(c(
  "ReliaLearnR",
  "WeibullR",
  "ReliaGrowR",
  "WeibullR.ALT",
  "DiagrammeR",
  "ReliaPlotR",
  "ReliaShiny"
))
```

Chapter Overview

1. **RAM** — Foundational reliability, availability, and maintainability metrics.
2. **Reliability Block Diagrams** — Series, parallel, mixed, and k-out-of-n systems.
3. **Life Data Analysis** — Weibull distribution, censoring, MRR vs MLE, and contour plots.
4. **Reliability Testing** — Reliability growth models and accelerated life testing.
5. **Repairable Systems** — Power Law Process, MCF, and fleet-level analysis.
6. **Interactive Plots with ReliaPlotR** — Publishable interactive reliability charts.
7. **Web-Based Analysis with ReliaShiny** — Point-and-click reliability analysis.

About the Author

Paul Govan is the author of ReliaLearnR, ReliaGrowR, ReliaPlotR, and ReliaShiny. He can be reached at paul.govan2@gmail.com.

1 Reliability, Availability, and Maintainability (RAM)

1.1 Introduction

Reliability, Availability, and Maintainability (RAM) are the foundational metrics of reliability engineering. They describe how well a system performs its intended function, how often it is usable, and how quickly it can be restored when it fails. This chapter introduces these concepts and builds toward calculating probability of failure, B_n life, and system-level reliability.

1.2 Learning Objectives

By the end of this chapter, you will be able to:

- Define key reliability metrics: reliability, availability, and failure rate.
- Describe the significance of MTTR, MTTF, and MTBF.
- Calculate probability of failure using reliability data.
- Interpret B_n (or L_n) life values.
- Explain the bathtub curve and identify which failure region a component occupies given its β value.
- Compute system reliability for series and parallel configurations.

1.3 What is RAM?

Reliability: The ability of an item to perform its intended function without failure over a specified period.

Availability: The proportion of time an item is in a functioning condition.

Maintainability: The ease and speed with which an item can be restored to operational status after a failure.

These three concepts are interrelated: a highly reliable item will be available more often, and an item that is easy to maintain can be restored to service quickly, further improving availability.

1.4 Reliability

Reliability is the probability that an item will not fail under defined conditions for a specified period:

$$\text{Reliability} = \left(1 - \frac{\text{Failed Time}}{\text{Total Time}}\right) \times 100$$

Example: A motor ran for 5 years total and was failed for 10 of those days.

$$\text{Reliability} = \left(1 - \frac{10}{5 \times 365}\right) \times 100 = 94.5\%$$

Unreliability is the complement:

$$\text{Unreliability} = 1 - \text{Reliability} = \frac{\text{Failed Time}}{\text{Total Time}} \times 100$$

The `ReliaLearnR` package includes helper functions for these calculations:

```
library(ReliaLearnR)

# Motor: 5 years total, 10 days failed
rel(outageTime = 10, totalTime = 5 * 365)
```

```
[1] 0.9945205
```

Review

A machine ran for 3 years total and was failed for 5 of those days. What are the reliability and unreliability?

Answer

Reliability = $(1 - 5/(3 \times 365)) \times 100 = 99.5\%$. Unreliability = 0.5%.

1.5 Availability

Availability accounts for both failed time **and** scheduled maintenance — any time the item is unavailable for service:

$$\text{Availability} = \left(1 - \frac{\text{Unavailable Time}}{\text{Total Time}}\right) \times 100$$

Example: A motor ran for 5 years, was failed for 10 days, and had 15 days of scheduled maintenance.

$$\text{Availability} = \left(1 - \frac{10 + 15}{5 \times 365}\right) \times 100 = 93.2\%$$

```
# Motor: 5 years, 10 days failed + 15 days maintenance = 25 unavailable  
avail(unavailTime = 25, totalTime = 5 * 365)
```

```
[1] 0.9863014
```

Note: standby time (available but not in use) does not count as unavailable time.

Review

What is the difference between reliability and availability?

Answer

Availability includes scheduled maintenance downtime; reliability does not. $\text{Availability} \leq \text{Reliability}$ (all failed time reduces both, but only availability is also reduced by planned maintenance).

1.6 Mean Time to Repair (MTTR)

MTTR measures maintainability — the average time required to repair a failed item:

$$\text{MTTR} = \frac{\sum_{i=1}^n \text{RepairTime}_i}{\text{RepairCount}}$$

Example: 5 failures with repair times of 5, 10, 15, 8, and 12 days.

$$\text{MTTR} = \frac{5 + 10 + 15 + 8 + 12}{5} = 10 \text{ days}$$

1.7 Mean Time to Failure (MTTF)

MTTF is a reliability measure for *non-repairable* items. Once the item fails, it is replaced rather than repaired.

$$\text{MTTF} = \frac{\text{Total Time}}{\text{FailureCount}}$$

Example: 100 motors run for 5 years total; 5 failures occur.

$$\text{MTTF} = \frac{5 \times 100}{5} = 100 \text{ years}$$

```
mttf(failures = 5, totalTime = 5 * 100)
```

[1] 100

1.8 Mean Time Between Failures (MTBF)

MTBF is the reliability measure for *repairable systems*, where the item is restored to service after each failure.

$$\text{MTBF} = \frac{\text{Total Time}}{\text{FailureCount}}$$

The formula is identical to MTTF, but the interpretation differs: MTBF is the average time *between* successive failures on the same system.

Example: A motor fails 5 times during 10,000 total operating hours.

$$\text{MTBF} = \frac{10,000}{5} = 2,000 \text{ hours}$$

```
mtbf(failures = 5, totalTime = 10000)
```

[1] 2000

Try It

A fleet of 50 generators runs for 8 years. During that time, 20 failures occur. Calculate the MTBF in years.

```
fleet_size <- 50
years      <- 8
failures   <- 20
# Calculate MTBF = (fleet_size * years) / failures
```

Solution

```
fleet_size <- 50
years      <- 8
failures   <- 20
MTBF <- (fleet_size * years) / failures
MTBF # 20 years
```

```
[1] 20
```

Review

What is the difference between MTTF and MTBF?

Answer

MTTF is for **non-repairable** items (replaced after failure); MTBF is for **repairable** systems (restored to service after failure). Both use the same formula, but MTBF represents the average time *between* successive failures of the same system.

1.9 Failure Rate

The **failure rate** λ is the inverse of MTBF:

$$\lambda = \frac{1}{\text{MTBF}} = \frac{\text{FailureCount}}{\text{Total Time}}$$

A key assumption of MTBF is a **constant** failure rate throughout the item's life. While this is not always true in practice, it is a reasonable approximation for many systems during their useful life period.

Example: 100 motors run for 10 years; 20 failures occur.

$$\lambda = \frac{20}{10 \times 100} = 0.02 \text{ failures per year}$$

```
fr(failures = 20, totalTime = 10 * 100)
```

```
[1] 0.02
```

1.10 Probability of Failure

Under the constant failure rate assumption, failure times follow an **exponential distribution**. The cumulative probability of failure prior to time t is:

$$F(t) = 1 - e^{-\lambda t}$$

And the cumulative probability of **survival** (reliability function) is:

$$R(t) = 1 - F(t) = e^{-\lambda t}$$

Example: A motor has a failure rate of 0.1 failures/year. Probability of surviving 10 years:

$$R(10) = e^{-10 \times 0.1} = e^{-1} \approx 36.8\%$$

```
lambda <- 0.1
t      <- 10
R <- exp(-lambda * t)
R
```

```
[1] 0.3678794
```

1.10.1 The Reliability Curve

The shape of $R(t) = e^{-\lambda t}$ depends entirely on λ . Below are curves for three different failure rates:

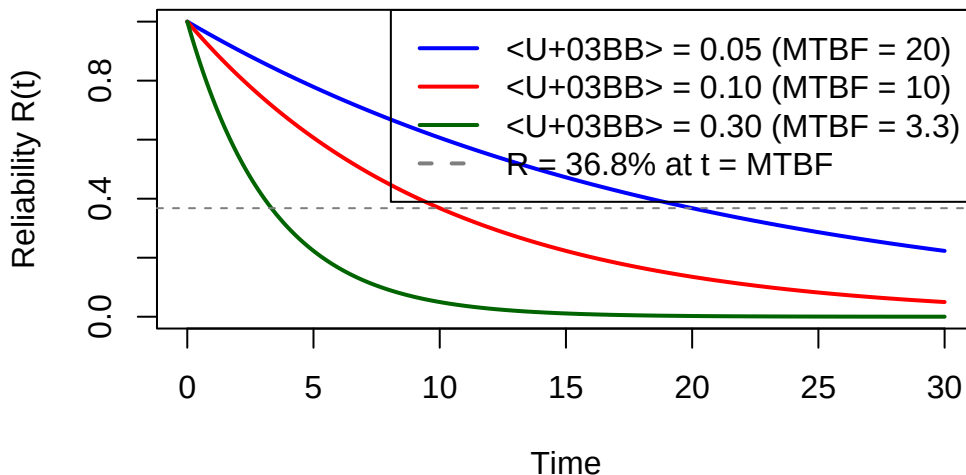
```

t <- seq(0, 30, by = 0.1)

plot(t, exp(-0.05 * t), type = "l", col = "blue", lwd = 2,
      xlab = "Time", ylab = "Reliability R(t)",
      main = "Exponential Reliability for Different Failure Rates",
      ylim = c(0, 1))
lines(t, exp(-0.1 * t), col = "red", lwd = 2)
lines(t, exp(-0.3 * t), col = "darkgreen", lwd = 2)
abline(h = exp(-1), col = "gray50", lty = 2)
legend("topright",
      legend = c("λ = 0.05 (MTBF = 20)", "λ = 0.10 (MTBF = 10)", "λ = 0.30 (MTBF = 3.3)",
                  "R = 36.8% at t = MTBF"),
      col = c("blue", "red", "darkgreen", "gray50"),
      lty = c(1, 1, 1, 2), lwd = 2)

```

Exponential Reliability for Different Failure Rates



Note that at $t = \text{MTBF} = 1/\lambda$, reliability is always $e^{-1} \approx 36.8\%$ regardless of λ . This is a universal property of the exponential distribution.

i Try It

A pump has a failure rate of 0.05 failures/year. Calculate the probability of survival at $t = 10$ years.

```
lambda <- 0.05
t      <- 10
# R(t) = exp(-lambda * t)
```

Solution

```
lambda <- 0.05
t      <- 10
R <- exp(-lambda * t)
R # ~0.607 (60.7% survival)
```

```
[1] 0.6065307
```

1.11 B_n or L_n Life

The B_n **life** is the time at which $n\%$ of a population is expected to have failed. Some industries use L_n instead, but the concept is identical.

To find B_n , set $F(t) = n/100$ and solve for t :

$$B_n = -\frac{\ln(1 - n/100)}{\lambda}$$

Example: Motor with $\lambda = 0.2$ failures/year. B10 life:

$$B_{10} = -\frac{\ln(1 - 0.1)}{0.2} = 0.526 \text{ years}$$

```
lambda <- 0.2
B10 <- -log(1 - 0.10) / lambda
B10 # 0.526 years
```

```
[1] 0.5268026
```

Try It

A component has a failure rate of 0.05 failures/year. Calculate the B10 life.

```
lambda <- 0.05
# B10 = -log(1 - 0.10) / lambda
```


Solution

```
lambda <- 0.05  
B10 <- -log(1 - 0.10) / lambda  
B10 # ~2.1 years
```

```
[1] 2.10721
```

Review

What is the relationship between failure rate and MTBF?

Answer

Failure rate is the **inverse** of MTBF: $\lambda = 1/\text{MTBF}$, and $\text{MTBF} = 1/\lambda$.

1.12 From Exponential to Weibull

The exponential model assumes a **constant** failure rate. This is appropriate for electronic components that fail at random, but many mechanical components have failure rates that change over time.

The **Weibull distribution** (Abernethy 2004) generalizes the exponential by adding a shape parameter β :

$$R(t) = e^{-(t/\eta)^\beta}$$

β	Failure rate	Typical cause
< 1	Decreasing	Infant mortality
$= 1$	Constant	Random failures — exponential applies
> 1	Increasing	Wear-out (fatigue, corrosion, aging)

When $\beta = 1$, the Weibull reduces to the exponential with $\lambda = 1/\eta$.

1.13 The Bathtub Curve

The failure rate of a population of items over its lifetime traces a characteristic shape known as the **bathtub curve** — named for its distinctive U-profile. Rather than drawing it as a stylized cartoon, we can build it analytically by compositing the three Weibull failure modes:

$$h(t) = \underbrace{h_{\text{infant}}(t)}_{\beta < 1} + \underbrace{h_{\text{random}}(t)}_{\beta = 1} + \underbrace{h_{\text{wear-out}}(t)}_{\beta > 1}$$

where the Weibull hazard rate is $h(t) = (\beta/\eta)(t/\eta)^{\beta-1}$. Each term represents a physically distinct failure mechanism — the bathtub shape is not assumed; it *emerges* from their sum.

```
t <- seq(0.01, 15, by = 0.01)

# Three Weibull hazard rate components
h_infant <- function(t, b = 0.4, e = 2) (b/e) * (t/e)^(b-1) # infant mortality
h_random <- function(t, b = 1.0, e = 5) (b/e) * (t/e)^(b-1) # random failures
h_wearout <- function(t, b = 3.5, e = 10) (b/e) * (t/e)^(b-1) # wear-out

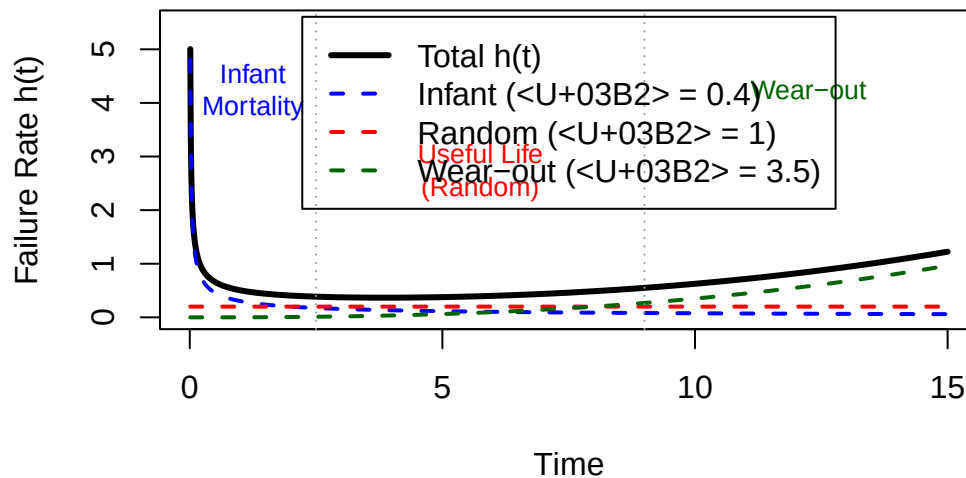
h_total <- h_infant(t) + h_random(t) + h_wearout(t)

plot(t, h_total, type = "l", col = "black", lwd = 3,
      xlab = "Time", ylab = "Failure Rate h(t)",
      main = "The Bathtub Curve: Three Composited Failure Modes",
      ylim = c(0, max(h_total) * 1.1))
lines(t, h_infant(t), col = "blue", lwd = 2, lty = 2)
lines(t, h_random(t), col = "red", lwd = 2, lty = 2)
lines(t, h_wearout(t), col = "darkgreen", lwd = 2, lty = 2)

abline(v = c(2.5, 9), col = "gray60", lty = 3)
text(1.25, max(h_total) * 0.85, "Infant\nMortality", col = "blue", cex = 0.85)
text(5.75, max(h_total) * 0.55, "Useful Life\n(Random)", col = "red", cex = 0.85)
text(12.25, max(h_total) * 0.85, "Wear-out", col = "darkgreen", cex = 0.85)

legend("top", inset = 0.02,
      legend = c("Total h(t)", "Infant ( = 0.4)", "Random ( = 1)", "Wear-out ( = 3.5)"),
      col = c("black", "blue", "red", "darkgreen"), lwd = c(3, 2, 2, 2), lty = c(1, 2, 2, 2))
```

The Bathtub Curve: Three Composited Failure Modes



Each region maps to a distinct engineering strategy:

- **Infant mortality** ($\beta < 1$): manufacturing defects, assembly errors, material flaws. Mitigated by *burn-in testing* — running units briefly before shipment to screen out defective items.
- **Useful life** ($\beta \approx 1$): random external shocks, load spikes, operator error. The constant failure rate means failures are unrelated to age; mitigated by *redundancy*.
- **Wear-out** ($\beta > 1$): fatigue, corrosion, material aging. Mitigated by *preventive replacement* scheduled before the B or B life is reached.

💡 Review

A pump has $\beta = 2.8$. Which region of the bathtub curve does it occupy, and what maintenance strategy is most appropriate?

Answer

$\beta > 1$ places it in the **wear-out** region. The failure rate is increasing with age, so a time-based **preventive replacement** strategy — replacing the pump before it reaches its B or B life — is most appropriate.

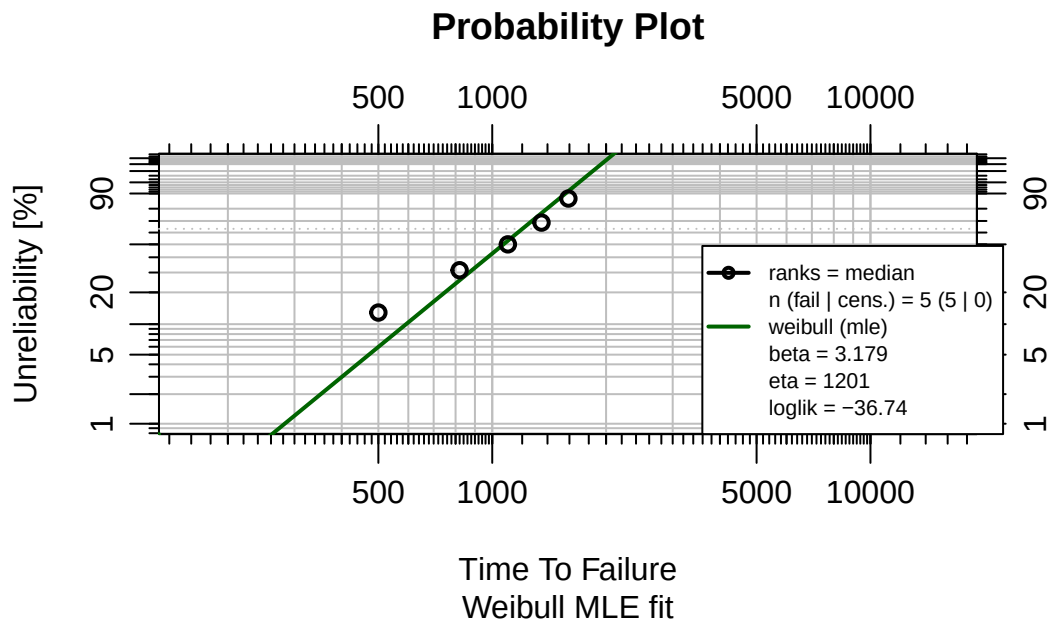
The bathtub curve is a population-level concept: individual units fail at random times, but the aggregate failure rate of a large fleet traces this shape. Fitting a separate Weibull to each failure mode is covered in Chapter 3.

The `WeibullR` package (Silkworth and Symynck 2022) provides functions for Weibull fitting in

R.

```
library(WeibullR)

failures <- c(500, 820, 1100, 1350, 1590)
fit <- MLEw2p(failures, show = TRUE)
```



A β close to 1 indicates random failures consistent with the exponential model. Higher values indicate wear-out behavior. The Life Data Analysis chapter covers Weibull analysis in depth.

1.14 System Reliability

Real systems combine many components. System reliability depends on how those components are arranged.

1.14.1 Series Systems

In a **series** configuration, every component must function for the system to function:

$$R_{\text{sys}} = R_1 \times R_2 \times \cdots \times R_n = \prod_{i=1}^n R_i$$

A series system is always *less* reliable than its weakest component.

```
R_components <- c(0.90, 0.90, 0.90)
R_series <- prod(R_components)
R_series # 72.9%
```

```
[1] 0.729
```

1.14.2 Parallel Systems

In a **parallel** configuration, only one component needs to function — redundancy. The system fails only when *all* components fail:

$$R_{\text{sys}} = 1 - (1 - R_1)(1 - R_2) \cdots (1 - R_n)$$

```
R_parallel <- 1 - prod(1 - R_components)
R_parallel # 99.9%
```

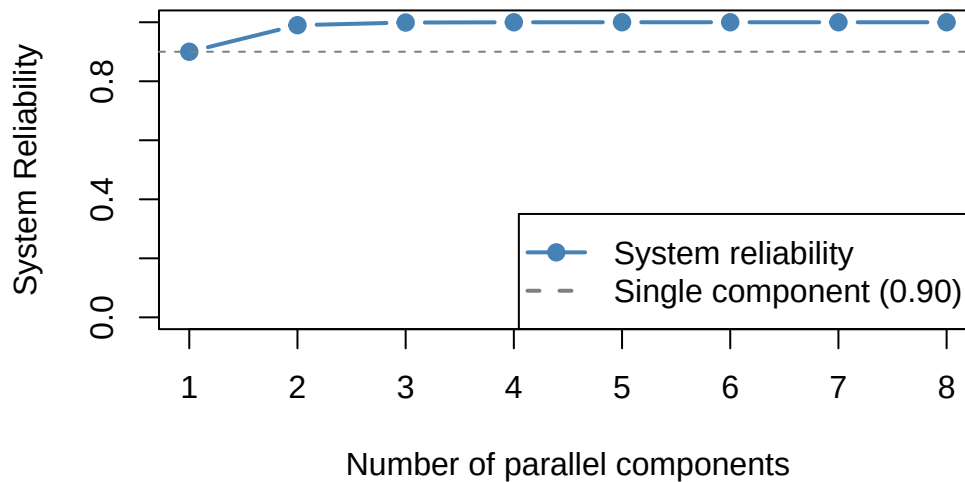
```
[1] 0.999
```

The effect of redundancy increases dramatically with the number of parallel components:

```
n_vals <- 1:8
R_c <- 0.90
R_par <- 1 - (1 - R_c)^n_vals

plot(n_vals, R_par, type = "b", col = "steelblue", lwd = 2, pch = 19,
     xlab = "Number of parallel components",
     ylab = "System Reliability",
     main = "Effect of Redundancy on System Reliability (R_component = 0.90)",
     ylim = c(0, 1))
abline(h = R_c, col = "gray50", lty = 2)
legend("bottomright",
     legend = c("System reliability", "Single component (0.90)"),
     col = c("steelblue", "gray50"), lty = c(1, 2), pch = c(19, NA), lwd = 2)
```

Effect of Redundancy on System Reliability ($R_{\text{component}} = 0.85$)



Try It

Four components each have reliability $R = 0.85$. Calculate the series and parallel system reliability.

```
R_comp <- 0.85
# Series:  R_sys = prod(R_components)
# Parallel: R_sys = 1 - prod(1 - R_components)
```

Solution

```
R_comp <- rep(0.85, 4)
R_series  <- prod(R_comp)
R_parallel <- 1 - prod(1 - R_comp)
R_series  # ~0.522 (52.2%)
```

```
[1] 0.5220062
```

```
R_parallel # ~0.9995 (99.95%)
```

```
[1] 0.9994937
```

Review

Three components each have 80% reliability. What is the series system reliability?

Answer

$$R_{\text{sys}} = 0.8 \times 0.8 \times 0.8 = 0.512 = 51.2\%$$

1.15 Summary

Key takeaways:

- $R(t) = e^{-\lambda t}$ — exponential reliability under a constant failure rate.
- At $t = \text{MTBF} = 1/\lambda$, reliability is always 36.8%.
- MTTF is for non-repairable items; MTBF is for repairable systems.
- Series systems: $R_{\text{sys}} = \prod R_i$ — reliability decreases with more components.
- Parallel systems: $R_{\text{sys}} = 1 - \prod(1 - R_i)$ — redundancy increases reliability.
- When $\beta \neq 1$, use the Weibull distribution (see Chapter 3).

2 Reliability Block Diagrams

2.1 Introduction

In Chapter 1, you learned to calculate reliability metrics for individual components. This chapter extends those concepts to *systems* — collections of components whose arrangement determines whether the system succeeds or fails.

A **Reliability Block Diagram (RBD)** is a graphical tool for modeling how component reliabilities combine to produce system-level reliability. RBDs are widely used in reliability engineering to analyze designs, identify vulnerabilities, and evaluate the benefit of redundancy (Billinton and Allan 1992).

2.2 Learning Objectives

By the end of this chapter, you will be able to:

- Define reliability block diagrams and relate them to physical systems.
- Compute system reliability for series, parallel, and mixed configurations.
- Interpret k-out-of-n (voting) systems and calculate their reliability.
- Describe the relationship between RBDs and Fault Tree Analysis.
- Apply RBD calculations to a realistic multi-component system.

2.3 What is a Reliability Block Diagram?

An RBD represents each component as a **block** with a known reliability value. Blocks are connected by lines showing logical dependencies — not physical connections. A path from left (input) to right (output) through functioning blocks represents a working system.

Each block has a reliability $R_i(t)$ derived from the exponential model (Chapter 1) or a Weibull fit (Chapter 3).

```
library(DiagrammeR)
```


2.4 Series Systems

In a **series** system, all components must function for the system to succeed:

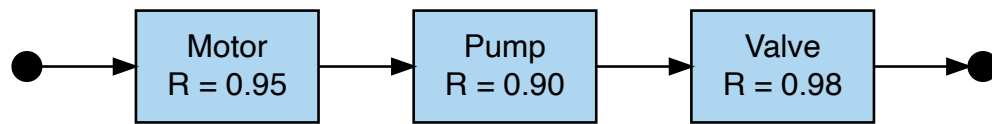
$$R_{\text{sys}} = R_1 \times R_2 \times \cdots \times R_n = \prod_{i=1}^n R_i$$

A series system is *always less reliable* than its weakest component. Adding more components in series can only reduce system reliability.

2.4.1 Example: Water Pumping System

A pumping system requires a motor ($R = 0.95$), a pump ($R = 0.90$), and a control valve ($R = 0.98$) to all be operational.

```
grViz("  
digraph series {  
    rankdir = LR  
    graph [bgcolor = transparent]  
    node [shape = rectangle, style = filled, fillcolor = '#AED6F1',  
        fontname = 'sans-serif', fontsize = 12, margin = '0.2,0.1']  
    edge [arrowsize = 0.8]  
    I [shape = point, width = 0.15, fillcolor = black]  
    A [label = 'Motor\nR = 0.95']  
    B [label = 'Pump\nR = 0.90']  
    C [label = 'Valve\nR = 0.98']  
    O [shape = point, width = 0.15, fillcolor = black]  
    I -> A -> B -> C -> O  
}  
")
```



```
R_motor <- 0.95
R_pump <- 0.90
R_valve <- 0.98

R_series <- R_motor * R_pump * R_valve
R_series # ~0.838
```

```
[1] 0.8379
```

The system reliability is approximately 83.8%, lower than any individual component.

Try It

A conveyor belt system has 5 components in series with reliabilities 0.98, 0.96, 0.99, 0.94, and 0.97. Calculate the system reliability.

```
R_components <- c(0.98, 0.96, 0.99, 0.94, 0.97)
# R_series <- prod(R_components)
```

Solution

```
R_components <- c(0.98, 0.96, 0.99, 0.94, 0.97)
R_series <- prod(R_components)
R_series # ~0.845
```

```
[1] 0.8492432
```

Review

In a series system, which component has the greatest influence on system reliability?

Answer

The **least reliable** component. It is the bottleneck — improving it produces the largest gain in system reliability.

2.5 Parallel Systems

In a **parallel** system, only one component needs to function — **active redundancy**. The system fails only if all components fail simultaneously:

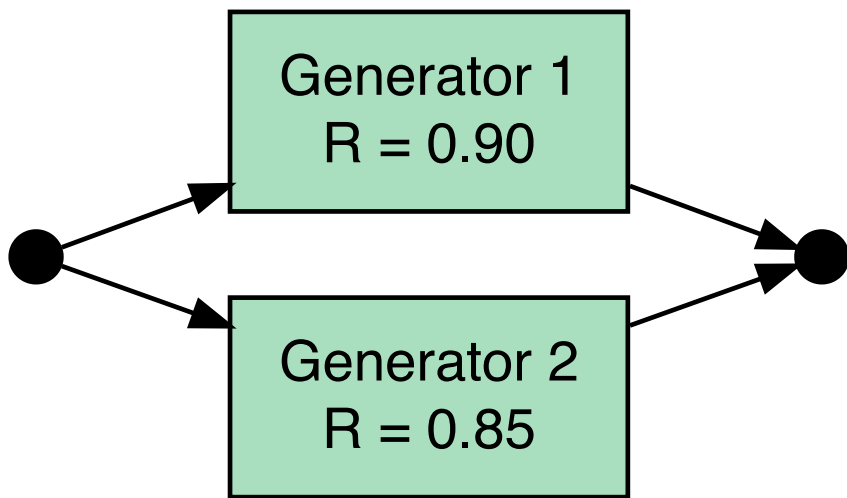
$$R_{\text{sys}} = 1 - \prod_{i=1}^n (1 - R_i)$$

Adding parallel components always increases system reliability.

2.5.1 Example: Backup Power System

A primary generator ($R = 0.90$) and a standby generator ($R = 0.85$) — either alone keeps the system running.

```
grViz("
digraph parallel {
  rankdir = LR
  graph [bgcolor = transparent]
  node [shape = rectangle, style = filled, fillcolor = '#A9DFBF',
        fontname = 'sans-serif', fontsize = 12, margin = '0.2,0.1']
  edge [arrowsize = 0.8]
  I [shape = point, width = 0.15, fillcolor = black]
  A [label = 'Generator 1\nR = 0.90']
  B [label = 'Generator 2\nR = 0.85']
  O [shape = point, width = 0.15, fillcolor = black]
  I -> A -> O
  I -> B -> O
}
")
```



```

R_primary <- 0.90
R_standby <- 0.85

R_parallel <- 1 - (1 - R_primary) * (1 - R_standby)
R_parallel # 0.985

```

```
[1] 0.985
```

The parallel system reliability is 98.5%, much higher than either generator alone.

The benefit of redundancy grows with each additional component:

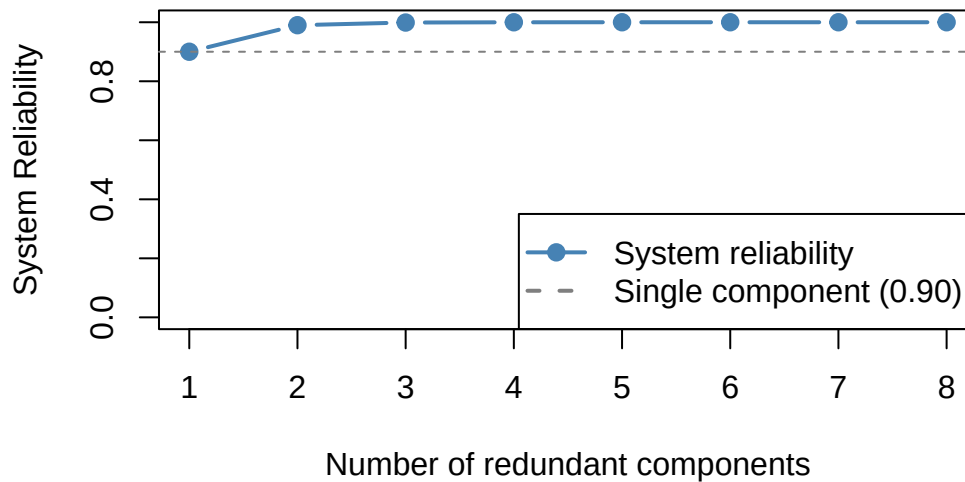
```

n_vals <- 1:8
Rc      <- 0.90
R_par   <- 1 - (1 - Rc)^n_vals

plot(n_vals, R_par, type = "b", col = "steelblue", lwd = 2, pch = 19,
     xlab = "Number of redundant components",
     ylab = "System Reliability",
     main = "Parallel System Reliability vs. Redundancy (R_component = 0.90)",
     ylim = c(0, 1))
abline(h = Rc, col = "gray50", lty = 2)
legend("bottomright",
     legend = c("System reliability", "Single component (0.90)"),
     col = c("steelblue", "gray50"), lty = c(1, 2), pch = c(19, NA), lwd = 2)

```

Parallel System Reliability vs. Redundancy ($R_{\text{component}} =$



Try It

A critical pump station has 3 pumps in parallel. Each pump has reliability 0.88. What is the system reliability?

```
R_components <- c(0.88, 0.88, 0.88)
# R_parallel <- 1 - prod(1 - R_components)
```

Solution

```
R_components <- c(0.88, 0.88, 0.88)
R_parallel <- 1 - prod(1 - R_components)
R_parallel # ~0.9983
```

```
[1] 0.998272
```

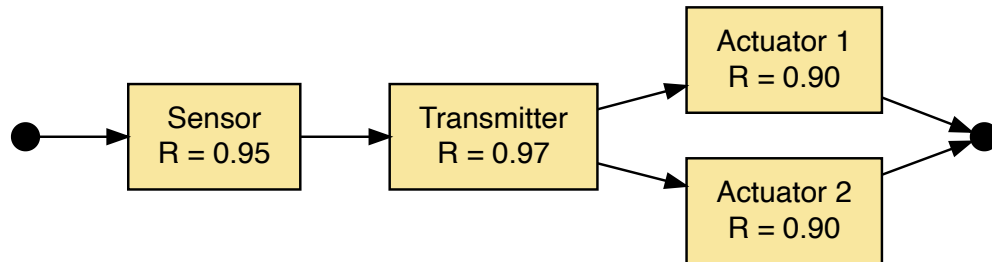
2.6 Mixed Systems

Most real systems combine series and parallel blocks. To analyze a **mixed** system, decompose it into subsystems and apply series and parallel rules step by step, working from the innermost blocks outward.

2.6.1 Example: Safety System

- **Subsystem A:** a sensor ($R = 0.95$) and a transmitter ($R = 0.97$) in series.
- **Subsystem B:** two redundant actuators (each $R = 0.90$) in parallel.
- Subsystems A and B in series (both must work).

```
grViz("
digraph mixed {
  rankdir = LR
  graph [bgcolor = transparent]
  node [shape = rectangle, style = filled, fillcolor = '#F9E79F',
        fontname = 'sans-serif', fontsize = 12, margin = '0.2,0.1']
  edge [arrowsize = 0.8]
  I [shape = point, width = 0.15, fillcolor = black]
  S [label = 'Sensor\nR = 0.95']
  T [label = 'Transmitter\nR = 0.97']
  A1 [label = 'Actuator 1\nR = 0.90']
  A2 [label = 'Actuator 2\nR = 0.90']
  O [shape = point, width = 0.15, fillcolor = black]
  I -> S -> T -> A1 -> O
  T -> A2 -> O
}
")
```

```
# Subsystem A (series)
R_A <- 0.95 * 0.97
R_A
```

```
[1] 0.9215
```

```
# Subsystem B (parallel)
R_B <- 1 - (1 - 0.90) * (1 - 0.90)
R_B
```

```
[1] 0.99
```

```
# Overall system (A and B in series)
R_system <- R_A * R_B
R_system
```

```
[1] 0.912285
```

Review

A system has two subsystems in series. Subsystem 1 has two components in parallel (each $R = 0.85$); Subsystem 2 is a single component with $R = 0.92$. What is the system reliability?

Answer

$$R_{\text{sub1}} = 1 - (1 - 0.85)^2 = 0.9775. \quad R_{\text{sys}} = 0.9775 \times 0.92 \approx 0.899.$$

2.7 k-out-of-n Systems

A **k-out-of-n** system succeeds if at least k of its n identical components function:

- $k = n$: all must work $\rightarrow$ series.
- $k = 1$: at least one works $\rightarrow$ parallel.
- $1 < k < n$: voting or load-sharing system.

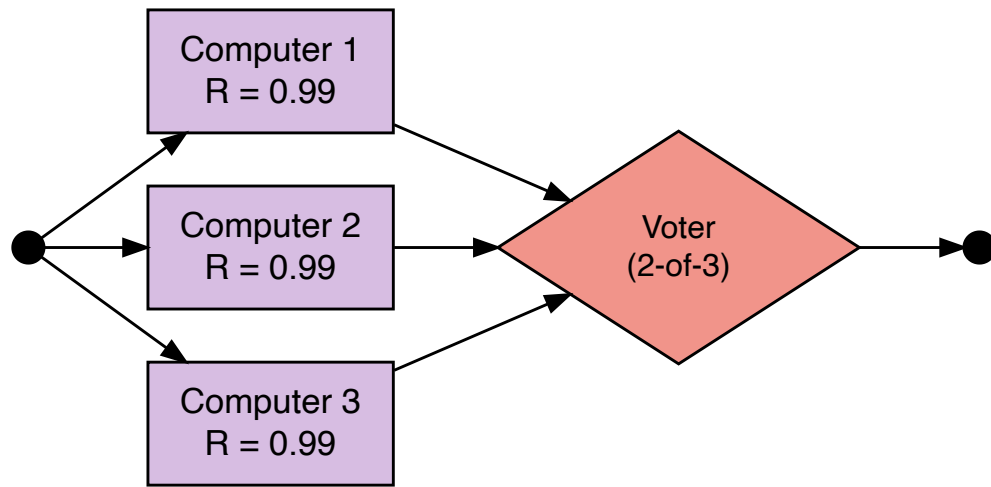
The reliability of a k-out-of-n system (identical components, each with reliability p) follows the binomial distribution:

$$R_{k/n} = \sum_{i=k}^n \binom{n}{i} p^i (1-p)^{n-i} = 1 - \text{pbinom}(k-1, n, 1-p)$$

2.7.1 Example: Flight Control Computers

A 2-out-of-3 voter: at least 2 of 3 computers must agree. Each computer has $R = 0.99$.

```
grViz("
digraph koon {
  rankdir = LR
  graph [bgcolor = transparent]
  node [shape = rectangle, style = filled, fillcolor = '#D7BDE2',
        fontname = 'sans-serif', fontsize = 12, margin = '0.2,0.1']
  edge [arrowsize = 0.8]
  I [shape = point, width = 0.15, fillcolor = black]
  C1 [label = 'Computer 1\nR = 0.99']
  C2 [label = 'Computer 2\nR = 0.99']
  C3 [label = 'Computer 3\nR = 0.99']
  V [label = 'Voter\n(2-of-3)', shape = diamond, fillcolor = '#F1948A', fontsize = 11]
  O [shape = point, width = 0.15, fillcolor = black]
  I -> C1 -> V
  I -> C2 -> V
  I -> C3 -> V
  V -> O
}
")
```



```

n <- 3      # total components
k <- 2      # minimum required
p <- 0.99   # individual reliability

R_voting <- 1 - pbinom(k - 1, n, 1 - p)
R_voting   # 0.9997

```

```
[1] 0.000298
```

Try It

A 3-out-of-5 redundant sensor array. Each sensor has reliability $p = 0.95$. Calculate the system reliability.

```

n <- 5
k <- 3
p <- 0.95
# R_sys <- 1 - pbinom(k - 1, n, 1 - p)

```

Solution

```

n <- 5
k <- 3
p <- 0.95
R_sys <- 1 - pbinom(k - 1, n, 1 - p)
R_sys   # ~0.9988

```

```
[1] 0.001158125
```

2.8 System MTTF

For a **series** system of components with constant failure rates, the system failure rate equals the sum of component failure rates:

$$\lambda_{\text{sys}} = \sum_{i=1}^n \lambda_i \quad \text{MTTF}_{\text{sys}} = \frac{1}{\lambda_{\text{sys}}}$$

For n identical parallel components each with failure rate λ :

$$\text{MTTF}_{\text{parallel}} = \frac{1}{\lambda} \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right)$$

2.8.1 Example: Two Pumps in Parallel

```
lambda <- 0.01 # failures per hour

MTTF_single <- 1 / lambda
MTTF_parallel <- (1 / lambda) * (1 + 1/2)

MTTF_single # 100 hours
```

```
[1] 100
```

```
MTTF_parallel # 150 hours - 50% longer with one spare
```

```
[1] 150
```

Review

Three components in series have failure rates 0.02, 0.03, and 0.05 failures/hour. What is the system MTTF?

Answer

$\lambda_{\text{sys}} = 0.02 + 0.03 + 0.05 = 0.10$, so $\text{MTTF} = 1/0.10 = 10$ hours.

2.9 Common Cause Failure

All the analyses above assume that component failures are **statistically independent** — if Component A fails, it tells us nothing about Component B's probability of failing. In practice, this assumption can break down.

Common Cause Failure (CCF) occurs when a single event simultaneously causes multiple components to fail. Typical sources include:

- A shared environment (vibration, temperature, contamination) that degrades all components together
- A common design flaw or manufacturing batch defect

- A maintenance error that affects multiple redundant units at once

CCF is especially dangerous because **redundancy does not protect against it**: if both pumps in a parallel system can be destroyed by the same flooding event, adding a third pump does not help.

2.9.1 The Beta-Factor Model

The simplest CCF model splits each component's total failure rate λ into two parts:

$$\lambda_{\text{ind}} = (1 - \beta_{\text{CCF}}) \cdot \lambda \quad (\text{independent failures})$$

$$\lambda_{\text{CCF}} = \beta_{\text{CCF}} \cdot \lambda \quad (\text{common-cause failures})$$

The **beta-factor** β_{CCF} is the fraction of failures attributable to common causes. Typical values range from 0.01 (1%) for well-separated, dissimilar components to 0.20 (20%) for closely coupled identical units.

System reliability for a two-component parallel system with CCF:

$$R_{\text{sys}} = \underbrace{[1 - (1 - R_{\text{ind}})^2]}_{\text{parallel, independent part}} \times \underbrace{e^{-\lambda_{\text{CCF}} t}}_{\text{common-cause part}}$$

```
lambda <- 0.001 # total component failure rate (per hour)
t      <- 1000  # mission time (hours)

beta_vals <- c(0, 0.05, 0.10, 0.20)

results <- sapply(beta_vals, function(b) {
  R_ind <- exp(-(1 - b) * lambda * t) # independent reliability
  R_ccf <- exp(-b * lambda * t)       # common-cause reliability
  R_sys <- (1 - (1 - R_ind)^2) * R_ccf
  round(R_sys, 4)
})

data.frame(beta_CCF = beta_vals, R_system = results)
```

	beta_CCF	R_system
1	0.00	0.6004
2	0.05	0.5935
3	0.10	0.5862
4	0.20	0.5705

Even a modest $\beta_{CCF} = 0.10$ visibly reduces the system reliability that redundancy would otherwise provide.

Review

Why doesn't adding more parallel components help when β_{CCF} is large?

Answer

When β_{CCF} is large, a significant fraction of failures are **common cause** — they simultaneously affect all components regardless of how many there are. Adding more identical, coupled components may even increase the common-cause failure rate if they share the same environment or design flaw. Effective mitigation requires **diversity** (different designs, manufacturers, or locations) rather than simple redundancy.

CCF data and beta-factor values for different equipment types are tabulated in IEC 61508 (functional safety) and MIL-HDBK-217F (electronic reliability). For nuclear applications, the NUREG/CR-5497 handbook provides detailed CCF databases.

2.10 Introduction to Fault Tree Analysis

Fault Tree Analysis (FTA) (Vesely et al. 1981) is a top-down approach that starts with an undesirable event (the **top event**) and works backward to identify combinations of component failures that could cause it.

While an RBD asks “What must work for the system to succeed?”, a fault tree asks “What can cause the system to fail?”

2.10.1 Gates

- **AND gate:** top event occurs only if *all* inputs occur — corresponds to **parallel** RBD components (all must fail).
- **OR gate:** top event occurs if *any* input occurs — corresponds to **series** RBD components (any failure causes system failure).

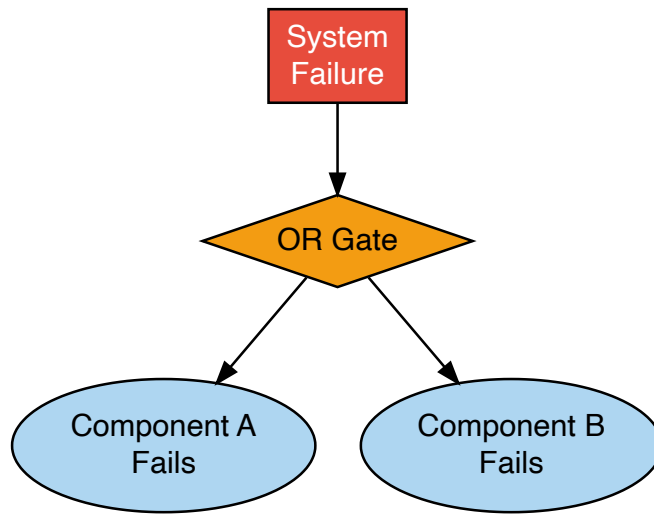
2.10.2 RBD—Fault Tree Duality

RBD configuration	Fault tree gate for system failure
Series (all must work)	OR gate (any failure causes system failure)
Parallel (any can work)	AND gate (all must fail for system failure)


```

grViz("
digraph fta {
    rankdir = TB
    graph [bgcolor = transparent]
    node [fontname = 'sans-serif', fontsize = 12, style = filled]
    edge [arrowsize = 0.8]
    Top [label = 'System\nFailure', shape = rectangle, fillcolor = '#E74C3C', fontcolor = white]
    OR  [label = 'OR Gate', shape = diamond, fillcolor = '#F39C12']
    F1  [label = 'Component A\nFails', shape = ellipse, fillcolor = '#AED6F1']
    F2  [label = 'Component B\nFails', shape = ellipse, fillcolor = '#AED6F1']
    Top -> OR
    OR  -> F1
    OR  -> F2
}
")

```



This OR gate corresponds to a two-component series RBD: any single failure causes system failure. For complex fault trees, the R package **FaultTree** on CRAN provides computational support.

Review

A fault tree has an AND gate combining two component failures. Which RBD configuration does this correspond to?

Answer

Parallel — an AND gate means the system fails only if both components fail simultaneously, which is exactly the parallel (redundant) configuration.

2.11 Case Study: Industrial Cooling System

An industrial cooling system has the following architecture:

1. **Water supply subsystem:** two pumps in parallel (each $R = 0.92$), followed by a filter in series ($R = 0.99$).
2. **Control subsystem:** a primary controller ($R = 0.97$) and a backup controller in parallel ($R = 0.95$).
3. Both subsystems must work (series at system level).

```
# Step 1 - Water supply subsystem
R_pump_parallel <- 1 - (1 - 0.92)^2
R_water <- R_pump_parallel * 0.99
R_water
```

```
[1] 0.983664
```

```
# Step 2 - Control subsystem
R_control <- 1 - (1 - 0.97) * (1 - 0.95)
R_control
```

```
[1] 0.9985
```

```
# Step 3 - Overall system (both subsystems in series)
R_system <- R_water * R_control
R_system
```

```
[1] 0.9821885
```

Try It

Modify the case study: the filter is upgraded to $R = 0.999$. Recalculate the system reliability.

```
R_pump1 <- 0.92
R_pump2 <- 0.92
R_filter <- 0.999 # upgraded from 0.99
R_ctrl1 <- 0.97
R_ctrl2 <- 0.95
```

Solution

```
R_pump1 <- 0.92
R_pump2 <- 0.92
R_filter <- 0.999
R_ctrl1 <- 0.97
R_ctrl2 <- 0.95

R_pump_parallel <- 1 - (1 - R_pump1) * (1 - R_pump2)
R_water <- R_pump_parallel * R_filter
R_control <- 1 - (1 - R_ctrl1) * (1 - R_ctrl2)
R_system <- R_water * R_control
R_system # ~0.985
```

```
[1] 0.9911175
```

2.12 Summary

Key takeaways:

- **Series:** $R_{\text{sys}} = \prod R_i$ — every component must work; reliability decreases with more components.
- **Parallel:** $R_{\text{sys}} = 1 - \prod(1 - R_i)$ — redundancy; only one component needs to work.
- **Mixed:** decompose into subsystems, apply formulas step by step.
- **k-out-of-n:** $1 - \text{pbinom}(k-1, n, 1-p)$ for voting/load-sharing systems.
- **Series MTTF:** $1/\sum \lambda_i$; **Parallel MTTF:** $(1/\lambda) \sum_{i=1}^n (1/i)$.
- **FTA duality:** series RBD → OR gate; parallel RBD → AND gate.

3 Life Data Analysis

3.1 Introduction

Life Data Analysis is the study of how systems function over time. Life data tells us how long a system will last, when it will likely fail, and how often it will need maintenance. In reliability engineering it is often called **Weibull analysis** because the Weibull distribution is the dominant model (Abernethy 2004).

3.2 Learning Objectives

By the end of this chapter, you will be able to:

- Describe the purpose of Weibull analysis in reliability engineering.
- Differentiate between types of data censoring: right-censored and interval-censored.
- Differentiate between Weibull models: 2-parameter, 3-parameter, and Weibayes.
- Apply Median Rank Regression (MRR) and Maximum Likelihood Estimation (MLE).
- Compare Weibull and lognormal fits using goodness-of-fit statistics.
- Determine how many failures are needed for reliable Weibull estimates.
- Interpret results using probability plots and contour plots.

3.3 Life Distributions

3.3.1 The Weibull Distribution

The **Weibull distribution** is a flexible continuous probability distribution widely used in life data analysis. Its flexibility comes from a shape parameter β that allows it to mimic the normal, exponential, and log-normal distributions.

3.3.2 The Cumulative Distribution Function

The Weibull reliability (survival) function is:

$$R(t) = 1 - F(t) = e^{-(t/\eta)^\beta}$$

where $R(t)$ is the cumulative probability of *survival* prior to time t , β is the shape parameter, and η is the scale parameter.

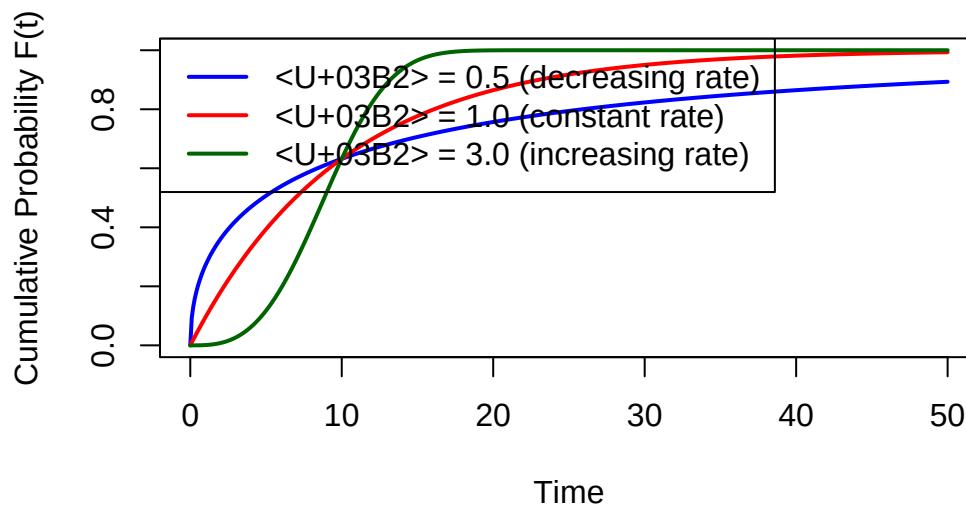
The shape parameter β has a direct physical interpretation:

β	Failure rate	Typical cause
< 1	Decreasing	Infant mortality
$= 1$	Constant	Random failures (exponential)
> 1	Increasing	Wear-out (fatigue, corrosion, aging)

```
t      <- seq(0, 50, by = 0.1)
etas <- 10

# Plot CDFs for three values of beta
plot(t, pweibull(t, shape = 0.5, scale = etas), type = "l", col = "blue", lwd = 2,
      xlab = "Time", ylab = "Cumulative Probability F(t)",
      main = "Weibull CDF for Different Shape Parameters ( = 10)",
      ylim = c(0, 1))
lines(t, pweibull(t, shape = 1.0, scale = etas), col = "red", lwd = 2)
lines(t, pweibull(t, shape = 3.0, scale = etas), col = "darkgreen", lwd = 2)
legend("topleft",
      legend = c(" = 0.5 (decreasing rate)", " = 1.0 (constant rate)",
                  " = 3.0 (increasing rate)"),
      col = c("blue", "red", "darkgreen"), lwd = 2)
```

Weibull CDF for Different Shape Parameters (β) = :



Key property: at $t = \eta$, $F(\eta) = 1 - e^{-1} \approx 63.2\%$ for any value of β . This is why η is called the **characteristic life**.

Review

Which value of β indicates an increasing failure rate?

Answer

$\beta > 1$. A shape parameter greater than 1 indicates a wear-out failure mode where the failure rate increases over time.

3.4 WeibullR

The `WeibullR` package (Silkworth and Symynck 2022; Silkworth 2020) is the primary R tool for Weibull analysis.

3.4.1 Getting Started

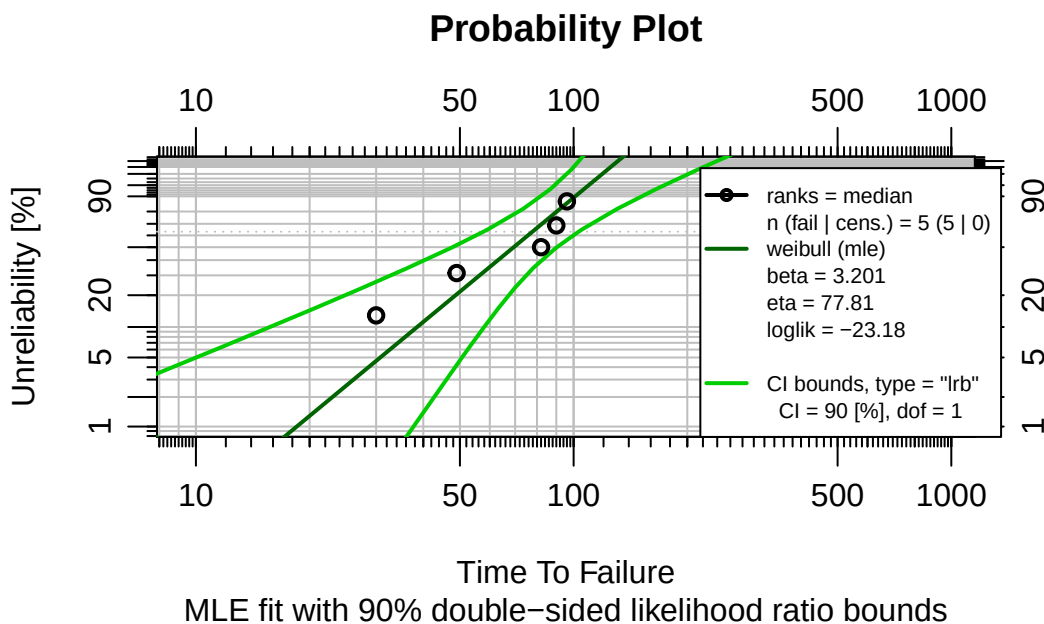
```
install.packages("WeibullR")
```

```
library(WeibullR)
```

3.4.2 Fitting a Weibull Model

A factory has 5 machines that failed at times 30, 49, 82, 90, and 96. Use `MLEw2p()` to fit a 2-parameter Weibull model using Maximum Likelihood Estimation and display the probability plot.

```
failures <- c(30, 49, 82, 90, 96)
fit <- MLEw2p(failures, bounds = TRUE, show = TRUE)
```



3.4.3 Reading the Probability Plot

The **Weibull probability plot** shows time on the horizontal axis (log scale) and cumulative unreliability $F(t)$ on the vertical axis (log-log scale). This double-log transformation linearizes Weibull data — the result is a straight line when the Weibull model fits.

- Read off $F(t)$ at any time to get the probability of failure by that time.
- Reliability at time t is $R(t) = 1 - F(t)$.
- β (shape) and η (scale) are shown in the legend.

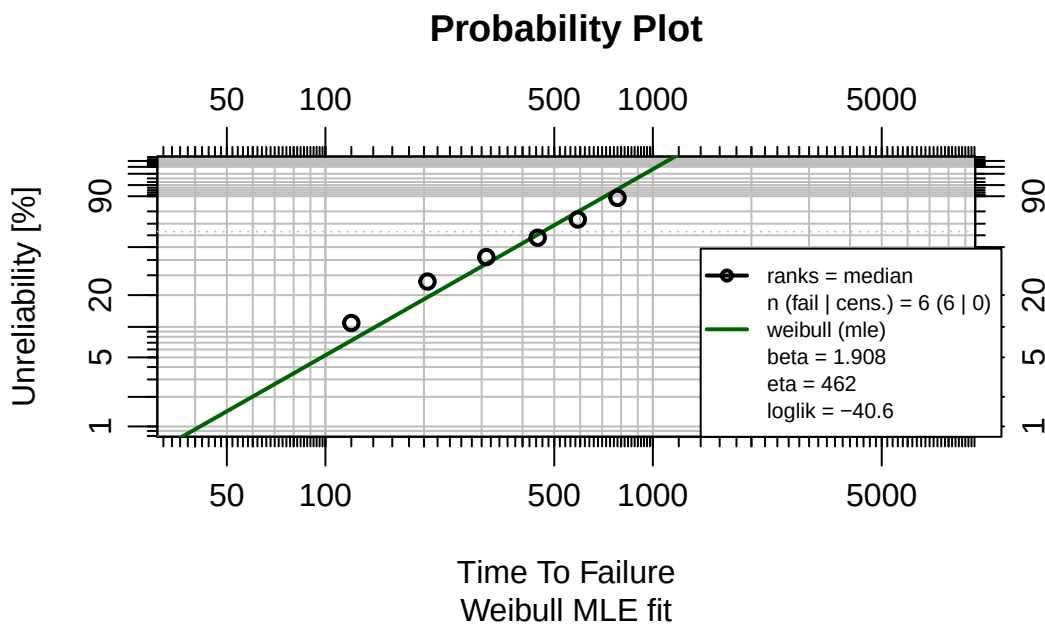
Try It

A batch of industrial pumps failed at times (hours): 120, 205, 310, 445, 590, 780. Use `MLEw2p()` to fit a 2-parameter Weibull model.

```
failures <- c(120, 205, 310, 445, 590, 780)
# fit <- MLEw2p(failures, show = TRUE)
```

Solution

```
failures <- c(120, 205, 310, 445, 590, 780)
fit <- MLEw2p(failures, show = TRUE)
```



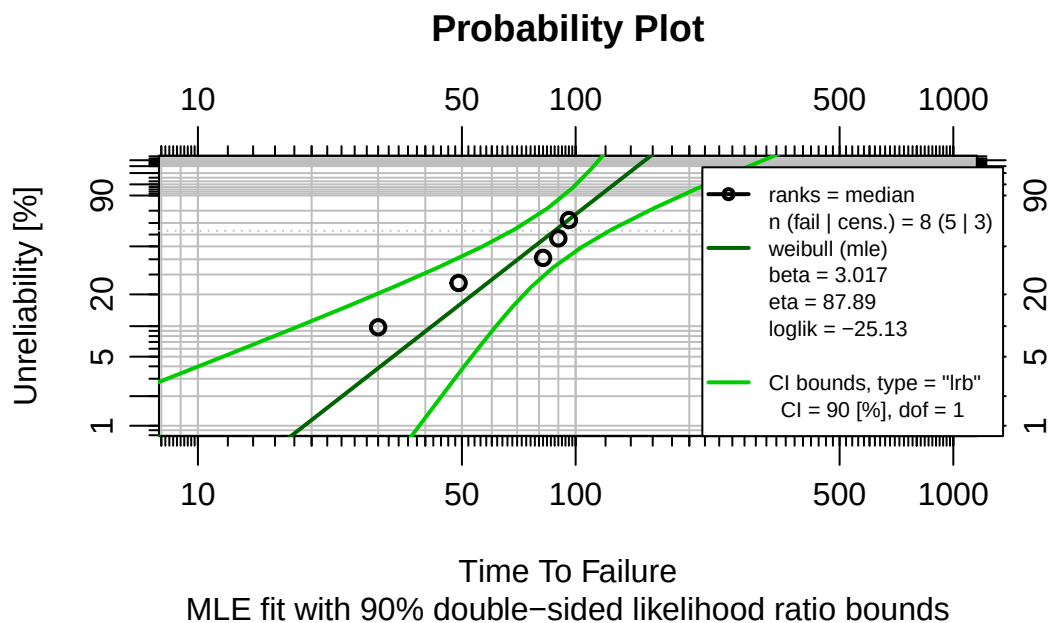
3.5 Data Censoring

In life data analysis, data is often **censored** — the exact time to failure is not known for all units.

3.5.1 Right-Censored Data (Suspensions)

Right censoring occurs when a unit has operated for a period of time without failing (a **suspension**). The unit was removed from service before failing.

```
failures    <- c(30, 49, 82, 90, 96)
suspensions <- c(100, 45, 10)
fit <- MLEw2p(failures, suspensions, bounds = TRUE, show = TRUE)
```



Adding suspensions changes the fit: the model accounts for units that survived longer, adjusting β and η accordingly.

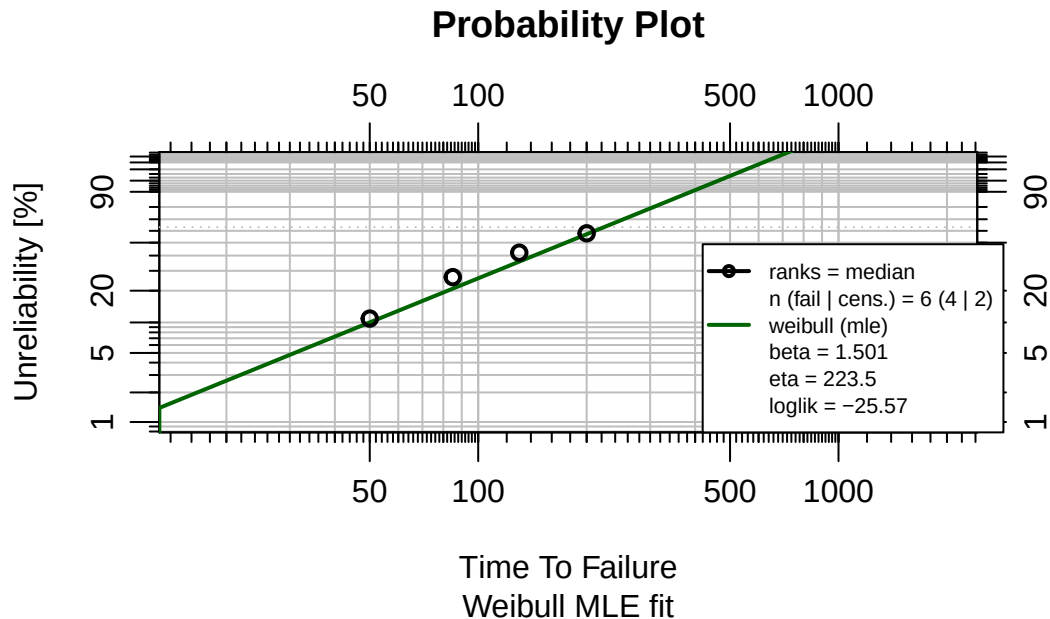
i Try It

Four compressors failed at 50, 85, 130, and 200 hours. Two additional units were removed at 250 hours without failing. Fit a Weibull model with suspensions.

```
failures    <- c(50, 85, 130, 200)
suspensions <- c(250, 250)
# fit <- MLEw2p(failures, suspensions, show = TRUE)
```

Solution

```
failures    <- c(50, 85, 130, 200)
suspensions <- c(250, 250)
fit <- MLEw2p(failures, suspensions, show = TRUE)
```

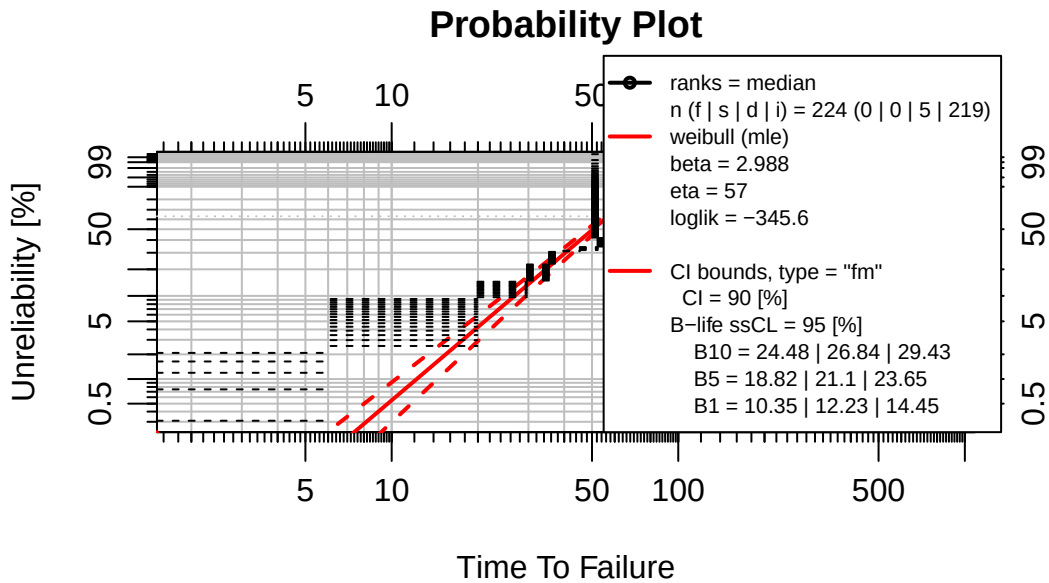


3.5.2 Interval-Censored Data

Interval censoring occurs when failures are only detected during periodic inspections — the exact failure time is unknown, but it is known to have occurred within an interval.

```
inspection_data <- data.frame(
  left  = c(0, 6.12, 19.92, 29.64, 35.4, 39.72, 45.32, 52.32),
  right = c(6.12, 19.92, 29.64, 35.4, 39.72, 45.32, 52.32, 63.48),
  qty   = c(5, 16, 12, 18, 18, 2, 6, 17)
)
suspensions <- data.frame(time = 63.48, event = 0, qty = 73)

obj1 <- wblr(suspensions, interval = inspection_data)
obj1 <- wblr.fit(obj1, method.fit = "mle", col = "red")
obj1 <- wblr.conf(obj1, method.conf = "fm", lty = 2)
plot(obj1)
```



Black horizontal lines represent the inspection intervals; the solid red line is the model fit; dashed red lines are confidence bounds.

3.5.3 Grouped Data

Life data is often collected in groups. The `qty` column in the inspection data above indicates how many units failed in each interval — 5 failed between time 0 and 6.12, 16 between 6.12 and 19.92, and so on.

Review

In the interval-censored example above, how many total failures occurred?

Answer

$5 + 16 + 12 + 18 + 18 + 2 + 6 + 17 = 94$ failures. The 73 suspensions were units that did not fail by the last inspection time (63.48). Total units in the dataset: $94 + 73 = 167$.

3.6 Parameter Estimation Methods

3.6.1 MRR vs MLE

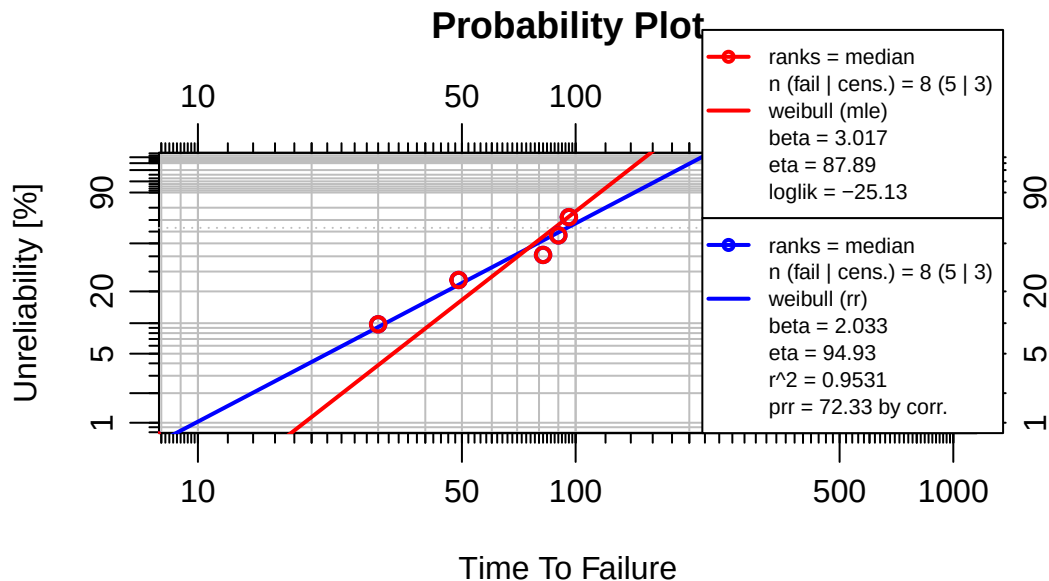
Median Rank Regression (MRR) estimates parameters by minimizing the sum of squared errors between observed and predicted values (least squares).

Maximum Likelihood Estimation (MLE) finds the parameters that make the observed data most probable — it maximizes the likelihood function.

Comparing both methods on the same data:

```
failures    <- c(30, 49, 82, 90, 96)
suspensions <- c(100, 45, 10)

MRRfit <- wblr.fit(wblr(failures, suspensions, col = "blue"), method.fit = "rr")
MLEfit  <- wblr.fit(wblr(failures, suspensions, col = "red"), method.fit = "mle")
plot.wblr(list(MRRfit, MLEfit))
```



Both methods fit the same data. Slight differences in β and η are visible — the better fit can be determined objectively using the Anderson-Darling statistic.

3.7 Goodness of Fit

The **Anderson-Darling (AD) statistic** measures how closely the fitted CDF matches the empirical distribution. A **lower AD** value indicates a better fit:

AD Value	Interpretation
< 0.3	Good fit
$0.3 - 0.6$	Acceptable fit
> 0.6	Poor fit

WeibullR displays the AD statistic in the plot legend. Use it to compare fitting methods or distributions.

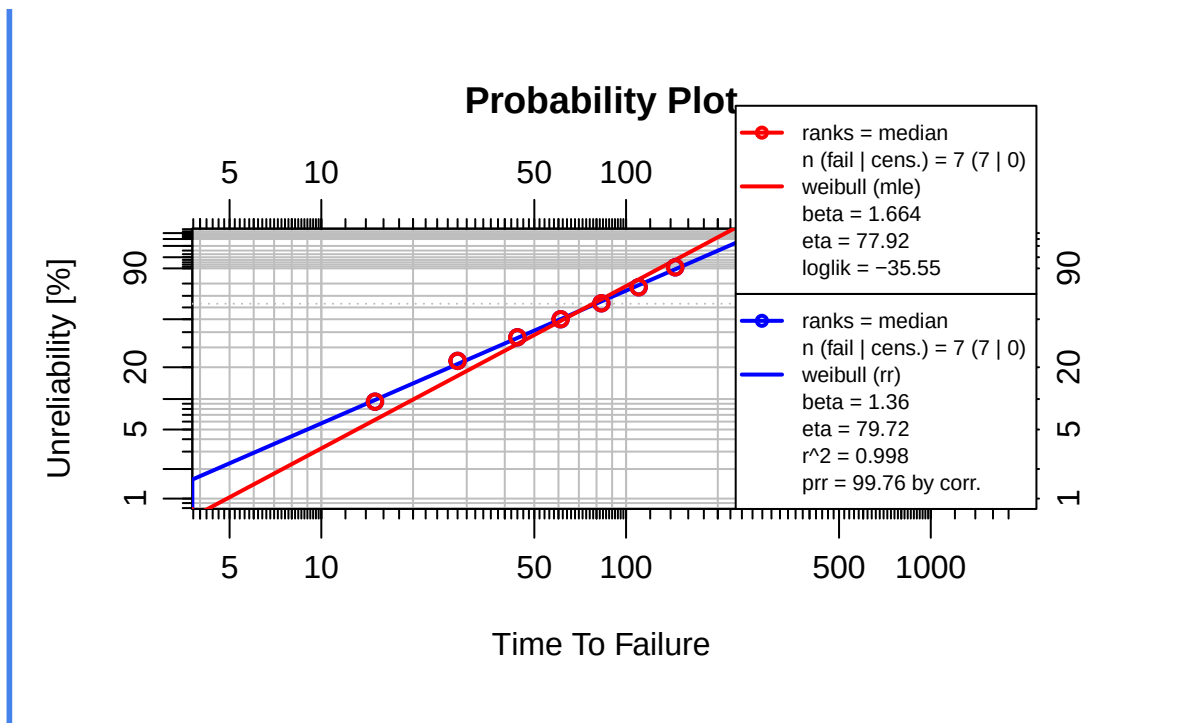
Try It

Fit the following failure data with both MRR and MLE, then compare their AD statistics on a combined probability plot.

```
failures <- c(15, 28, 44, 61, 83, 110, 145)
```

Solution

```
failures <- c(15, 28, 44, 61, 83, 110, 145)
obj_mrr <- wblr.fit(wblr(failures, col = "blue"), method.fit = "rr")
obj_mle <- wblr.fit(wblr(failures, col = "red"), method.fit = "mle")
plot.wblr(list(obj_mrr, obj_mle))
```



3.8 Choosing a Distribution

The Weibull distribution is the default in reliability engineering, but it is not always the best fit. The **lognormal** distribution often fits failure data driven by fatigue crack growth, corrosion, or cumulative damage — mechanisms where the logarithm of time-to-failure is normally distributed.

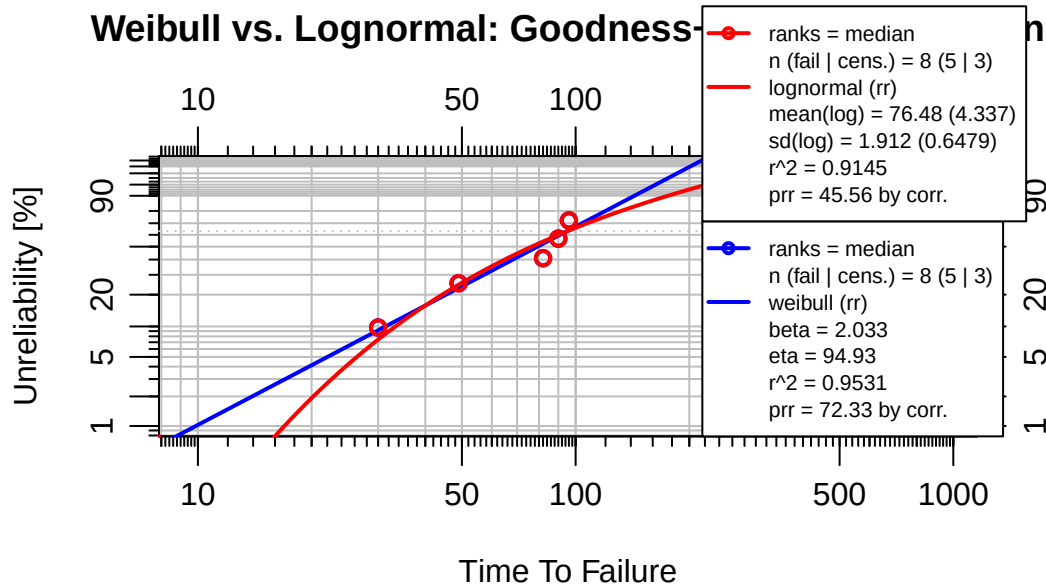
Distribution	When it tends to fit best
Weibull	Wear-out, mechanical fatigue, early-life failures
Lognormal	Corrosion, fatigue crack propagation, electronic degradation

`wblr.fit()` supports both via the `dist` argument. The goodness-of-fit metric in the plot legend (r^2 for rank regression) is the tiebreaker — **higher r^2 indicates a better fit**:

```
failures    <- c(30, 49, 82, 90, 96)
suspensions <- c(100, 45, 10)
```

```
obj_wb <- wblr.fit(wblr(failures, suspensions, col = "blue"),
  method.fit = "rr")
obj_ln <- wblr.fit(wblr(failures, suspensions, col = "red"),
  method.fit = "rr", dist = "lognormal")

plot.wblr(list(obj_wb, obj_ln),
  main = "Weibull vs. Lognormal: Goodness-of-Fit Comparison")
```



For this dataset, Weibull achieves $r^2 \approx 0.953$ vs. lognormal at $r^2 \approx 0.914$ — Weibull fits better. The choice should be driven by the data, not by habit.

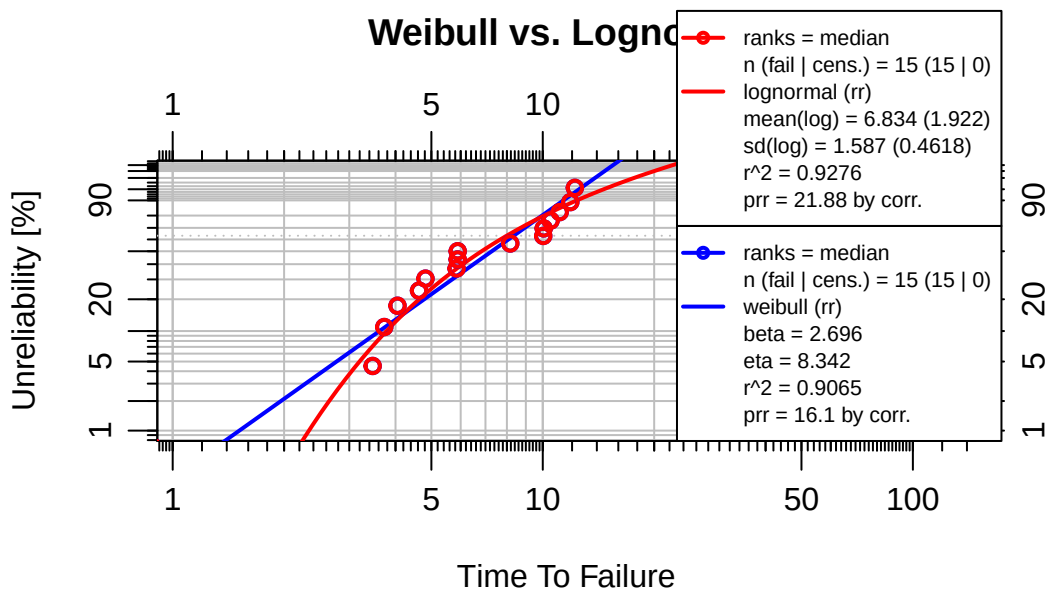
i Try It

Fit the 3-parameter Weibull failure dataset below with both Weibull and lognormal distributions. Which fits better?

```
failures_3p <- c(3.47, 3.73, 4.05, 4.63, 4.82, 5.85, 5.89, 5.89,
  8.17, 10.03, 10.06, 10.50, 11.11, 11.87, 12.21)
# obj_wb <- wblr.fit(wblr(failures_3p, col = "blue"), method.fit = "rr")
# obj_ln <- wblr.fit(wblr(failures_3p, col = "red"), method.fit = "rr", dist = "lognormal")
# plot.wblr(list(obj_wb, obj_ln))
```

Solution


```
failures_3p <- c(3.47, 3.73, 4.05, 4.63, 4.82, 5.85, 5.89, 5.89,
                 8.17, 10.03, 10.06, 10.50, 11.11, 11.87, 12.21)
obj_wb <- wblr.fit(wblr(failures_3p, col = "blue"), method.fit = "rr")
obj_ln <- wblr.fit(wblr(failures_3p, col = "red"), method.fit = "rr", dist = "lognormal")
plot.wblr(list(obj_wb, obj_ln),
           main = "Weibull vs. Lognormal")
```



Compare the r^2 values in the legend. For this dataset the lognormal often achieves a slightly higher r^2 , suggesting cumulative-damage behavior — the logarithm of time-to-failure is approximately normally distributed.

3.9 Sample Size Considerations

Weibull parameter estimates are most reliable when based on at least 10–15 **observed failures**. With fewer failures, uncertainty grows substantially: confidence bounds widen, r^2 loses discriminating power, and estimated β and η can deviate significantly from the true values.

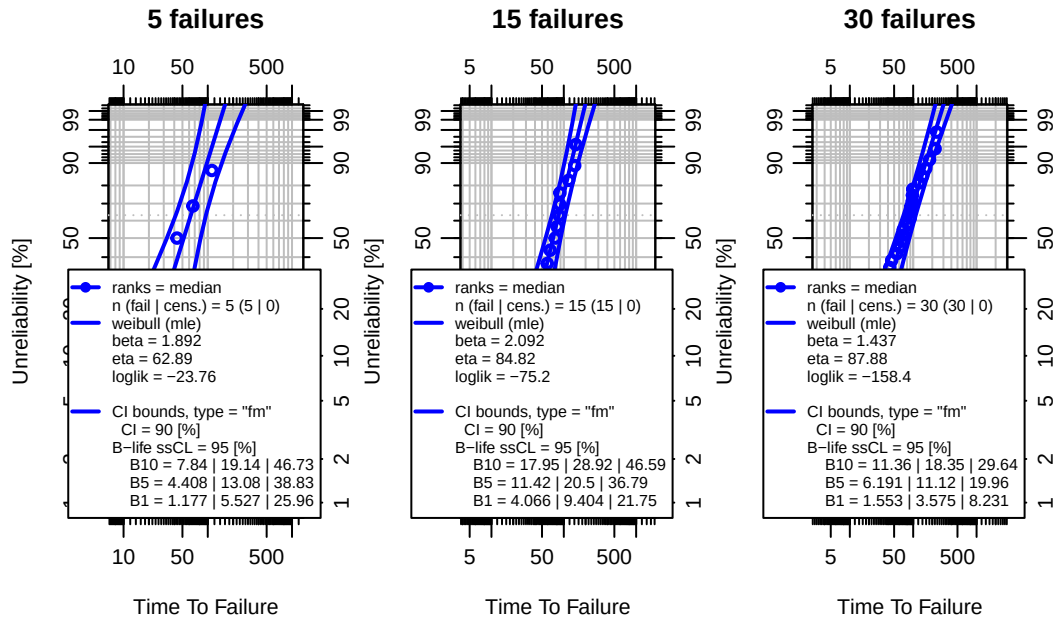
The following code simulates datasets of different sizes from the same Weibull ($\beta = 2$, $\eta = 100$) and fits each — showing how confidence bounds tighten as sample size increases:

```
set.seed(42)
eta_true <- 100; beta_true <- 2
```

```

par(mfrow = c(1, 3))
for (n in c(5, 15, 30)) {
  f <- sort(rweibull(n, shape = beta_true, scale = eta_true))
  obj <- wblr.conf(wblr.fit(wblr(f, col = "blue"), method.fit = "mle"),
                  method.conf = "fm")
  plot(obj, main = paste0(n, " failures"))
}

```



```

par(mfrow = c(1, 1))

```

With only 5 failures the confidence bounds span most of the probability axis. At 30 failures they are narrow enough for confident decision-making.

Practical guidance:

Failures observed	Guidance
< 5	Estimates unreliable; use Weibayes (fix β from prior knowledge)
5–10	Usable with caution; widen safety margins
10–20	Acceptable for most engineering decisions

Failures observed	Guidance
> 20	Reliable estimates; confidence bounds are tight

When fewer than 5 failures are available, the Weibayes model (next section) provides a structured way to incorporate prior knowledge about β .

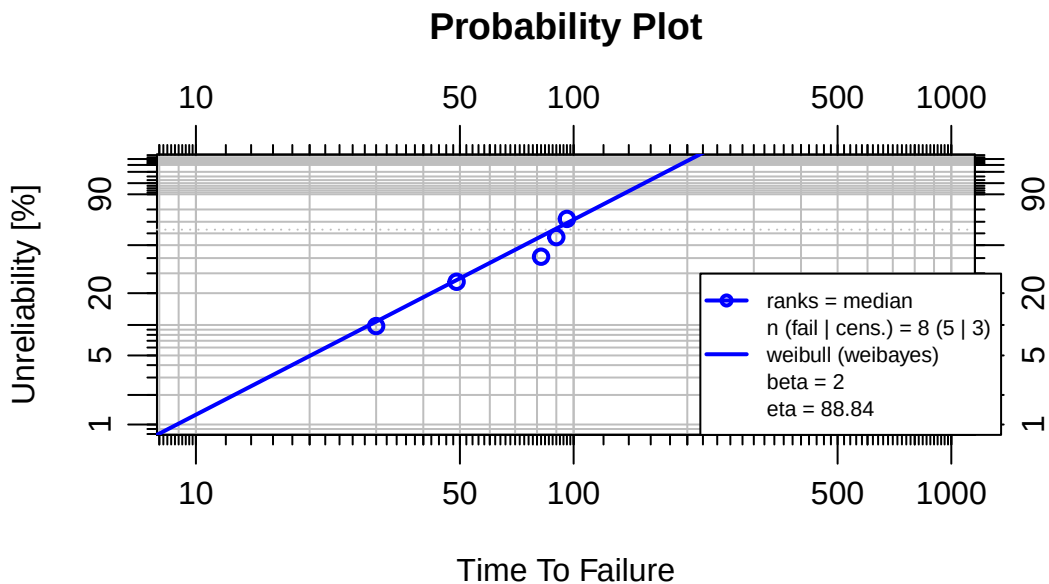
3.10 Other Weibull Models

3.10.1 The Weibayes Model

A **Weibayes** (one-parameter Weibull) has a fixed β based on prior knowledge or experience. This is appropriate when failure data are scarce.

```
failures    <- c(30, 49, 82, 90, 96)
suspensions <- c(100, 45, 10)

obj <- wblr.fit(wblr(failures, suspensions, col = "blue"),
               method.fit = "weibayes", weibayes.beta = 2)
plot(obj)
```



Confidence bounds are not shown for Weibayes because β is fixed by assumption.

3.10.2 The 3-Parameter Weibull

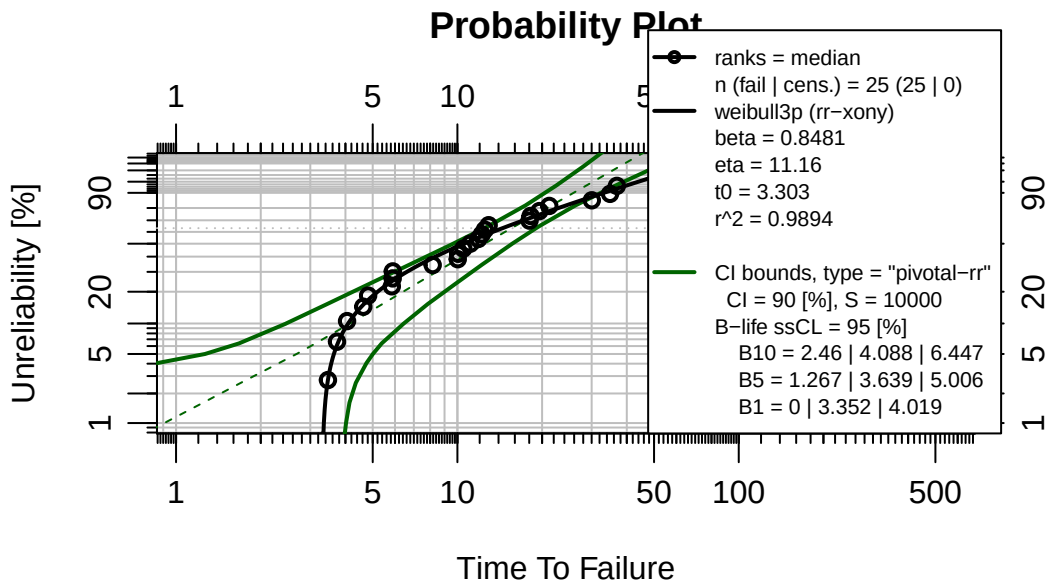
The **3-parameter Weibull** adds a failure-free period t_0 — the time before which no failures can occur:

$$R(t) = e^{-((t-t_0)/\eta)^\beta}$$

This is appropriate for components that must age, wear, or fatigue before they can fail.

```
failures_3p <- c(3.46623, 3.732711, 4.052996, 4.628703, 4.8157, 5.84517, 5.888313, 5.892967,
8.168362, 10.02799, 10.06062, 10.49785, 11.11493, 11.87369, 12.21122, 12.51122,
12.91357, 18.04246, 18.20712, 19.57305, 21.20873, 30.03917, 34.88001, 36.87305,
53.91168)

fit_3p <- wblr.conf(wblr.fit(wblr(failures_3p), dist = "weibull3p"), col = "darkgreen")
plot(fit_3p)
```



The failure-free period is approximately 3.3 time units — visible as the curve flattening at low probabilities.

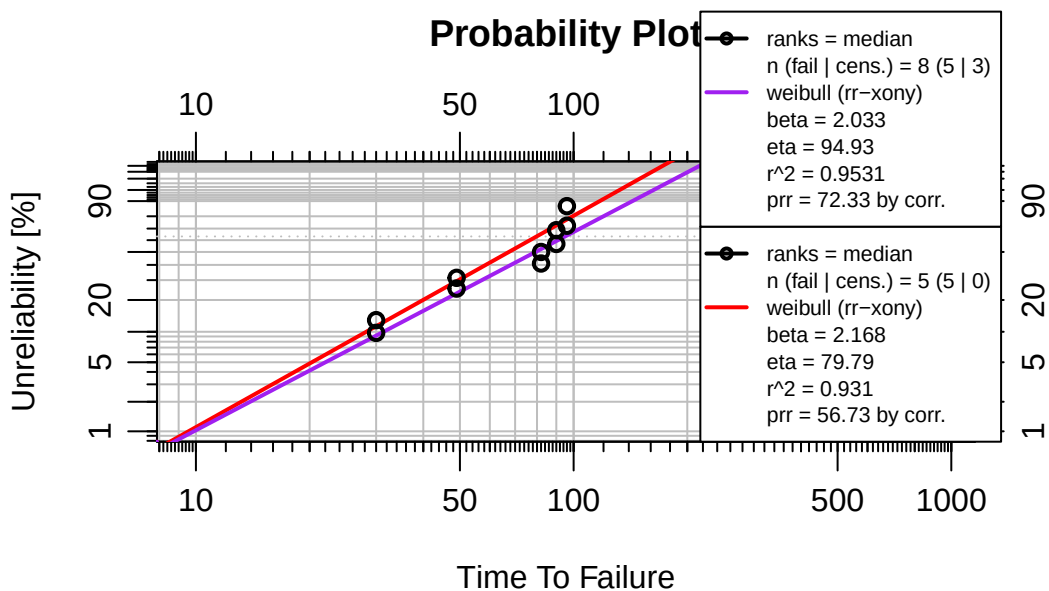
3.11 Multi-Plots

Multi-plots overlay multiple Weibull fits on one chart, making comparisons easy.

```
failures <- c(30, 49, 82, 90, 96)
suspensions <- c(100, 45, 10)

obj1 <- wblr.fit(wblr(failures), col = "red")
obj2 <- wblr.fit(wblr(failures, suspensions), col = "purple")

plot.wblr(list(obj1, obj2))
```



The red model (without suspensions) has a higher beta and lower eta than the purple model (with suspensions). Adding suspensions shifts the fitted distribution to account for units that survived longer than the observed failures.

3.12 Competing Failure Modes

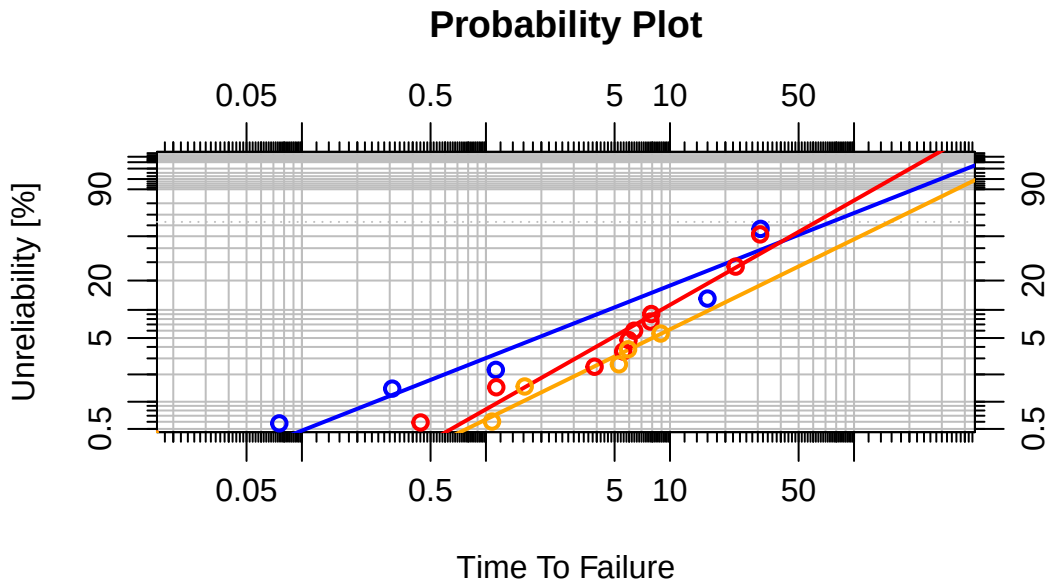
When a component can fail in several distinct ways (corrosion, fatigue, overload), fitting a single Weibull to the combined data produces a poor fit because each mode has its own distribution.

The approach: fit a separate Weibull to each failure mode, treating failures from other modes as suspensions.

```
set.seed(123)
data <- data.frame(
  time = c(
    rweibull(5, 0.5, 20), # Failure Mode A
    rweibull(10, 1, 10), # Failure Mode B
    rweibull(5, 2, 5), # Failure Mode C
    rweibull(100, 2, 10) # Suspensions
  ),
  event = c(rep(1, 20), rep(0, 100)),
  failure_mode = c(rep("A", 5), rep("B", 10), rep("C", 5), rep("", 100))
)

# Separate dataset per failure mode (others treated as suspensions)
dat1 <- dat2 <- dat3 <- data
dat1$event[dat1$failure_mode != "A"] <- 0
dat2$event[dat2$failure_mode != "B"] <- 0
dat3$event[dat3$failure_mode != "C"] <- 0

obj1 <- wblr.fit(wblr(dat1, col = "blue"))
obj2 <- wblr.fit(wblr(dat2, col = "red"))
obj3 <- wblr.fit(wblr(dat3, col = "orange"))
plot.wblr(list(obj1, obj2, obj3), is.plot.legend = FALSE)
```

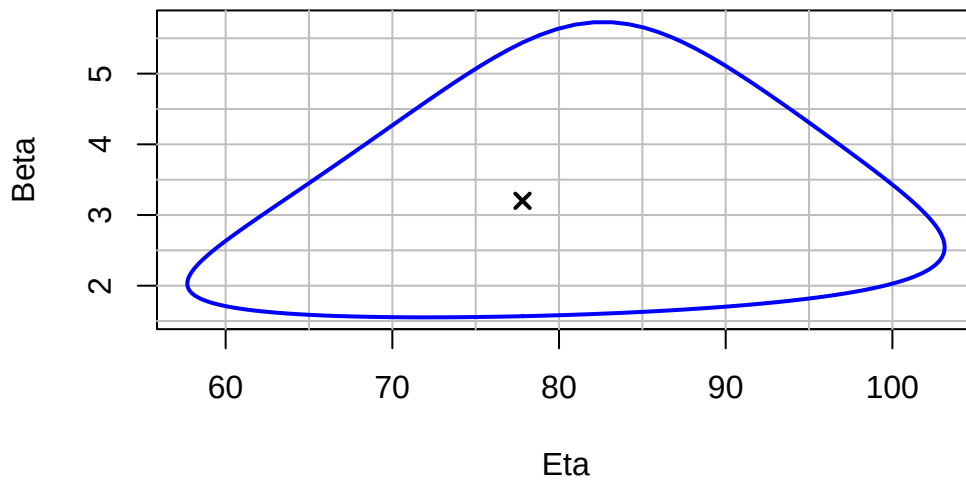


Each failure mode has a distinct β : Mode A ($\beta < 1$, decreasing rate — infant mortality), Mode B ($\beta \approx 1$, random), Mode C ($\beta > 1$, wear-out).

3.13 Contour Plots

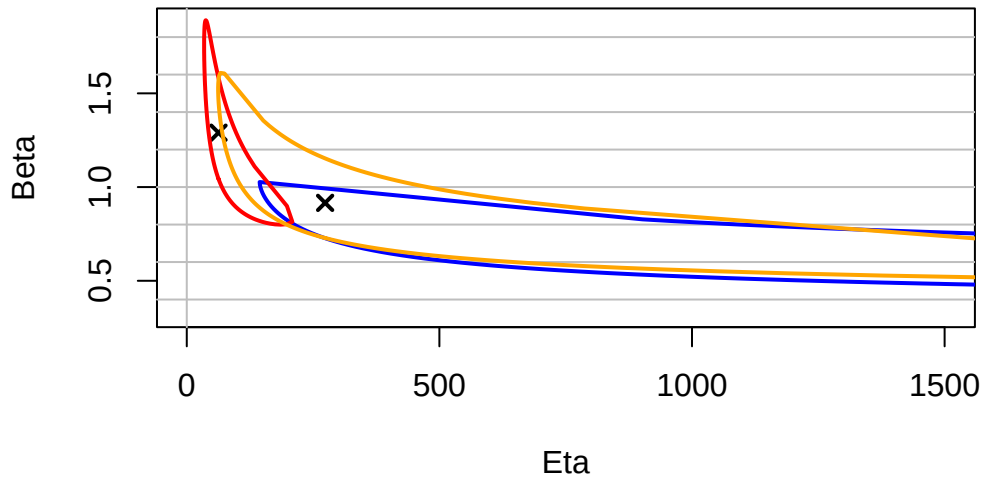
A **contour plot** shows the joint confidence region for β and η . Overlapping contours indicate that two distributions are not significantly different at the chosen confidence level.

```
failures <- c(30, 49, 82, 90, 96)
obj <- wblr.conf(wblr.fit(wblr(failures, col = "blue"), method.fit = "mle"), method.conf = "mle")
plot_contour(obj, CL = 0.9)
```



3.13.1 Comparing Contour Plots

```
obj1 <- wblr.conf(wblr.fit(wblr(dat1, col = "blue"), method.fit = "mle"), method.conf = "l")
obj2 <- wblr.conf(wblr.fit(wblr(dat2, col = "red"), method.fit = "mle"), method.conf = "l")
obj3 <- wblr.conf(wblr.fit(wblr(dat3, col = "orange"), method.fit = "mle"), method.conf = "l")
plot_contour(list(obj1, obj2, obj3), CL = 0.9, xlim = c(1, 1500))
```

Overlapping contours at 90% confidence level indicate that the three failure modes are not statistically distinguishable at this confidence level.

3.14 Summary

Key takeaways:

- The Weibull CDF: $R(t) = e^{-(t/\eta)^\beta}$.
- $\beta < 1$: decreasing rate (infant mortality); $\beta = 1$: constant (exponential); $\beta > 1$: increasing (wear-out).
- At $t = \eta$: $F(\eta) = 63.2\%$ for any β .
- Suspensions are units that did not fail — always include them in the analysis.
- MRR and MLE often give slightly different fits — use the AD statistic to choose.
- Weibayes is appropriate for scarce data with prior knowledge of β .
- 3-parameter Weibull handles failure-free periods.
- Multi-plots and contour plots are powerful tools for comparing distributions.

4 Reliability Growth Analysis

4.1 Introduction

Reliability Growth Analysis (RGA) monitors and improves reliability over time by analyzing failure data collected during testing. It identifies trends in the failure rate, confirming whether corrective actions are having the intended effect.

4.2 Learning Objectives

By the end of this chapter, you will be able to:

- Define key reliability growth concepts: Crow-AMSAA and Duane models.
- Use `rdt()` to plan a reliability demonstration test (required test time and sample size).
- Fit reliability growth models using R.
- Apply the Crow-AMSAA model to assess reliability growth.
- Use piecewise NHPP models to detect design changes during testing.
- Forecast future failures from a fitted growth model using `predict_rga()`.
- Interpret reliability growth plots and make decisions from the results.

4.3 Reliability Growth Analysis

The `ReliaGrowR` package (Govan 2024) provides functions for reliability growth analysis in R.

```
library(ReliaGrowR)
```

4.4 Test Planning

Before analyzing test data, it helps to answer two design questions: *How long must we test?* and *How many failures should we expect?* `ReliaGrowR` provides tools for both.

4.4.1 Reliability Demonstration Tests — `rdt()`

A **Reliability Demonstration Test (RDT)** is a pass/fail test designed to demonstrate that a product meets a reliability target at a specified confidence level. `rdt()` calculates the required test time given the sample size (or vice versa).

```
# How long must we test 10 units to demonstrate R(1000 hrs) >= 0.90
# at 80% confidence? Assume Weibull shape beta = 2, zero allowable failures.
plan <- rdt(
  target      = 0.90,    # target reliability
  mission_time = 1000,   # mission time (hours)
  conf_level   = 0.80,   # confidence level
  beta        = 2,       # assumed Weibull shape parameter
  n           = 10,      # number of test units
  f           = 0        # allowable failures
)
print(plan)
```

Reliability Demonstration Test (RDT) Plan

Distribution: Weibull
Weibull Shape Parameter (Beta): 2
Allowed Failures (f): 0
Target Reliability: 0.9
Mission Time: 1000
Input Sample Size (n): 10
Required Test Time (T): 1235.94

Each unit must be tested for approximately 1,236 hours without failure. If the test passes, we have demonstrated $R(1000) \geq 0.90$ with 80% confidence.

Try It

Compare the required test time for sample sizes of 5, 10, and 20 units (keep all other parameters the same). What is the trade-off?

```
# Vary n to see the effect on required test time
for (n in c(5, 10, 20)) {
  # plan <- rdt(target = 0.90, mission_time = 1000, conf_level = 0.80,
  #             beta = 2, n = n, f = 0)
  # print(plan)
}
```

Solution

```
for (n in c(5, 10, 20)) {  
  plan <- rdt(target = 0.90, mission_time = 1000, conf_level = 0.80,  
             beta = 2, n = n, f = 0)  
  print(plan)  
}
```

Reliability Demonstration Test (RDT) Plan

Distribution: Weibull
Weibull Shape Parameter (Beta): 2
Allowed Failures (f): 0
Target Reliability: 0.9
Mission Time: 1000
Input Sample Size (n): 5
Required Test Time (T): 1747.89
Reliability Demonstration Test (RDT) Plan

Distribution: Weibull
Weibull Shape Parameter (Beta): 2
Allowed Failures (f): 0
Target Reliability: 0.9
Mission Time: 1000
Input Sample Size (n): 10
Required Test Time (T): 1235.94
Reliability Demonstration Test (RDT) Plan

Distribution: Weibull
Weibull Shape Parameter (Beta): 2
Allowed Failures (f): 0
Target Reliability: 0.9
Mission Time: 1000
Input Sample Size (n): 20
Required Test Time (T): 873.94

More test units means shorter required test time per unit — but total test effort ($n \times T$) stays roughly constant. Larger fleets reduce calendar time at the cost of more physical units.

4.4.2 Forecasting Growth — predict_rga()

Once a growth test is underway, `predict_rga()` projects cumulative failures beyond the current endpoint — useful for deciding whether to continue testing or whether a reliability target will be met by a program deadline.

```
times    <- c(100, 200, 300, 400, 500)
failures <- c(1, 2, 1, 3, 2)

fit <- rga(times, failures)
pred <- predict_rga(fit, times = seq(500, 1000, by = 100))
print(pred)
```

Reliability Growth Forecast (Crow-AMSAA)

Time	Cum.Failures	Lower (95%)	Upper (95%)
500	3.9	3.2	4.6
600	4.5	3.7	5.3
700	5.0	4.2	6.1
800	5.6	4.6	6.8
900	6.2	5.0	7.6
1000	6.7	5.4	8.3

The lower and upper bounds reflect the uncertainty in the Crow-AMSAA parameter estimates. A narrow interval indicates the growth trend is well characterized; a wide interval suggests more test time is needed.

Review

The fitted Crow-AMSAA model gives $\hat{\beta} = 0.72$. Is the system still improving at time 1,000?

Answer

Yes — $\beta < 1$ means the cumulative failure rate is **decreasing** (the growth curve is concave), so reliability is still improving at time 1,000. Testing should continue until β stabilizes near the growth target.

4.5 The Duane Model

The **Duane Model** (Duane 1964) is one of the earliest and most widely used models for reliability growth. It is a log-log plot of cumulative MTBF vs cumulative time, where the MTBF is the total operating time divided by the number of failures up to that time.

$$\text{CMTBF}(t) = K \cdot t^{\beta-1}$$

The slope of the line on the log-log plot indicates the rate of reliability growth:

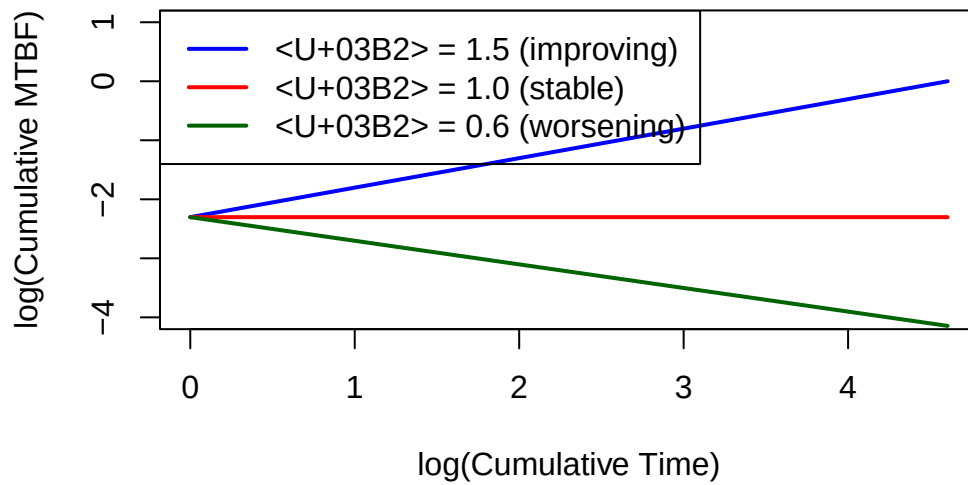
Slope (β)	Meaning
> 1	Reliability improving (failure rate decreasing)
$= 1$	No change (stable)
< 1	Reliability worsening (failure rate increasing)

Plotting the Duane model for three beta values:

```
t <- seq(1, 100, by = 0.1)

# Three scenarios
plot(log(t), log(0.1 * t^(1.5 - 1)), type = "l", col = "blue", lwd = 2,
     xlab = "log(Cumulative Time)", ylab = "log(Cumulative MTBF)",
     main = "Duane Plot: Three Reliability Growth Scenarios",
     ylim = c(-4, 1))
lines(log(t), log(0.1 * t^(1.0 - 1)), col = "red", lwd = 2)
lines(log(t), log(0.1 * t^(0.6 - 1)), col = "darkgreen", lwd = 2)
legend("topleft",
     legend = c(" = 1.5 (improving)", " = 1.0 (stable)", " = 0.6 (worsening)"),
     col = c("blue", "red", "darkgreen"), lwd = 2)
```

Duane Plot: Three Reliability Growth Scenarios

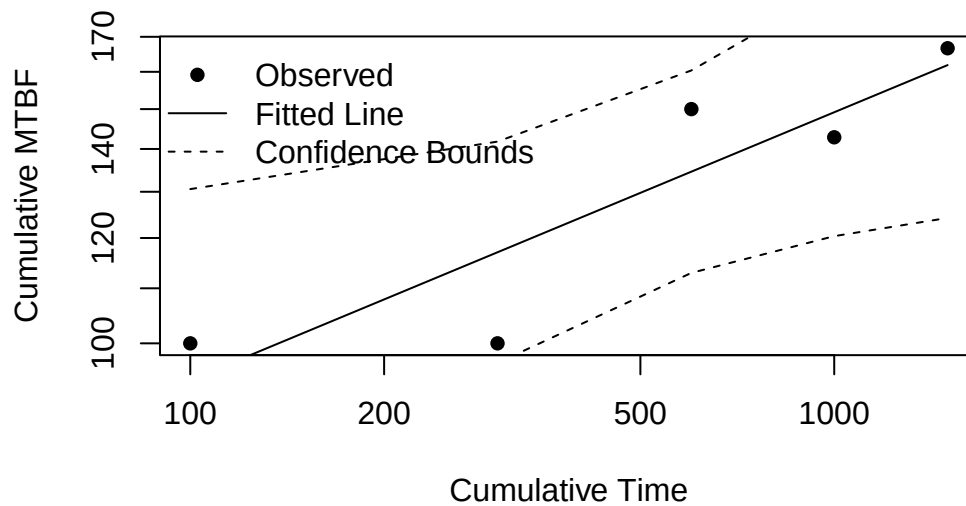


4.5.1 Fitting with ReliaGrowR

```
times    <- c(100, 200, 300, 400, 500)
failures <- c(1, 2, 1, 3, 2)

fit <- duane(times, failures)
plot(fit, main = "Duane Model Example",
     xlab = "Cumulative Time", ylab = "Cumulative MTBF")
```

Duane Model Example



Try It

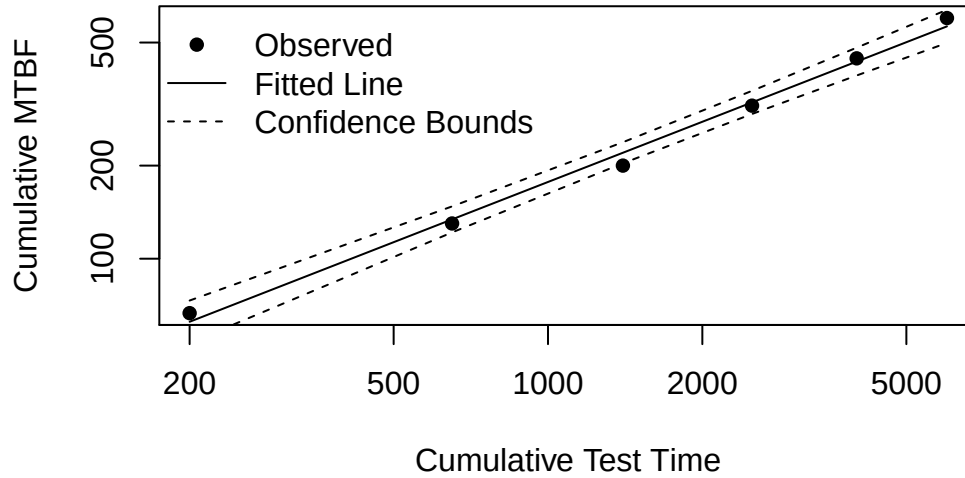
A new system was tested and the following cumulative failure counts were recorded. Fit a Duane model and plot the result.

```
times    <- c(200, 450, 750, 1100, 1500, 2000)
failures  <- c(3, 2, 2, 1, 1, 1)
# fit <- duane(times, failures)
# plot(fit)
```

Solution

```
times    <- c(200, 450, 750, 1100, 1500, 2000)
failures  <- c(3, 2, 2, 1, 1, 1)
fit <- duane(times, failures)
plot(fit, main = "Duane Reliability Growth Plot",
     xlab = "Cumulative Test Time", ylab = "Cumulative MTBF")
```


Duane Reliability Growth Plot



Review

What does a Duane plot with a slope greater than 1 indicate?

Answer

The system's reliability is **improving** — the failure rate is decreasing over time.

4.6 The Crow-AMSAA Model

The **Crow-AMSAA Model** (Crow 1975) models cumulative failures vs cumulative time using a Non-Homogeneous Poisson Process (NHPP):

$$N(t) = \lambda_0 \cdot t^\beta$$

The shape parameter β interpretation:

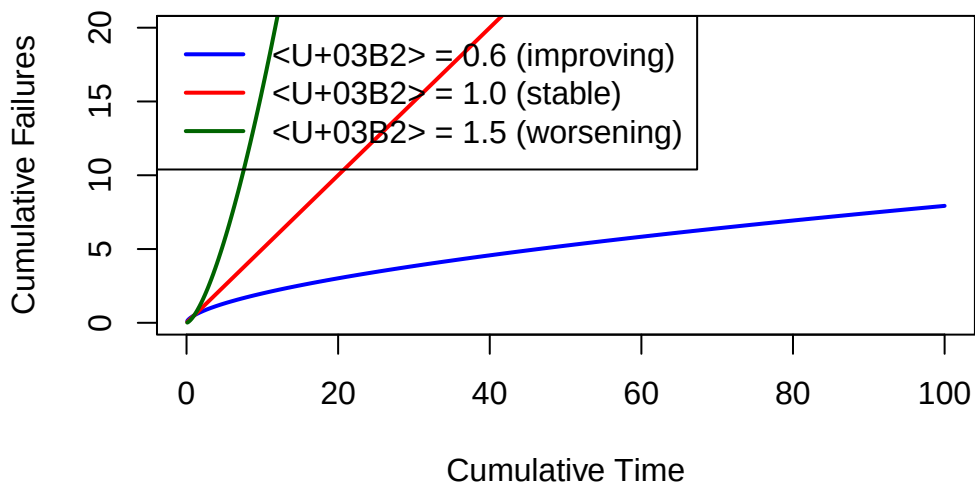
β	Meaning
> 1	Failures increasing (reliability worsening)
$= 1$	Constant rate (stable)
< 1	Failures decreasing (reliability improving)

Note: The Crow-AMSAA β interpretation is **reversed** compared to the Duane model — because the Duane plots MTBF (higher is better), while Crow-AMSAA plots cumulative failures (lower slope is better).

```
t <- seq(0.1, 100, by = 0.1)

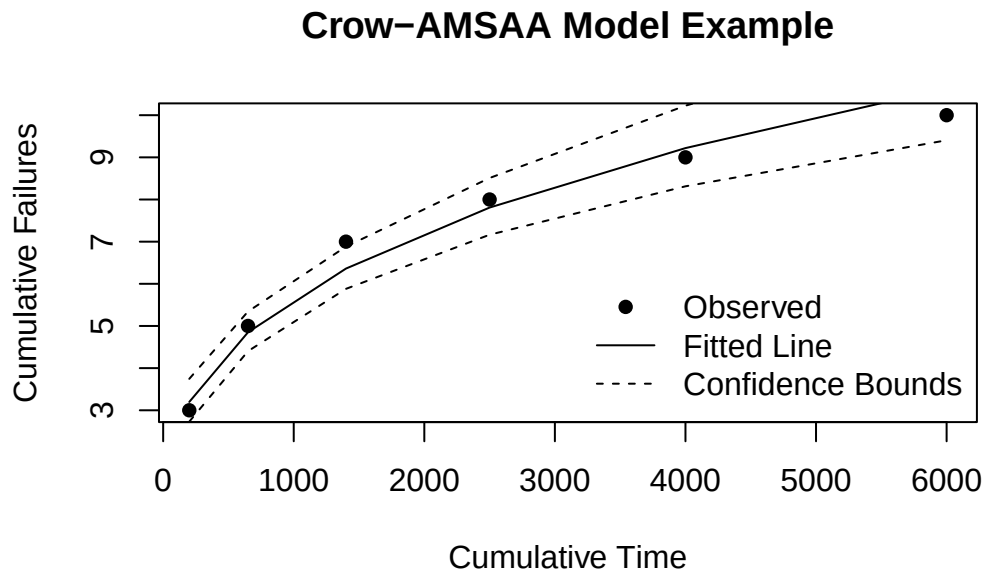
# Three beta scenarios for cumulative failures
plot(t, 0.5 * t^0.6, type = "l", col = "blue", lwd = 2,
      xlab = "Cumulative Time", ylab = "Cumulative Failures",
      main = "Crow-AMSAA: Three Reliability Scenarios",
      ylim = c(0, 20))
lines(t, 0.5 * t^1.0, col = "red", lwd = 2)
lines(t, 0.5 * t^1.5, col = "darkgreen", lwd = 2)
legend("topleft",
      legend = c(" = 0.6 (improving)", " = 1.0 (stable)", " = 1.5 (worsening)"),
      col = c("blue", "red", "darkgreen"), lwd = 2)
```

Crow-AMSAA: Three Reliability Scenarios



4.6.1 Fitting with ReliaGrowR

```
result <- rga(times, failures)
plot(result, main = "Crow-AMSAA Model Example",
      xlab = "Cumulative Time", ylab = "Cumulative Failures")
```



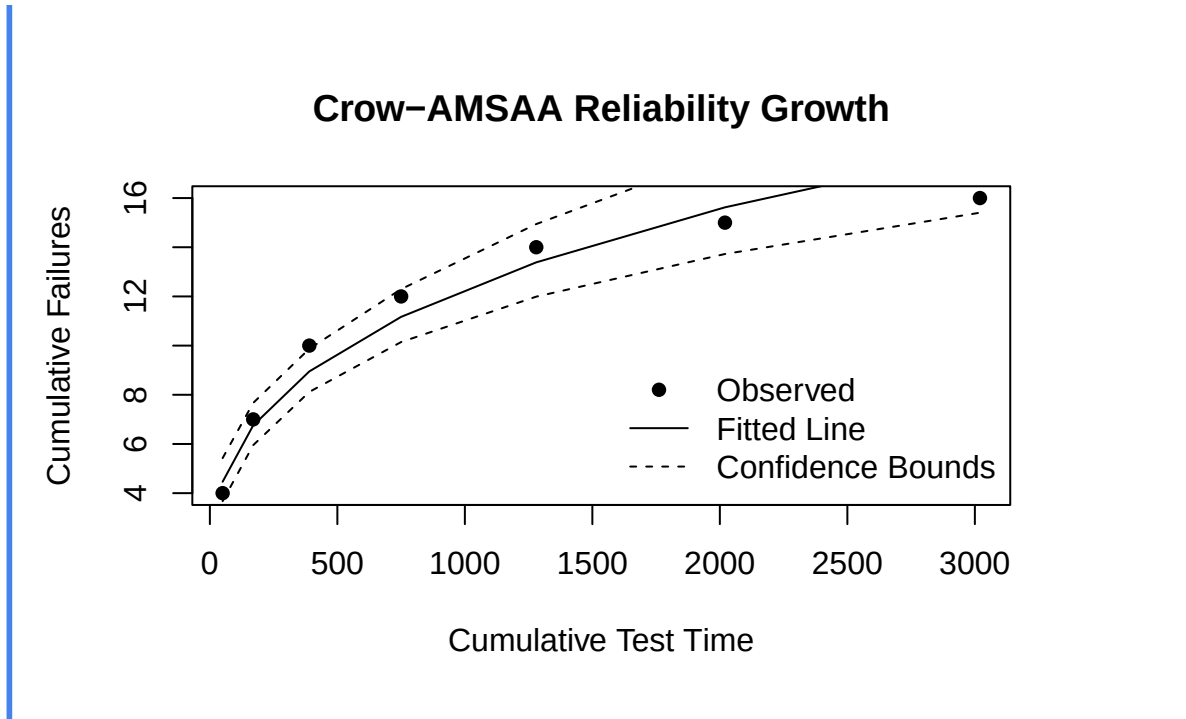
Try It

A development test recorded these cumulative failure counts. Use `rga()` with `model_type = "Crow-AMSAA"` to fit the model.

```
times    <- c(50, 120, 220, 360, 530, 740, 1000)
failures <- c(4, 3, 3, 2, 2, 1, 1)
# fit <- rga(times, failures, model_type = "Crow-AMSAA")
# plot(fit)
```

Solution

```
times    <- c(50, 120, 220, 360, 530, 740, 1000)
failures <- c(4, 3, 3, 2, 2, 1, 1)
fit <- rga(times, failures, model_type = "Crow-AMSAA")
plot(fit, main = "Crow-AMSAA Reliability Growth",
     xlab = "Cumulative Test Time", ylab = "Cumulative Failures")
```



4.6.2 The Piecewise NHPP Model

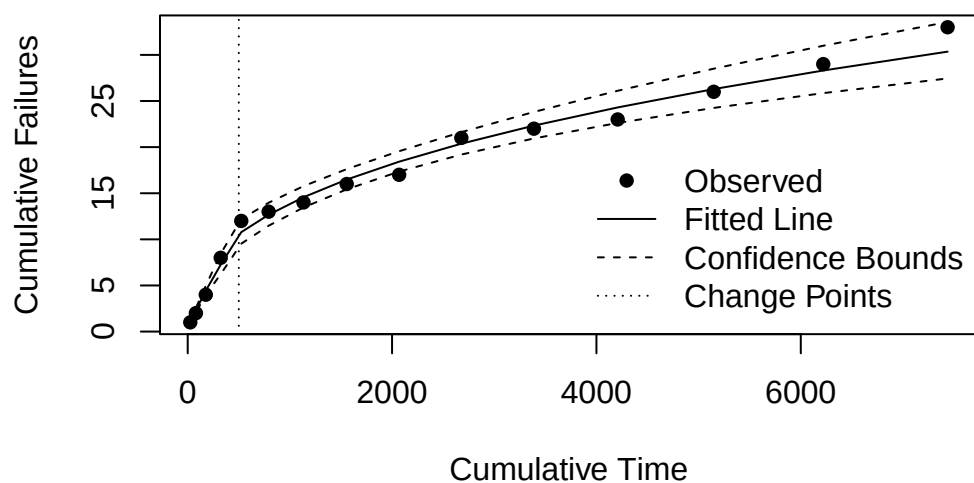
The **Piecewise NHPP** fits a separate Power Law to each segment of the time axis, separated by breakpoints. This is useful when a design change occurs during testing.

$$N_i(t) = N(t_{i-1}) + \lambda_i(t - t_{i-1})^{\beta_i}, \quad t_{i-1} < t \leq t_i$$

```
times    <- c(25, 55, 97, 146, 201, 268, 341, 423, 513, 609, 710, 820, 940, 1072, 1217)
failures <- c(1, 1, 2, 4, 4, 1, 1, 2, 1, 4, 1, 1, 3, 3, 4)
breaks   <- 500

result <- rga(times, failures, model_type = "Piecewise NHPP", breaks = breaks)
plot(result, main = "Piecewise NHPP Model",
      xlab = "Cumulative Time", ylab = "Cumulative Failures")
```

Piecewise NHPP Model



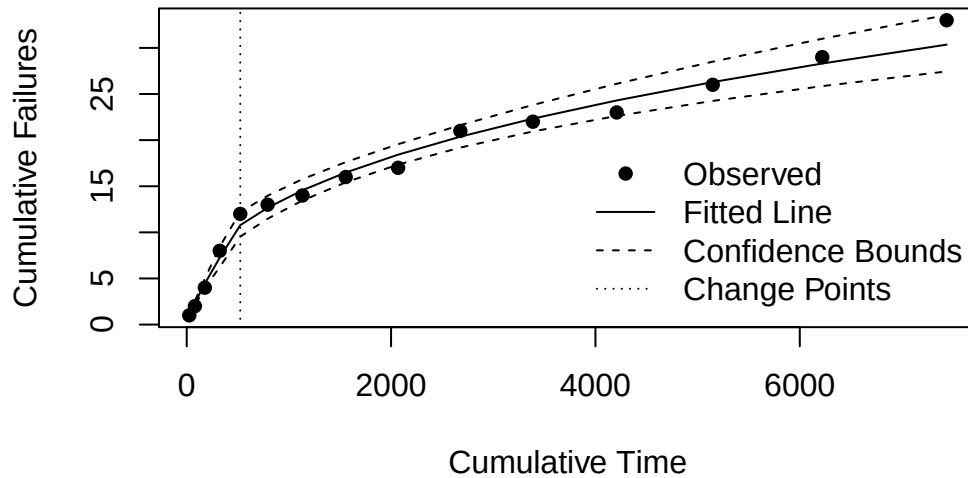
The two segments have different slopes — the first segment (pre-change) has a higher slope than the second (post-change), confirming the design change reduced the failure rate.

4.6.3 Change Point Detection

Without a known breakpoint, `rga()` can detect the change point automatically:

```
result_auto <- rga(times, failures, model_type = "Piecewise NHPP")
plot(result_auto, main = "Piecewise NHPP with Change Point Detection",
      xlab = "Cumulative Time", ylab = "Cumulative Failures")
```

Piecewise NHPP with Change Point Detection



The automatically detected change point is near time 500, consistent with the known design change.

4.7 Summary

Key takeaways:

- **Duane model:** log-log of MTBF vs time; positive slope ($\beta > 1$) = improving.
- **Crow-AMSAA:** log of cumulative failures vs time; concave downward = improving ($\beta < 1$).
- **Piecewise NHPP:** fits multiple segments; use when a known change occurs during testing.
- For accelerated life testing (Arrhenius, Power Law, relationship plots), see Chapter 5.

5 Accelerated Life Testing

5.1 Introduction

Accelerated Life Testing (ALT) (Meeker and Escobar 1998; Nelson 1990) subjects products to elevated stress levels — temperature, voltage, mechanical load, humidity — to induce failures more quickly than would occur under normal operating conditions. The acceleration relationship is then used to translate accelerated-test results into normal-use life predictions.

This approach is essential when products are designed for long service lives (years or decades), making it impractical to wait for failures at use conditions. ALT compresses that time by running tests at higher stress, then mathematically projecting back to the use environment.

5.2 Learning Objectives

By the end of this chapter, you will be able to:

- Explain the purpose of accelerated life testing and the role of the Acceleration Factor.
- Apply the Arrhenius model for temperature-accelerated tests.
- Apply the Power Law model for non-thermal stress acceleration.
- Fit ALT models using the `WeibullR.ALT` package.
- Interpret probability plots and relationship plots from ALT analyses.
- Use parallelization to constrain the shape parameter across stress levels.

5.3 The Acceleration Factor

The **Acceleration Factor (AF)** quantifies how much faster a product fails under elevated stress compared to use conditions:

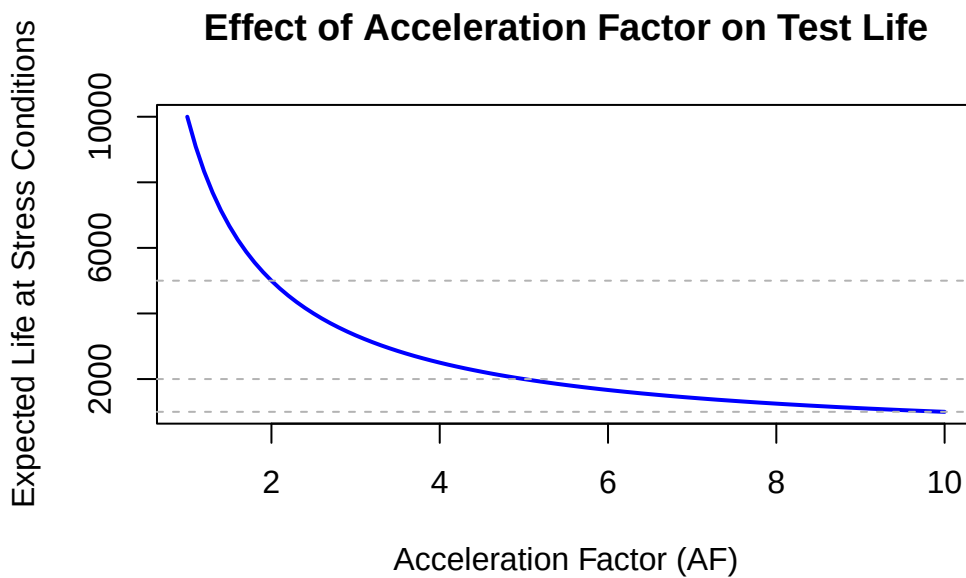
$$\text{Life at use conditions} = AF \times \text{Life at stress conditions}$$

The impact of AF on expected test life:

```
library(WeibullR.ALT)
```

```
AF_vals <- seq(1, 10, by = 0.1)
life_normal <- 10000 # assumed normal-use life

plot(AF_vals, life_normal / AF_vals, type = "l", col = "blue", lwd = 2,
     xlab = "Acceleration Factor (AF)",
     ylab = "Expected Life at Stress Conditions",
     main = "Effect of Acceleration Factor on Test Life")
abline(h = life_normal / c(2, 5, 10), col = "gray70", lty = 2)
```



An AF of 5 means the product fails five times faster under test stress — a 10,000-hour life in the field becomes a 2,000-hour test.

5.4 The Arrhenius Model

The **Arrhenius Model** describes the relationship between temperature and reaction rate, which maps directly to material degradation and failure:

$$AF = \exp \left(\frac{E_a}{k} \left(\frac{1}{T_{\text{use}}} - \frac{1}{T_{\text{stress}}} \right) \right)$$

Where: - E_a = activation energy in eV (electron volts) - k = Boltzmann constant (8.617×10^{-5} eV/K) - T_{use} , T_{stress} = temperatures in Kelvin

Higher activation energies mean the failure mechanism is more sensitive to temperature.

5.4.1 WeibullR.ALT: Arrhenius-Lognormal Example

The WeibullR.ALT package (Silkworth 2022) extends WeibullR with ALT models. Data: Class B insulation tested at three oven temperatures (170, 190, 220°C), from Nelson (1990) Table 4.1.

```
t41      <- NelsonData("table4.1")
set170   <- alt.data(t41$C170f, s = t41$C170s, stress = 170)
set190   <- alt.data(t41$C190f, s = t41$C190s, stress = 190)
set220   <- alt.data(t41$C220f, s = t41$C220s, stress = 220)

arrobj   <- alt.make(list(set170, set190, set220),
                      dist = "lognormal", alt.model = "arrhenius",
                      method.fit = "mle", view_dist_fits = FALSE)
```

The fitted curves shift left as temperature increases — higher temperature leads to shorter life.

Calculating the AF at 220°C vs 170°C with $E_a = 1.0$ eV:

```
E_a      <- 1.0
k        <- 8.617e-5
T_use    <- 170 + 273 # 443 K
T_stress <- 220 + 273 # 493 K

AF <- exp(E_a / k * (1 / T_use - 1 / T_stress))
cat("Acceleration Factor at 220°C vs 170°C:", round(AF, 1), "\n")
```

Acceleration Factor at 220°C vs 170°C: 14.3

At 220°C the insulation degrades several times faster than at 170°C.

i Try It

A component is tested at 85°C to accelerate failures. The use temperature is 25°C. The activation energy is 0.65 eV. Compute the AF.

```

E_a      <- 0.65
k        <- 8.617e-5
T_use    <- 25 + 273.15
T_stress <- 85 + 273.15
# AF = exp((E_a / k) * (1/T_use - 1/T_stress))

```

Solution

```

E_a      <- 0.65
k        <- 8.617e-5
T_use    <- 25 + 273.15
T_stress <- 85 + 273.15
AF <- exp((E_a / k) * (1 / T_use - 1 / T_stress))
cat("Acceleration Factor:", round(AF, 1), "\n")

```

Acceleration Factor: 69.3

5.5 The Power Law Model

The **Power Law Model** describes how a product's life is affected by non-thermal stresses (voltage, mechanical load, pressure):

$$AF = \left(\frac{S_{\text{stress}}}{S_{\text{use}}} \right)^n$$

Where n is the stress exponent — the sensitivity of the failure mechanism to the applied stress.

5.5.1 WeibullR.ALT: Power Law Example

Data: electrical insulating fluids tested at 7 voltage levels (26–38 kV).

```

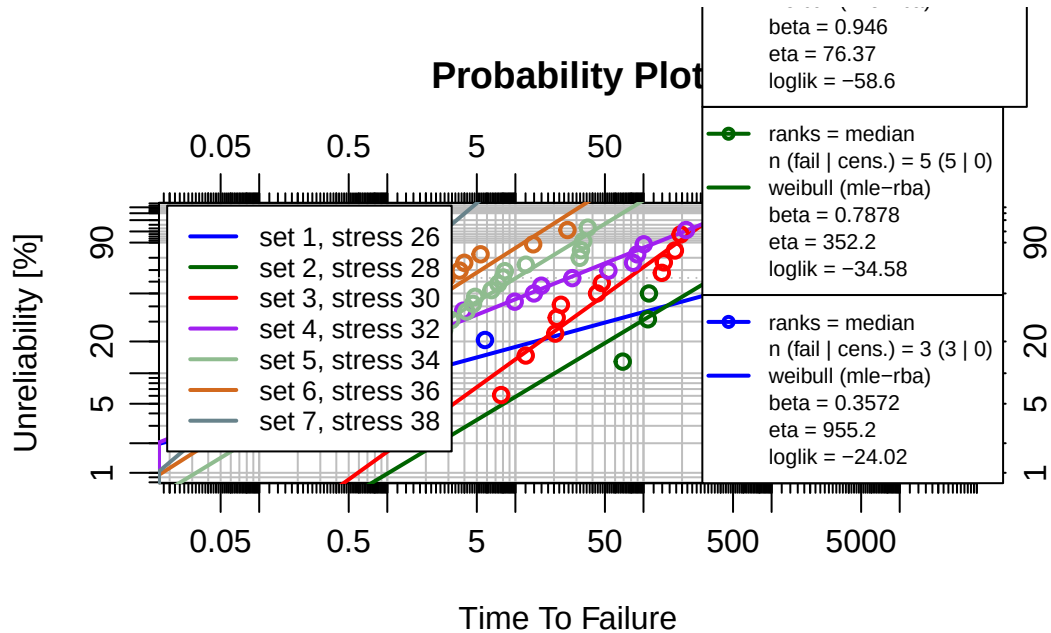
data_nelson <- NelsonData("table3.1")

data26 <- alt.data(data_nelson$kV26, stress = 26)
data28 <- alt.data(data_nelson$kV28, stress = 28)
data30 <- alt.data(data_nelson$kV30, stress = 30)
data32 <- alt.data(data_nelson$kV32, stress = 32)
data34 <- alt.data(data_nelson$kV34, stress = 34)

```

```
data36 <- alt.data(data_nelson$kV36, stress = 36)
data38 <- alt.data(data_nelson$kV38, stress = 38)

pwobj <- alt.make(list(data26, data28, data30, data32, data34, data36, data38),
  dist = "weibull", alt.model = "power")
```



Curves shift left as voltage increases — higher voltage stress leads to shorter life.

5.6 Multiple Stress Factors

The **Eyring Model** extends the Arrhenius approach to account for multiple stress factors (temperature, voltage, humidity):

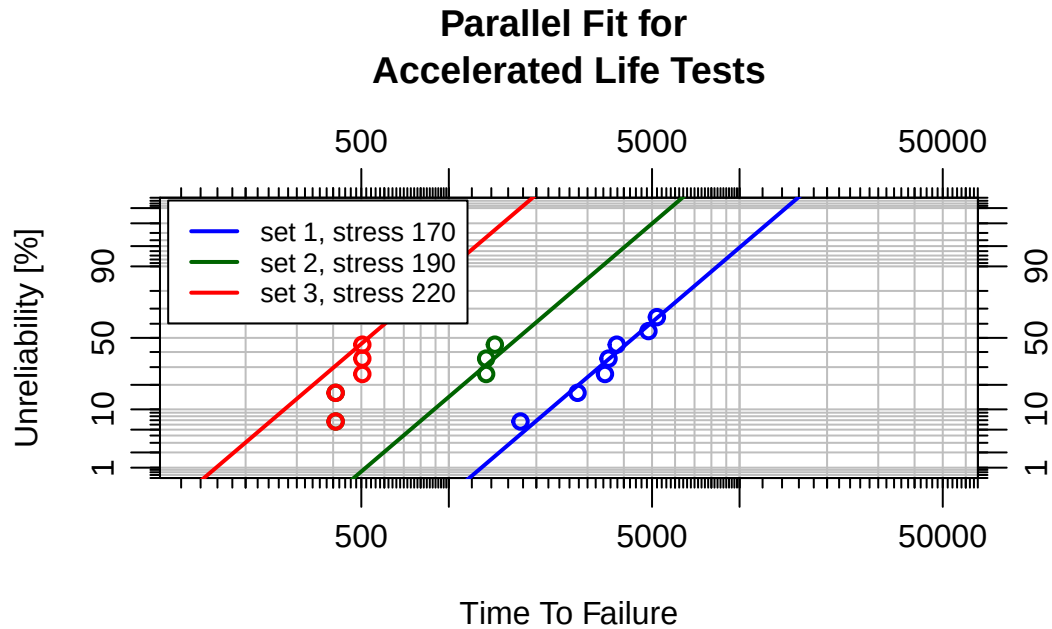
$$\text{Life}(S) = A \cdot \exp\left(\frac{B}{S} + C \cdot S\right)$$

The WeibullR.ALT package supports the Eyring model via `alt.make(..., alt.model = "eyring")`. See the package documentation for examples.

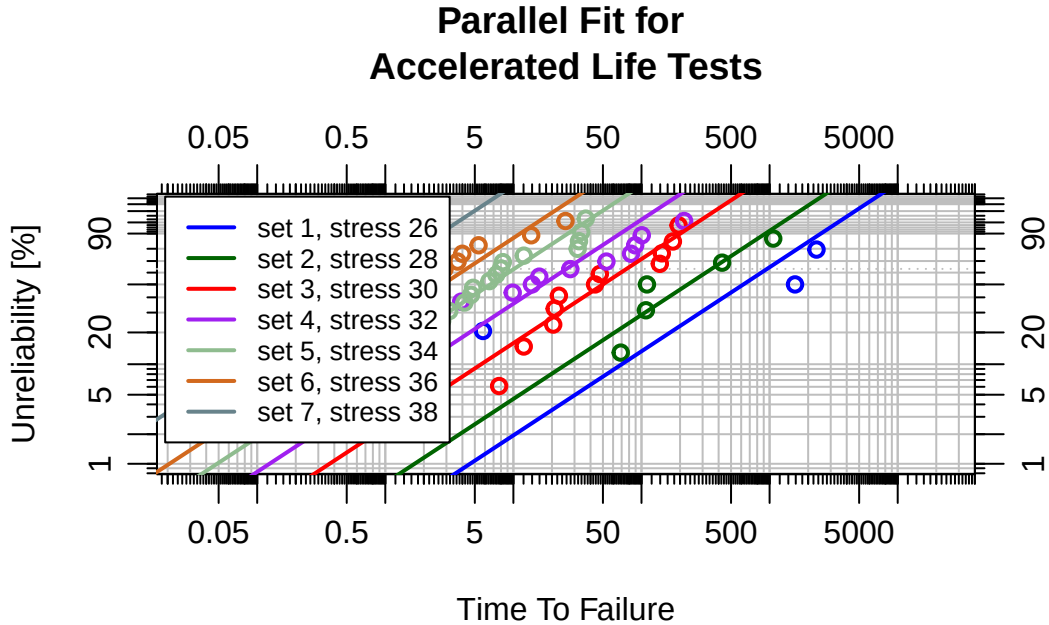
5.7 Parallelization

Parallelization constrains the shape parameter (slope) to be equal across all stress levels, simplifying the analysis:

```
prlobj <- alt.parallel(arrobj)
```



```
expobj <- alt.parallel(pwrobj)
```



With parallel slopes, the relationship between stress and life can be characterized by a single line (the relationship plot).

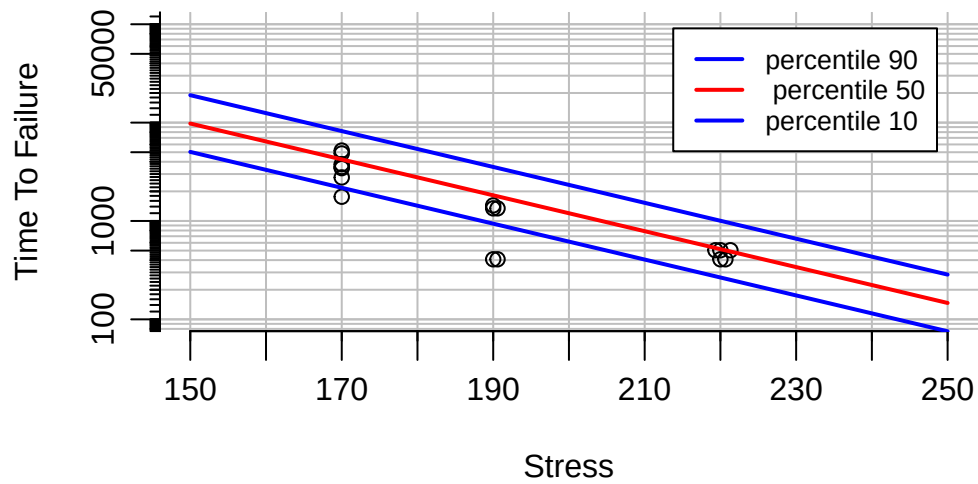
5.8 Relationship Plots

The **relationship plot** shows how life changes as stress increases (log-linear, stress on x-axis, log-life on y-axis). The slope characterizes the sensitivity of life to stress:

- **Negative slope:** higher stress $\rightarrow$ shorter life (typical).
- **Flat slope:** stress has little effect.
- **Positive slope:** higher stress $\rightarrow$ longer life (rare).

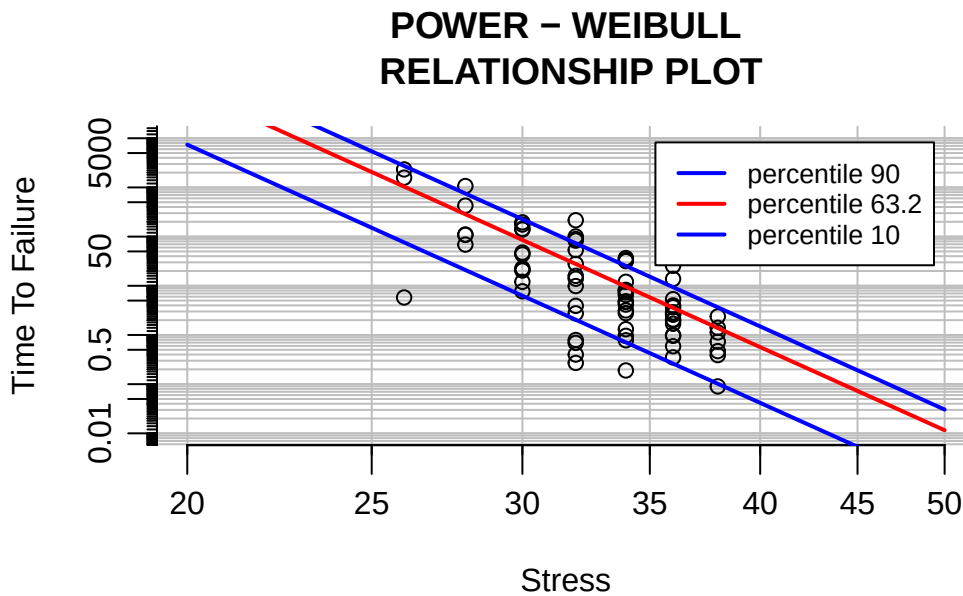
```
lnrojb <- alt.fit(prlojb)
plot(lnrojb, suppress.dev.new = TRUE)
```

ARRHENIUS – LOGNORMAL RELATIONSHIP PLOT



Each line corresponds to a different probability of failure (e.g., P10 = 10% failure probability, P63.2 = 63.2% failure probability = characteristic life).

```
lnrobj2 <- alt.fit(expobj)
plot(lnrobj2, suppress.dev.new = TRUE)
```



Both relationship plots confirm a negative slope — higher stress leads to shorter life in both datasets.

Review

What does a negative slope in a relationship plot indicate?

Answer

Higher stress leads to **shorter life**. This is the typical finding in ALT — stress accelerates the failure mechanism.

5.9 Summary

Key takeaways:

- **Acceleration Factor:** $\text{Life}_{\text{use}} = AF \times \text{Life}_{\text{stress}}$.
- **Arrhenius:** temperature-based acceleration; $AF = \exp(E_a/k \cdot (1/T_{\text{use}} - 1/T_{\text{stress}}))$.
- **Power Law:** non-thermal stress acceleration; $AF = (S_{\text{stress}}/S_{\text{use}})^n$.
- **Parallelization:** constrains slope to be equal across stress levels; required before fitting the relationship.
- **Relationship plots:** visualize life vs stress; negative slope = higher stress → shorter life.
- Use `alt.data()` → `alt.make()` → `alt.parallel()` → `alt.fit()` for the full workflow.

6 Repairable Systems Analysis

6.1 Introduction

Unlike life data analysis (Chapter 3), which focuses on the *time to first failure* of non-repairable components, **Repairable Systems Analysis** (Ascher and Feingold 1984; Meeker and Escobar 1998) models the entire sequence of recurring failures on a single system or fleet. This chapter covers the foundational theory and the **ReliaGrowR** tools for fitting models, computing fleet-level exposure, and estimating the Mean Cumulative Function.

6.2 Learning Objectives

By the end of this chapter, you will be able to:

- Define the distinction between repairable and non-repairable systems.
- Explain the Homogeneous Poisson Process (HPP) and Non-Homogeneous Poisson Process (NHPP).
- Describe the Power Law Process and its parameters.
- Fit and interpret the Crow-AMSAA model using `nhpp()` from ReliaGrowR.
- Apply the Piecewise NHPP model to detect changes in failure behavior.
- Compute fleet-level exposure using `exposure()`.
- Estimate the Mean Cumulative Function (MCF) using `mcf()`.
- Forecast future failure counts from a fitted Power Law model.

The **ReliaGrowR** package (Govan 2024) provides functions for repairable systems analysis in R.

```
library(ReliaGrowR)
```

6.3 Repairable vs Non-Repairable Systems

A **non-repairable system** is discarded after failure and replaced with a new, statistically identical unit. The unit of analysis is the *time to first failure*. Weibull analysis applies.

A **repairable system** is restored to service after each failure and continues accumulating operating time. The unit of analysis is the *sequence of inter-failure times* or cumulative failure counts over the system's life.

Two repair assumptions are common:

Repair type	Description	Model
Perfect repair	As-good-as-new after each repair	Renewal process
Minimal repair	As-bad-as-old — failure fixed, nothing else changed	NHPP

Most field maintenance data best fits the **minimal repair** assumption, making NHPP the standard framework.

The **recurrence rate** (intensity function) $\rho(t)$ is the instantaneous rate of failure occurrence at time t :

$$\rho(t) = \frac{d E[N(t)]}{dt}$$

6.4 Failure Processes: HPP and NHPP

6.4.1 Homogeneous Poisson Process (HPP)

The **HPP** assumes a *constant* failure rate λ throughout the system's life:

$$E[N(t)] = \lambda t \quad \rho(t) = \lambda \quad \text{MTBF} = 1/\lambda$$

The HPP is appropriate when failures occur at random with no trend.

6.4.2 Non-Homogeneous Poisson Process (NHPP)

The **NHPP** allows the intensity function to change with time:

$$\rho(t) = \frac{d\Lambda(t)}{dt}$$

When $\rho(t)$ increases, the system is deteriorating; when it decreases, the system is improving.

```

t <- seq(1, 100, by = 1)

# HPP (constant rate)
hpp_intensity <- rep(0.05, length(t))

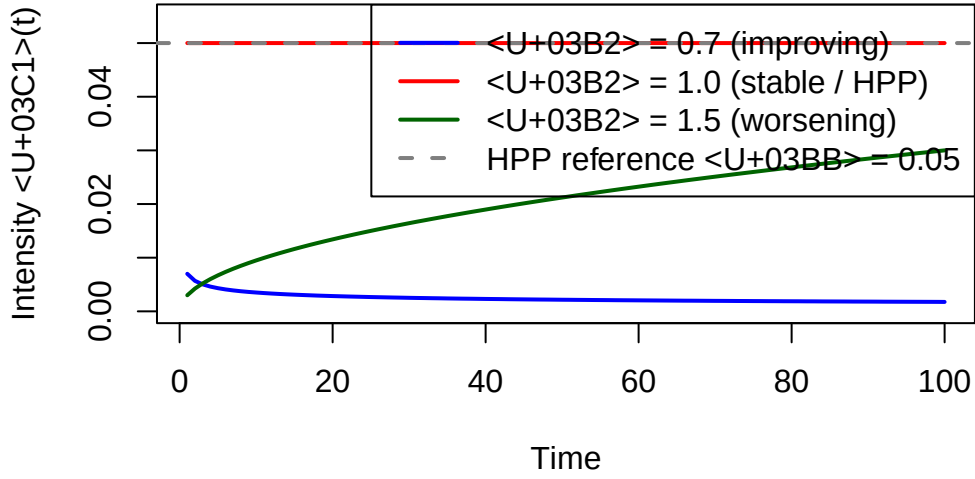
# NHPP Power Law scenarios
nhpp_improving <- 0.01 * 0.7 * t^(0.7 - 1) # beta = 0.7
nhpp_stable <- 0.05 * 1.0 * t^(1.0 - 1) # beta = 1.0
nhpp_worsening <- 0.002 * 1.5 * t^(1.5 - 1) # beta = 1.5

ylim_max <- max(c(hpp_intensity, nhpp_improving, nhpp_stable, nhpp_worsening)) * 1.1

plot(t, nhpp_improving, type = "l", col = "blue", lwd = 2,
      ylim = c(0, ylim_max),
      xlab = "Time", ylab = "Intensity (t)",
      main = "NHPP Power Law Intensity Functions")
lines(t, nhpp_stable, col = "red", lwd = 2)
lines(t, nhpp_worsening, col = "darkgreen", lwd = 2)
abline(h = 0.05, col = "gray50", lwd = 2, lty = 2)
legend("topright",
      legend = c(" = 0.7 (improving)", " = 1.0 (stable / HPP)",
                  " = 1.5 (worsening)", "HPP reference = 0.05"),
      col = c("blue", "red", "darkgreen", "gray50"),
      lwd = 2, lty = c(1, 1, 1, 2))

```

NHPP Power Law Intensity Functions



6.5 The Power Law Process

The **Power Law Process** is the most widely used NHPP for repairable systems:

$$E[N(t)] = \lambda t^\beta \quad \rho(t) = \lambda \beta t^{\beta-1}$$

Parameter interpretation:

- $\lambda > 0$ — scale parameter; overall magnitude of the failure rate.
- $\beta < 1$: decreasing intensity (system improving).
- $\beta = 1$: constant intensity = HPP.
- $\beta > 1$: increasing intensity (system deteriorating).

The **instantaneous MTBF** at time t :

$$\text{MTBF}(t) = \frac{1}{\rho(t)} = \frac{1}{\lambda \beta t^{\beta-1}}$$

Maximum likelihood estimates given n failures at times $t_1 < \dots < t_n$ over total time T :

$$\hat{\beta} = \frac{n}{\sum_{i=1}^n \ln(T/t_i)} \quad \hat{\lambda} = \frac{n}{T^{\hat{\beta}}}$$

Review

A repairable system is observed to have an increasing failure rate over time. Which model is most appropriate?

Answer

NHPP Power Law with $\beta > 1$. An increasing intensity function is characteristic of a deteriorating system. HPP (constant rate) cannot capture this trend.

6.6 NHPP Analysis with `nhpp()`

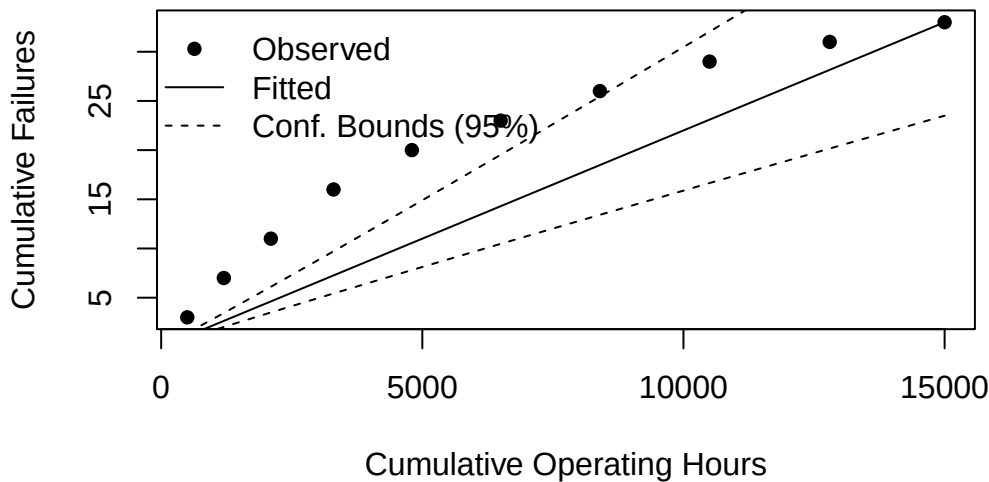
The `nhpp()` function fits the Power Law Process (Crow-AMSAA model) (Crow 1975) to cumulative failure data by maximum likelihood, returning estimates of λ and β .

A curve that bends downward (concave) indicates $\hat{\beta} < 1$ — improving system. Upward bending (convex) indicates $\hat{\beta} > 1$ — deteriorating system.

```
times      <- c(500, 1200, 2100, 3300, 4800, 6500, 8400, 10500, 12800, 15000)
failures    <- c(3, 4, 4, 5, 4, 3, 3, 3, 2, 2)

fit <- nhpp(time = times, event = failures)
plot(fit,
      main = "Crow-AMSAA Model: Industrial Pump Fleet",
      xlab = "Cumulative Operating Hours",
      ylab = "Cumulative Failures")
```

Crow-AMSAA Model: Industrial Pump Fleet



The concave curve and $\hat{\beta} < 1$ indicate the pump fleet's failure rate is decreasing — a favorable operating regime.

Try It

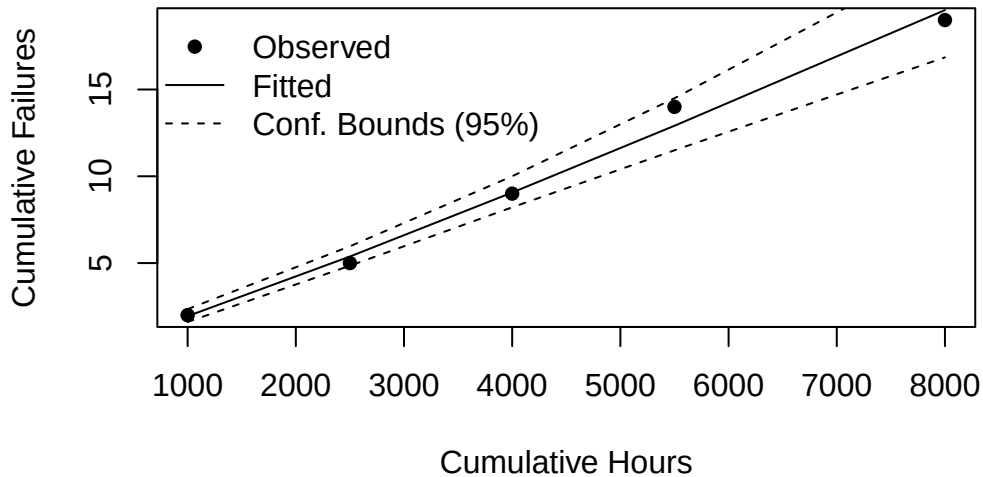
A compressor fleet recorded these cumulative operating hours and failure counts. Fit a Power Law model using `nhpp()` with `method = "LS"`.

```
times    <- c(1000, 2500, 4000, 5500, 8000)
failures <- c(2, 3, 4, 5, 5)
# fit <- nhpp(time = times, event = failures, method = "LS")
# plot(fit)
```

Solution

```
times    <- c(1000, 2500, 4000, 5500, 8000)
failures <- c(2, 3, 4, 5, 5)
fit <- nhpp(time = times, event = failures, method = "LS")
plot(fit, main = "Crow-AMSAA: Compressor Fleet",
     xlab = "Cumulative Hours", ylab = "Cumulative Failures")
```

Crow-AMSAA: Compressor Fleet



6.7 Piecewise NHPP

The **Piecewise NHPP** (Guo et al. 2010) fits a separate Power Law to each segment of the time axis, allowing for structural changes in failure behavior (overhauls, design modifications, operational changes):

$$E[N(t)] = \begin{cases} \lambda_1 t^{\beta_1} & 0 < t \leq \tau \\ E[N(\tau)] + \lambda_2 (t - \tau)^{\beta_2} & t > \tau \end{cases}$$

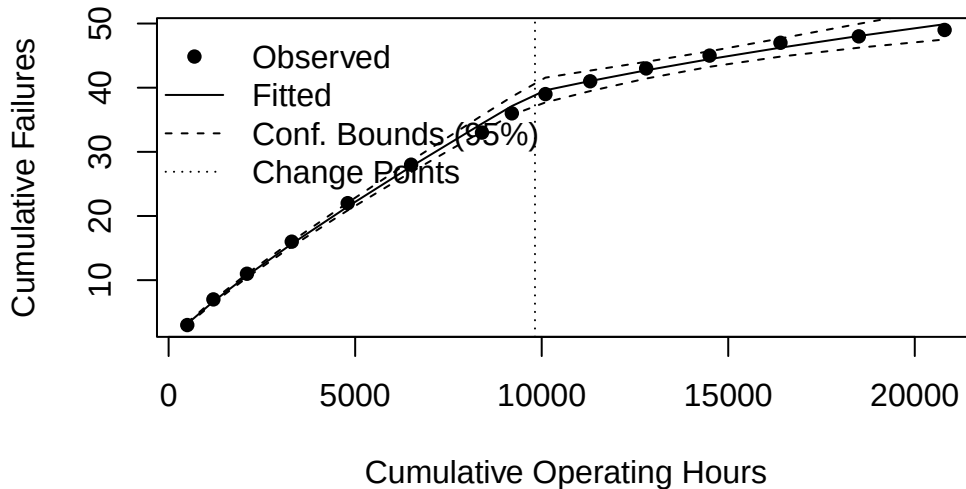
Pass a breakpoint estimate via the **breaks** argument. The **segmented** algorithm then optimizes the breakpoint location from the data.

```
times2    <- c(500, 1200, 2100, 3300, 4800, 6500, 8400, 9200, 10100, 11300,
               12800, 14500, 16400, 18500, 20800)
failures2 <- c(3, 4, 4, 5, 6, 6, 5, 3, 3, 2, 2, 2, 2, 1, 1)

fit_pw <- nhpp(time = times2, event = failures2, breaks = 8000, method = "LS")
plot(fit_pw,
     main = "Piecewise NHPP: Fleet Overhaul at Hour 8000",
```

```
xlab = "Cumulative Operating Hours",
ylab = "Cumulative Failures")
```

Piecewise NHPP: Fleet Overhaul at Hour 8000



The first segment (pre-overhaul, steeper slope) gives way to a flatter or downward-bending second segment, confirming the overhaul reduced the failure rate. The fitted breakpoint may differ slightly from the input value — here it is estimated near hour 9,820.

💡 Review

What is the primary reason to use a Piecewise NHPP instead of a single Crow-AMSAA model?

Answer

To account for a **known or suspected structural change** in the failure process (overhaul, design change, operational change). A piecewise model fits each segment independently, capturing the change in failure behavior.

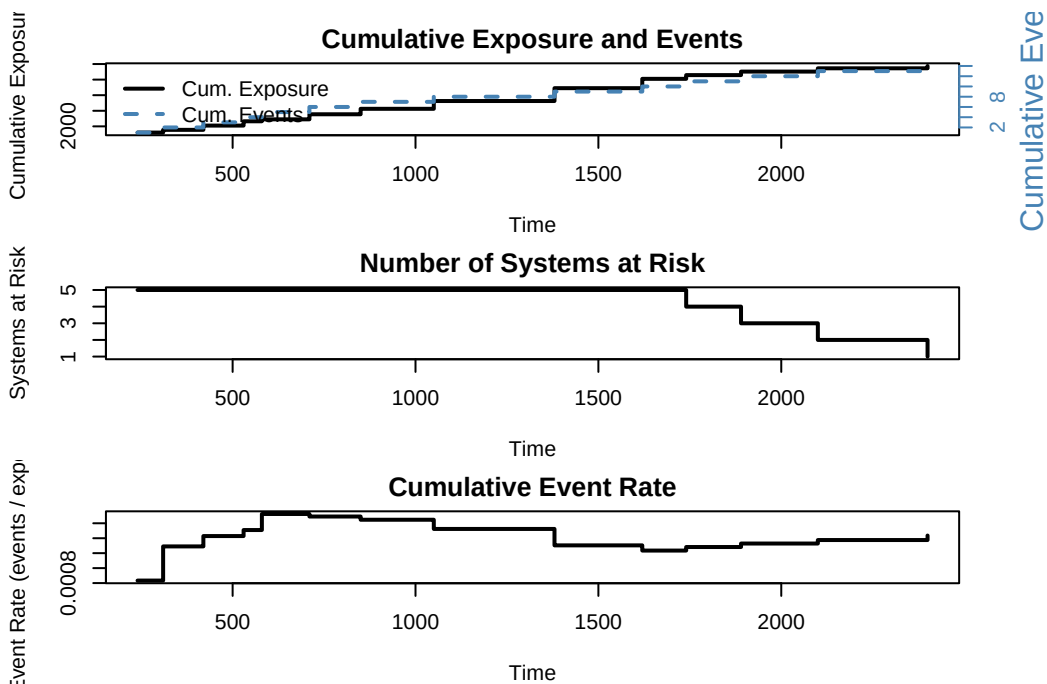
6.8 Exposure Analysis

When analyzing a *fleet*, individual units enter and leave observation at different times. **Exposure** is the total operating time accumulated across all units currently at risk.

The `exposure()` function computes: - **Cumulative exposure** — total system-time at risk up to each event time. - **Number at risk** — how many units were under observation at each time. - **Event rate** — cumulative events divided by cumulative exposure.

```
pump_data <- data.frame(
  id    = c(1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 5),
  time  = c(310, 850, 1620, 420, 1050, 2100, 580, 1890, 240, 710, 1380, 2400, 530, 1740),
  event = c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1)
)

exp_result <- exposure(id = pump_data$id, time = pump_data$time,
                      event = pump_data$event)
plot(exp_result)
```



The exposure plot shows how total observed system-time accumulates and how the fleet-wide event rate behaves over time.

💡 Review

Why is it important to account for exposure when comparing failure rates across systems with different observation periods?

Answer

Without normalizing by exposure, a system observed for twice as long will appear to

have twice as many failures even if its underlying rate is identical. Exposure provides the denominator — failures per unit time — so rates can be compared fairly.

6.9 Mean Cumulative Function

The **Mean Cumulative Function (MCF)** is a non-parametric estimate of the expected cumulative number of failures *per system* as a function of time. It makes no distributional assumptions and handles systems with different observation periods.

The Nelson-Aalen estimator:

$$\hat{M}(t) = \sum_{t_j \leq t} \frac{d_j}{n_j}$$

where d_j is the number of failures at time t_j and n_j is the number of systems at risk at t_j .

Pass `end_time` to `mcf()` when systems were observed beyond their last recorded failure — otherwise the risk set is deflated too early, inflating the rate estimate.

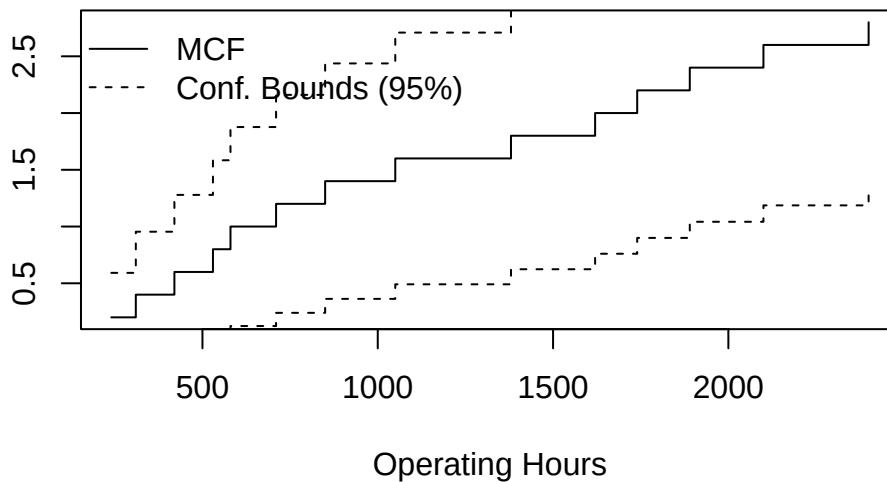
```
end_times <- c("1" = 3000, "2" = 3000, "3" = 3000, "4" = 3000, "5" = 3000)

mcf_result <- mcf(id = pump_data$id, time = pump_data$time,
                  event = pump_data$event,
                  end_time = end_times)

plot(mcf_result,
     main = "Mean Cumulative Function: Industrial Pump Fleet",
     xlab = "Operating Hours",
     ylab = "Expected Cumulative Failures per System")
```

Expected Cumulative Failures per System

Mean Cumulative Function: Industrial Pump Fleet



The slope of the MCF reveals the recurrence trend: steepening = increasing failure rate; flattening = improving.

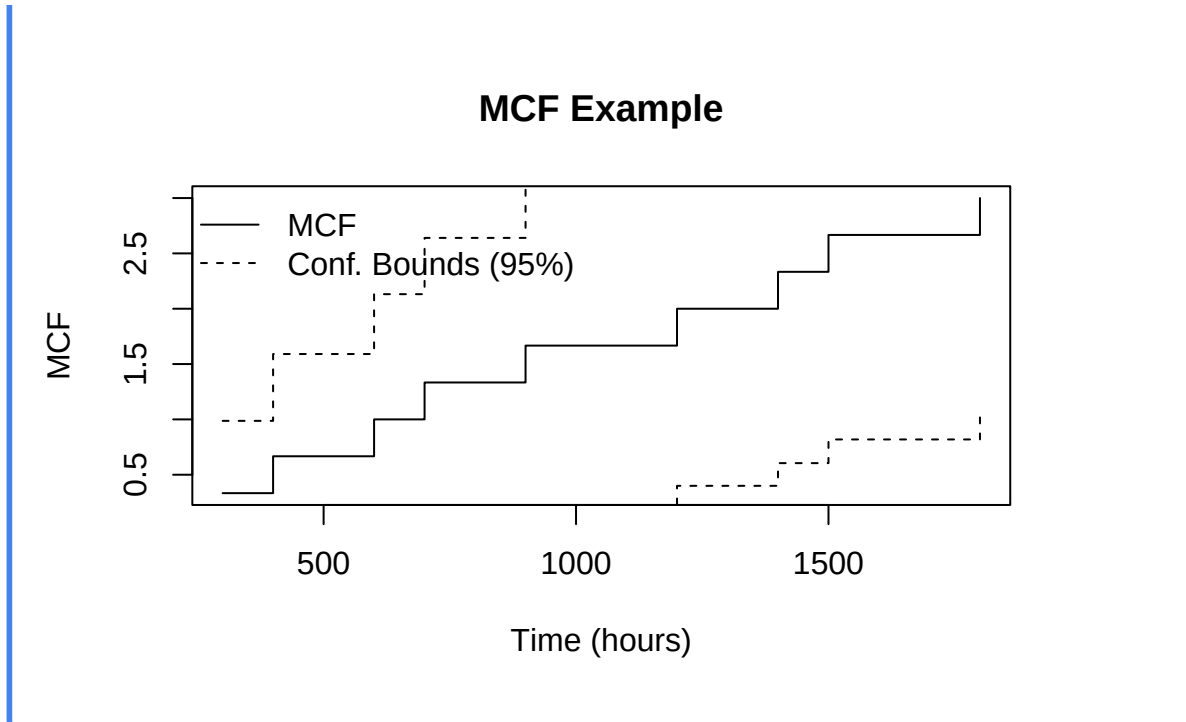
Try It

Three systems were observed with the following failure times. Each was observed until hour 2000. Compute and plot the MCF.

```
id    <- c(rep("1", 3), rep("2", 4), rep("3", 2))
time  <- c(400, 900, 1500, 300, 700, 1200, 1800, 600, 1400)
event <- rep(1, 9)
end_times2 <- c("1" = 2000, "2" = 2000, "3" = 2000)
# fit <- mcf(id, time, event, end_time = end_times2)
# plot(fit)
```

Solution

```
id    <- c(rep("1", 3), rep("2", 4), rep("3", 2))
time  <- c(400, 900, 1500, 300, 700, 1200, 1800, 600, 1400)
event <- rep(1, 9)
end_times2 <- c("1" = 2000, "2" = 2000, "3" = 2000)
fit <- mcf(id = id, time = time, event = event, end_time = end_times2)
plot(fit, main = "MCF Example", xlab = "Time (hours)", ylab = "MCF")
```



6.10 Interpreting Results and Predicting Future Failures

6.10.1 Reading $\hat{\beta}$

$\hat{\beta}$	Interpretation	Typical cause
< 1	Decreasing failure rate	Effective maintenance, infant mortality resolved
$= 1$	Constant failure rate	Random external shocks
> 1	Increasing failure rate	Wear-out, accumulating damage, aging

6.10.2 Cumulative vs Instantaneous MTBF

- **Cumulative MTBF** $= T/N(T)$ — historical average over all observed failures.
- **Instantaneous MTBF** at time $t = 1/\rho(t) = 1/(\hat{\lambda}\hat{\beta}t^{\hat{\beta}-1})$ — expected time to the *next* failure.

For $\hat{\beta} > 1$ (aging system), the instantaneous MTBF falls *below* the cumulative average at late times — the system's near-term reliability is worse than its history suggests.

6.10.3 Predicting Future Failures

Expected additional failures from current time T to a future time $T + \Delta t$:

$$E[\Delta N] = \hat{\lambda}(T + \Delta t)^{\hat{\beta}} - \hat{\lambda}T^{\hat{\beta}}$$

```
lambda    <- 0.0003
beta      <- 0.85
T_current <- 15000
T_future  <- 17000

expected_add <- lambda * T_future^beta - lambda * T_current^beta
cat("Expected additional failures:", round(expected_add, 2), "\n")
```

Expected additional failures: 0.12

Try It

A system has been operating for 5,000 hours. A Power Law fit yielded $\lambda = 0.0005$ and $\beta = 1.2$. Calculate the expected additional failures in the next 1,000 hours.

```
lambda    <- 0.0005
beta      <- 1.2
T_current <- 5000
T_future  <- 6000
# E[ΔN] = lambda * T_future^beta - lambda * T_current^beta
```

Solution

```
lambda    <- 0.0005
beta      <- 1.2
T_current <- 5000
T_future  <- 6000
expected_failures <- lambda * T_future^beta - lambda * T_current^beta
cat("Expected additional failures:", round(expected_failures, 2), "\n")
```

Expected additional failures: 3.36

Review

A system has a fitted Power Law model with $\beta = 1.35$. What is the operational implication?

Answer

The failure rate is **increasing** — wear-out or aging is occurring. The system may be approaching a major overhaul threshold. The instantaneous MTBF is below the cumulative historical average.

6.11 Summary

Key takeaways:

- Repairable systems analysis models the *sequence* of recurring failures, not just the first failure.
- The **Power Law Process** ($E[N(t)] = \lambda t^\beta$) is the standard NHPP for repairable systems.
- $\hat{\beta}$ determines whether the failure rate is improving (< 1), stable ($= 1$), or worsening (> 1).
- `nhpp()` fits the Crow-AMSAA model; add **breaks** for piecewise fitting.
- `exposure()` computes fleet-level operating time at risk.
- `mcf()` estimates cumulative failures per system non-parametrically.
- Future failure counts: $E[\Delta N] = \hat{\lambda}(T + \Delta t)^{\hat{\beta}} - \hat{\lambda}T^{\hat{\beta}}$.

7 Interactive Plots with ReliaPlotR

7.1 Introduction

The previous chapters produce static base-R plots. **ReliaPlotR** (Govan 2023a) is a companion package that turns those same fitted model objects into *interactive* plotly charts — with hover tooltips, zoom/pan controls, and built-in download buttons — without requiring you to change how you fit your models.

The API follows a single convention: pass the fitted object from **WeibullR**, **ReliaGrowR**, or **WeibullR.ALT** directly to the corresponding `plotly_*` function.

7.2 Installation

```
install.packages("ReliaPlotR")
```

```
library(ReliaPlotR)
library(WeibullR)
library(ReliaGrowR)
library(WeibullR.ALT)
```

ReliaPlotR is available on CRAN: <https://CRAN.R-project.org/package=ReliaPlotR>

7.3 Life Data Analysis Plots

These functions accept the `wblr` objects produced by **WeibullR** — exactly as shown in Chapter 3. No reformatting needed.

7.3.1 Weibull Probability Plot — `plotly_wblr()`

Pass any fitted `wblr` object (or a list of objects for overlays):

```
failures <- c(30, 49, 82, 90, 96)
obj <- wblr.conf(wblr.fit(wblr(failures)))
plotly_wblr(obj, fitCol = "blue", confCol = "blue",
            main = "Weibull Probability Plot")
```

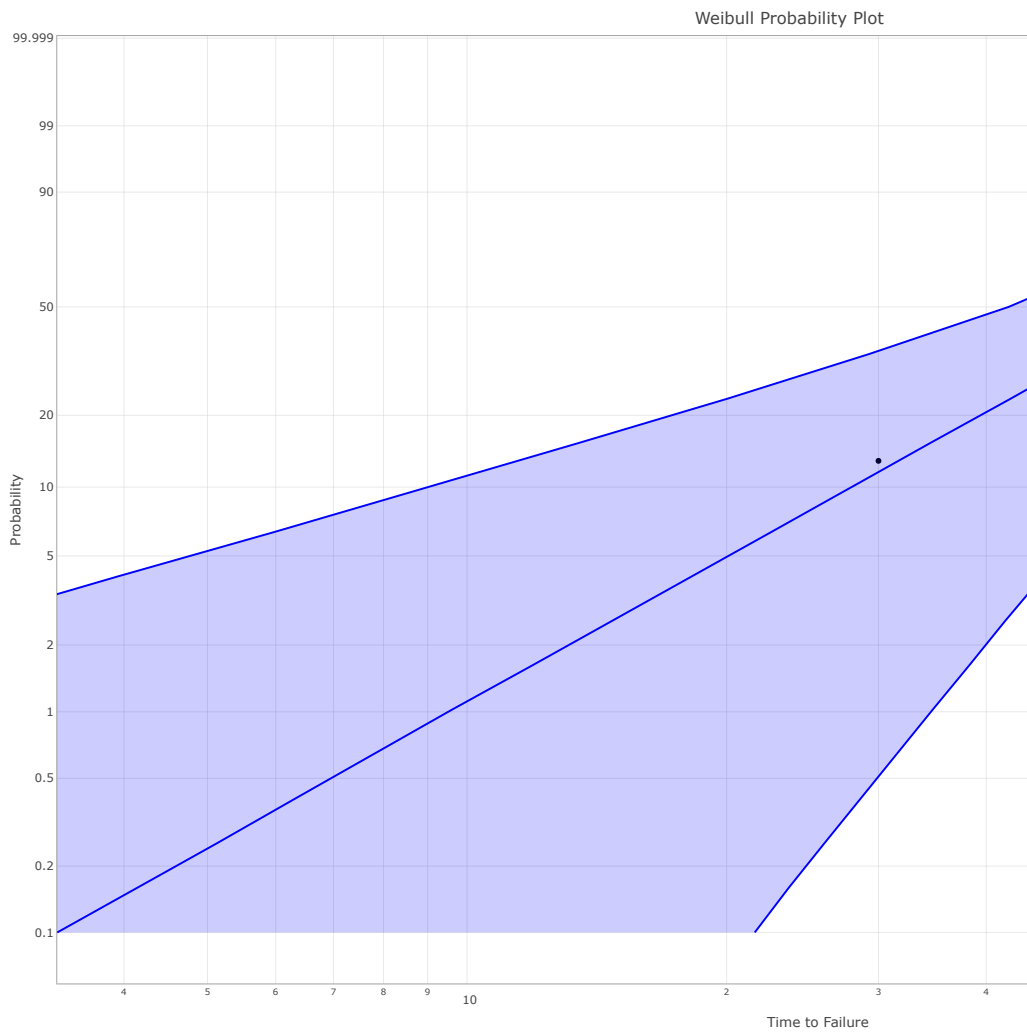


Figure 7.1: Interactive Weibull probability plot with confidence bounds

Hover over any point to read off its exact time and cumulative probability. The results table (, Anderson-Darling) appears in the lower right.

Including suspensions:

```
suspensions <- c(100, 45, 10)
obj_susp <- wblr.conf(wblr.fit(wblr(failures, suspensions)))
plotly_wblr(obj_susp, susp = suspensions,
            fitCol = "red", confCol = "red",
            main = "Weibull Plot with Suspensions")
```

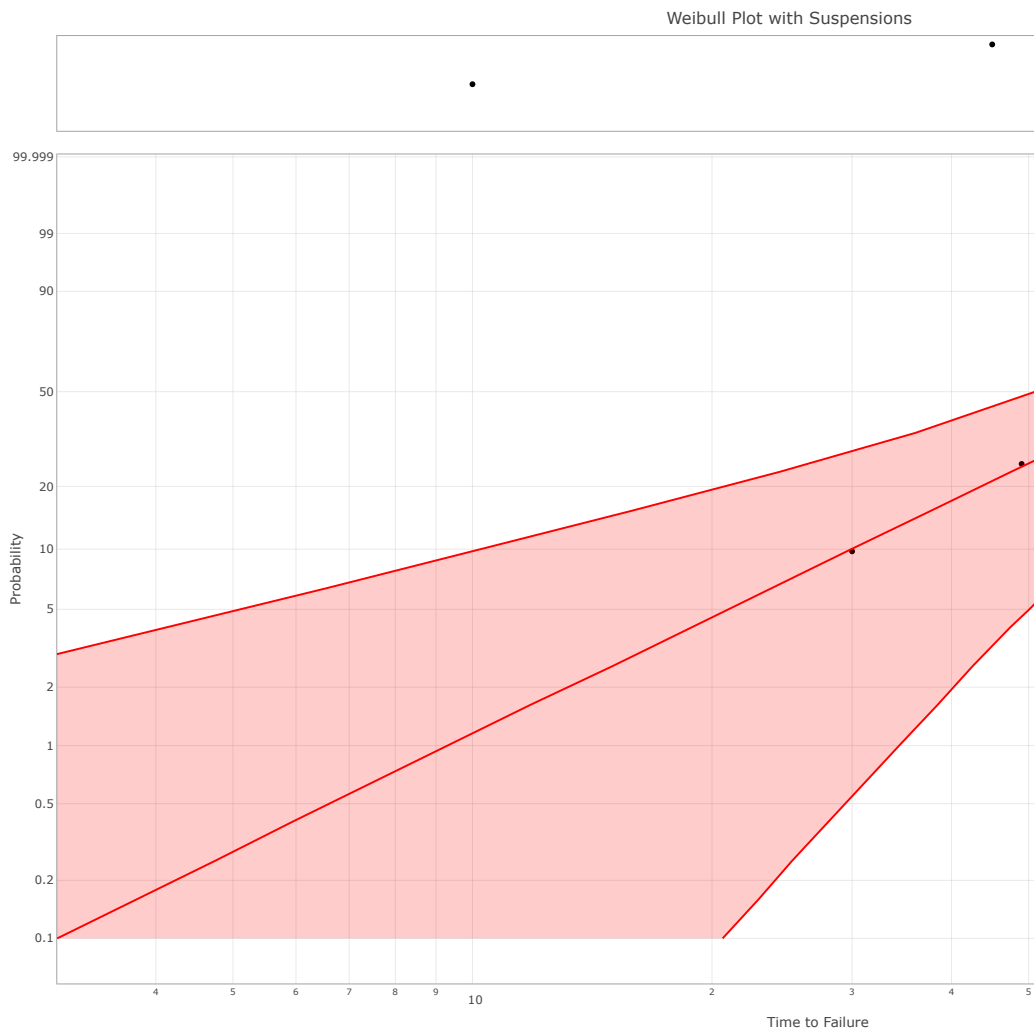


Figure 7.2: Weibull plot with right-censored suspensions

Overlaying two models — pass a list:

```
obj1 <- wblr.conf(wblr.fit(wblr(failures, col = "red")))
obj2 <- wblr.conf(wblr.fit(wblr(failures, suspensions, col = "purple")))
plotly_wblr(list(obj1, obj2),
             cols = c("red", "purple"),
             main = "Multi-Model Weibull Overlay")
```

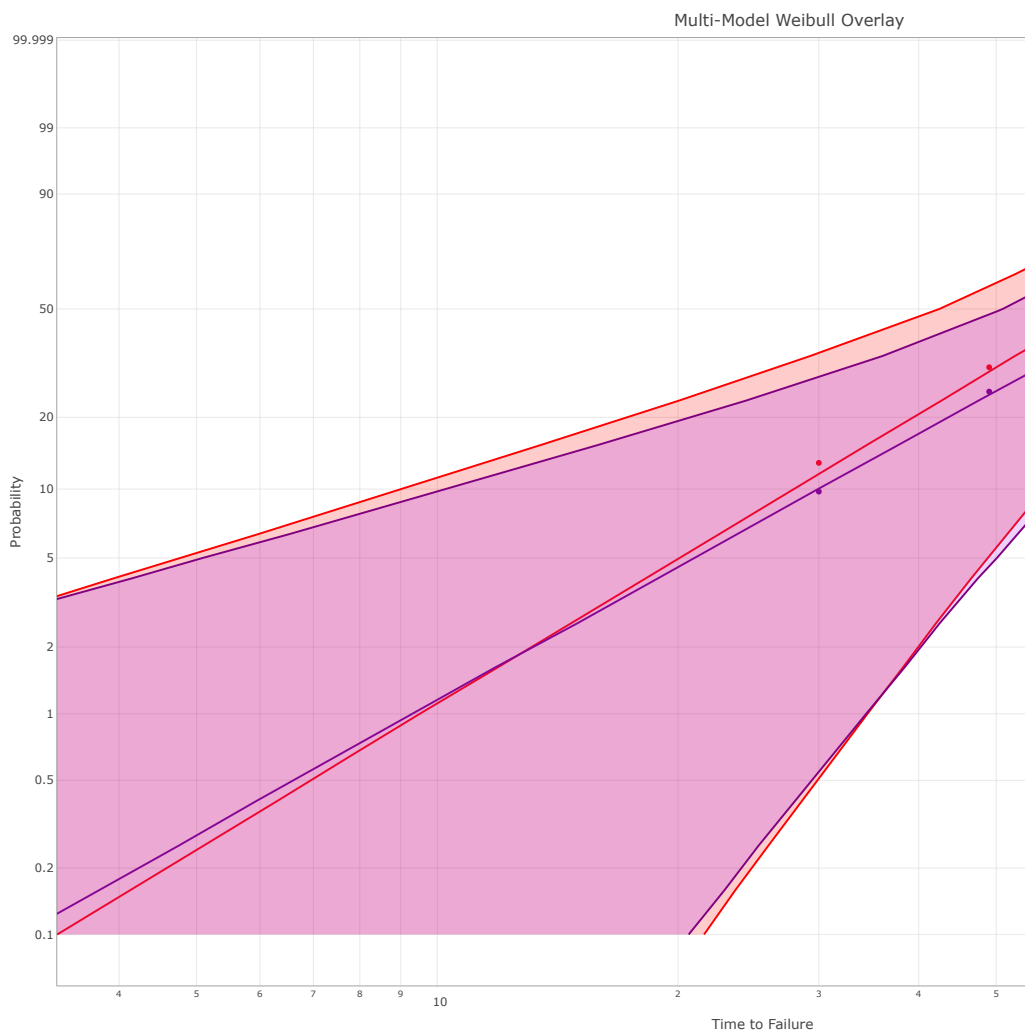


Figure 7.3: Multi-model overlay: with and without suspensions

7.3.2 Contour Plot — `plotly_contour()`

Pass a `wblr` object fitted with likelihood-ratio bounds (`method.conf = "lrb"`):

```
obj_lrb <- wblr.conf(wblr.fit(wblr(failures, col = "blue"), method.fit = "mle"),
                    method.conf = "lrb")
plotly_contour(obj_lrb,
               main = "Weibull Contour Plot (90% CL)",
               xlab = "Eta ( )", ylab = "Beta ( ")
```

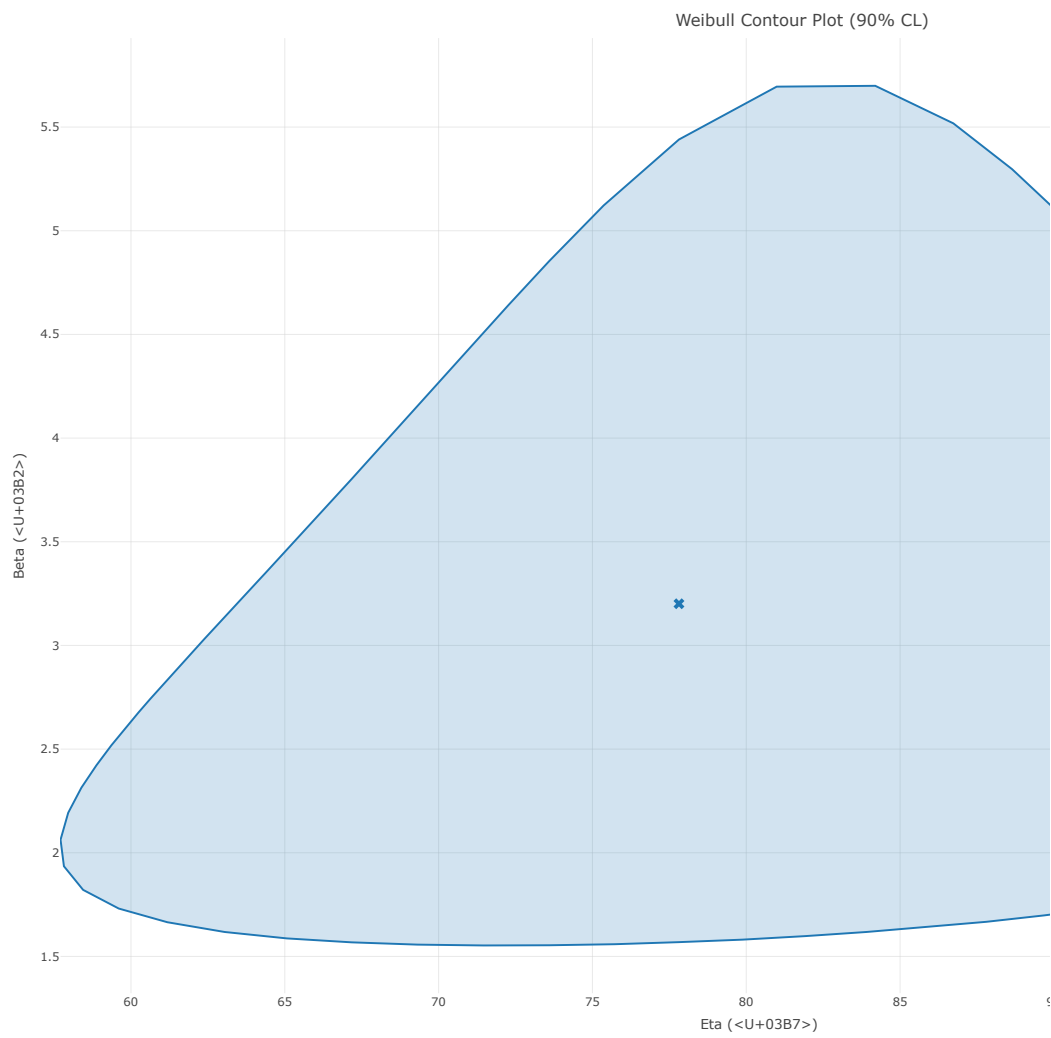


Figure 7.4: Interactive contour plot showing the 90% confidence region for η and β

Hover to read off any (,) combination inside the confidence region. Overlaying multiple contours:

```
set.seed(123)
data <- data.frame(
  time = c(rweibull(5, 0.5, 20), rweibull(10, 1, 10), rweibull(5, 2, 5),
           rweibull(100, 2, 10)),
  event = c(rep(1, 20), rep(0, 100)),
  failure_mode = c(rep("A", 5), rep("B", 10), rep("C", 5), rep("", 100))
)
dat1 <- dat2 <- dat3 <- data
dat1$event[dat1$failure_mode != "A"] <- 0
dat2$event[dat2$failure_mode != "B"] <- 0
dat3$event[dat3$failure_mode != "C"] <- 0

c1 <- wblr.conf(wblr.fit(wblr(dat1, col = "blue"), method.fit = "mle"), method.conf = "lr")
c2 <- wblr.conf(wblr.fit(wblr(dat2, col = "red"), method.fit = "mle"), method.conf = "lr")
c3 <- wblr.conf(wblr.fit(wblr(dat3, col = "orange"), method.fit = "mle"), method.conf = "lr")

plotly_contour(list(c1, c2, c3),
  cols = c("blue", "red", "orange"),
  main = "Contour Comparison: Three Failure Modes")
```

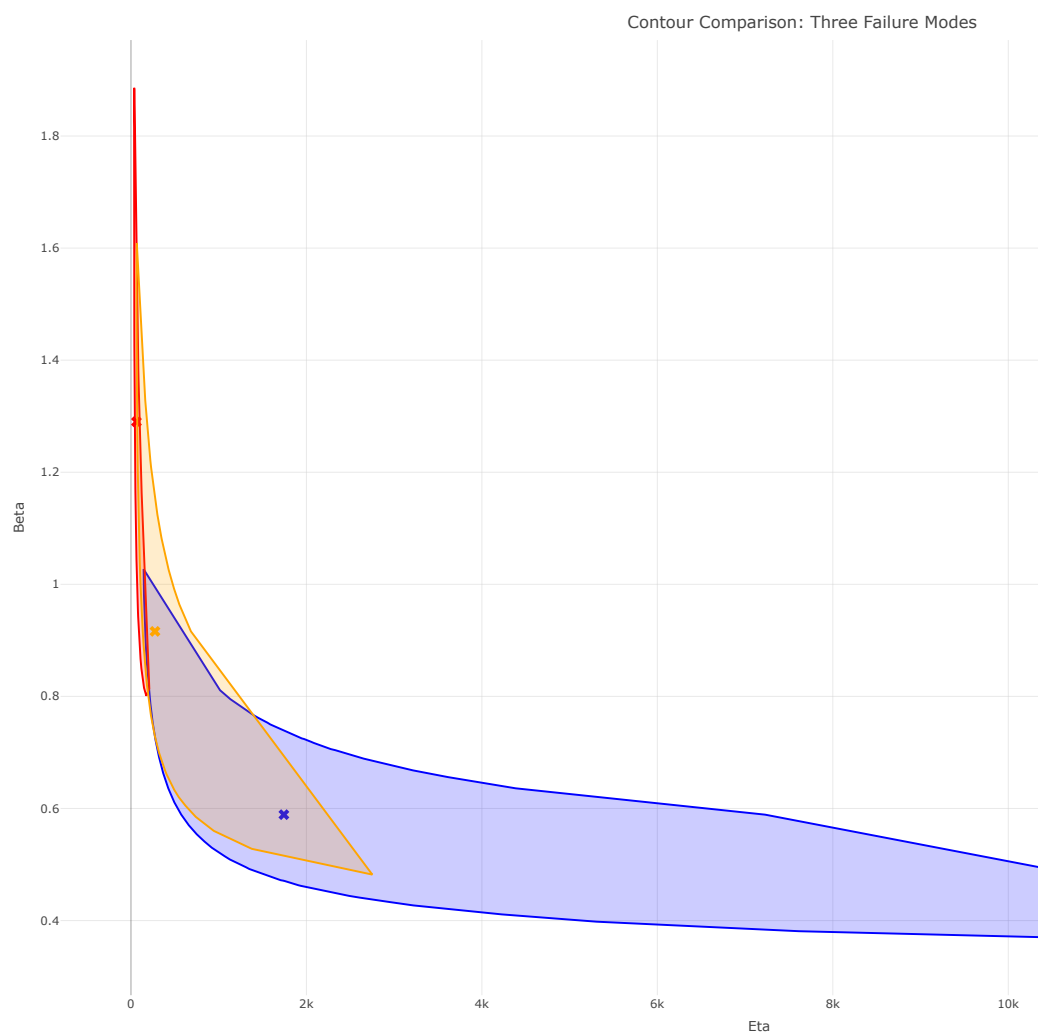


Figure 7.5: Overlaid contour plots for three competing failure modes

7.4 Reliability Growth Plots

These functions accept fitted objects from `ReliaGrowR` — the same objects plotted in Chapter 4.

7.4.1 Duane Plot — `plotly_duane()`

```
times    <- c(100, 200, 300, 400, 500)
failures <- c(1, 2, 1, 3, 2)

fit_duane <- duane(times, failures)
plotly_duane(fit_duane,
             main = "Duane Reliability Growth",
             xlab = "Cumulative Time",
             ylab = "Cumulative MTBF")
```

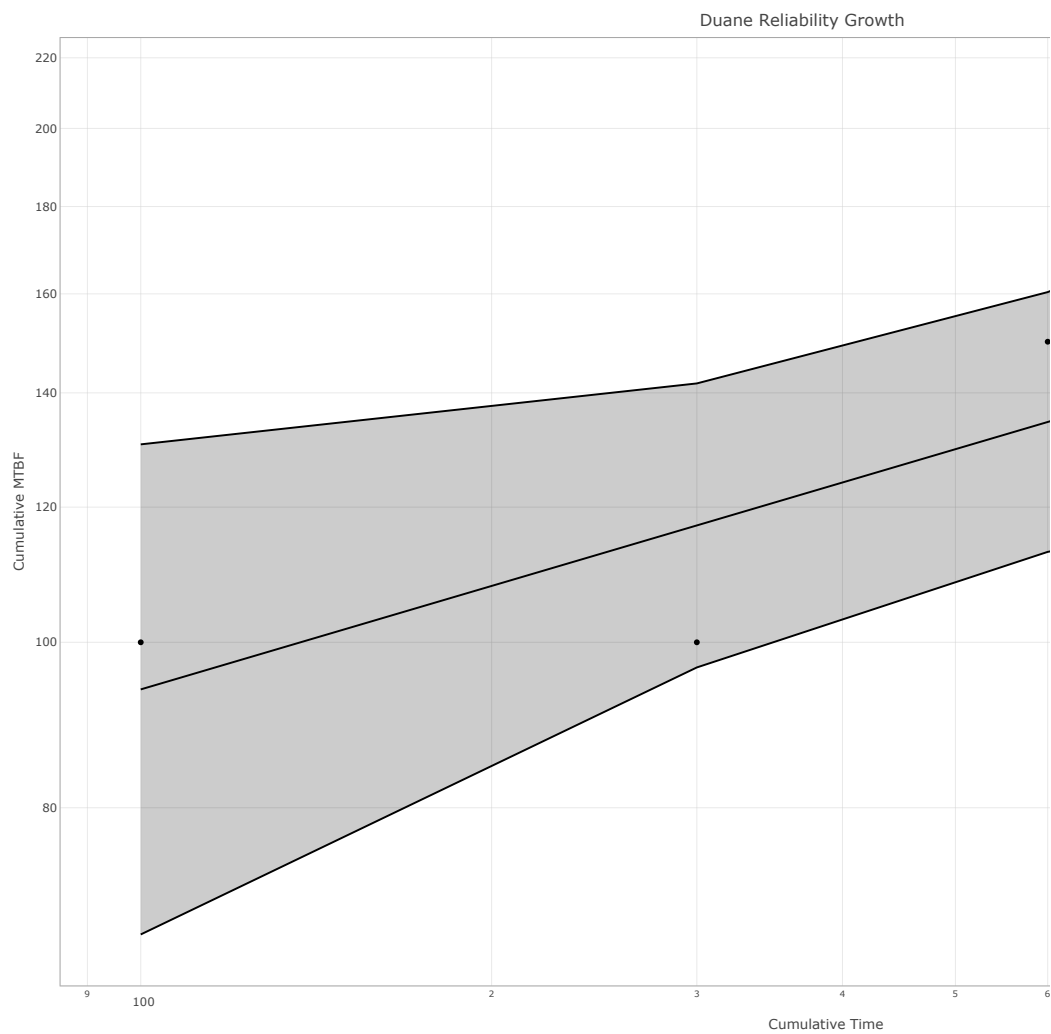


Figure 7.6: Interactive Duane reliability growth plot

7.4.2 Crow-AMSAA Plot — `plotly_rga()`

```
fit_rga <- rga(times, failures)
plotly_rga(fit_rga,
  main = "Crow-AMSAA Model",
  xlab = "Cumulative Time",
  ylab = "Cumulative Failures")
```

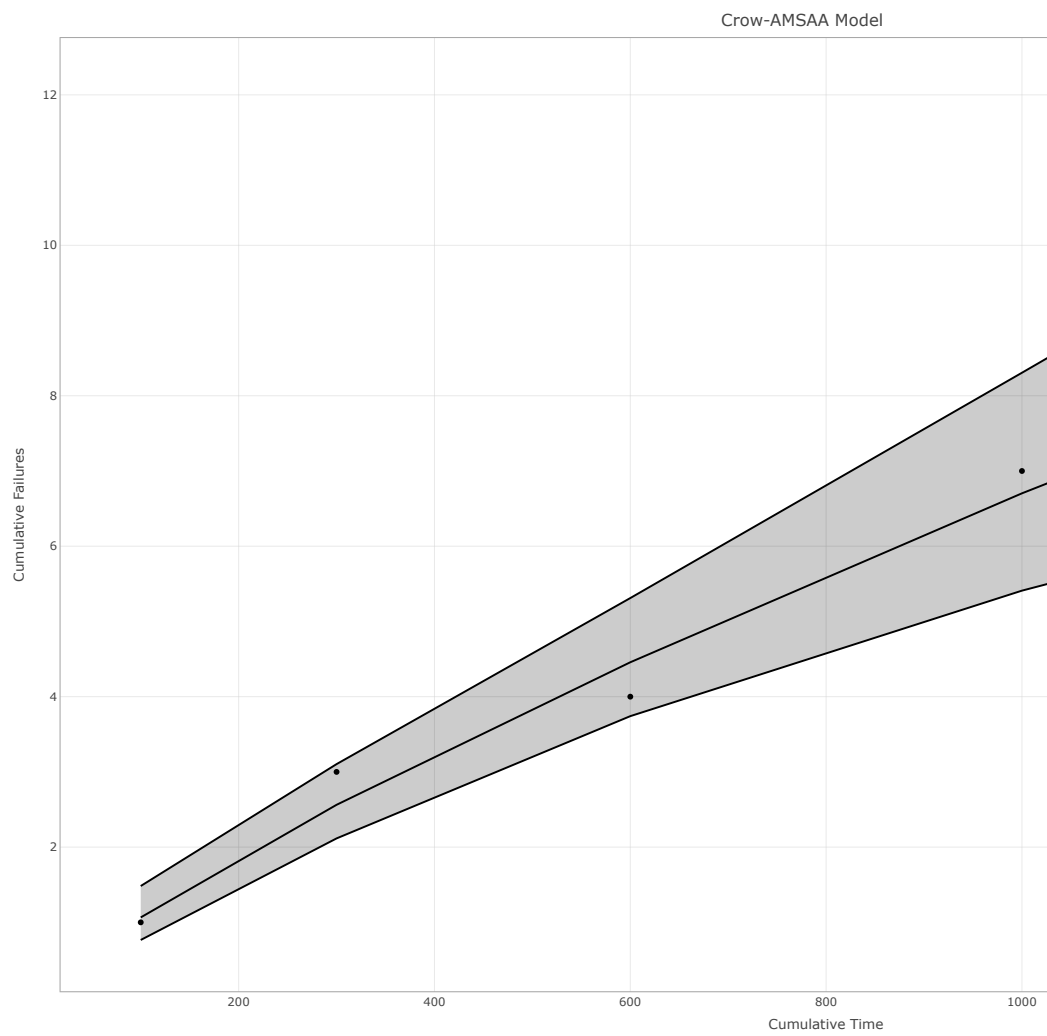


Figure 7.7: Interactive Crow-AMSAA cumulative failures plot

Hover over any data point to read the exact cumulative time and failure count. The fitted curve and confidence bounds can be toggled via the legend.

7.5 Accelerated Life Testing Plots

These functions accept `alt.make()` and `alt.fit()` objects from `WeibullR.ALT` — see Chapter 5.

7.5.1 ALT Probability Plot — `plotly_alt()`

```
t41      <- NelsonData("table4.1")
set170 <- alt.data(t41$C170f, s = t41$C170s, stress = 170)
set190 <- alt.data(t41$C190f, s = t41$C190s, stress = 190)
set220 <- alt.data(t41$C220f, s = t41$C220s, stress = 220)

arrobj <- alt.make(list(set170, set190, set220),
                    dist = "lognormal", alt.model = "arrhenius",
                    method.fit = "mle", view_dist_fits = FALSE)
prlobj <- alt.parallel(arrobj)

plotly_alt(prlobj,
            main = "Arrhenius-Lognormal ALT Probability Plot",
            xlab = "Time to Failure (hours)",
            ylab = "Probability of Failure")
```

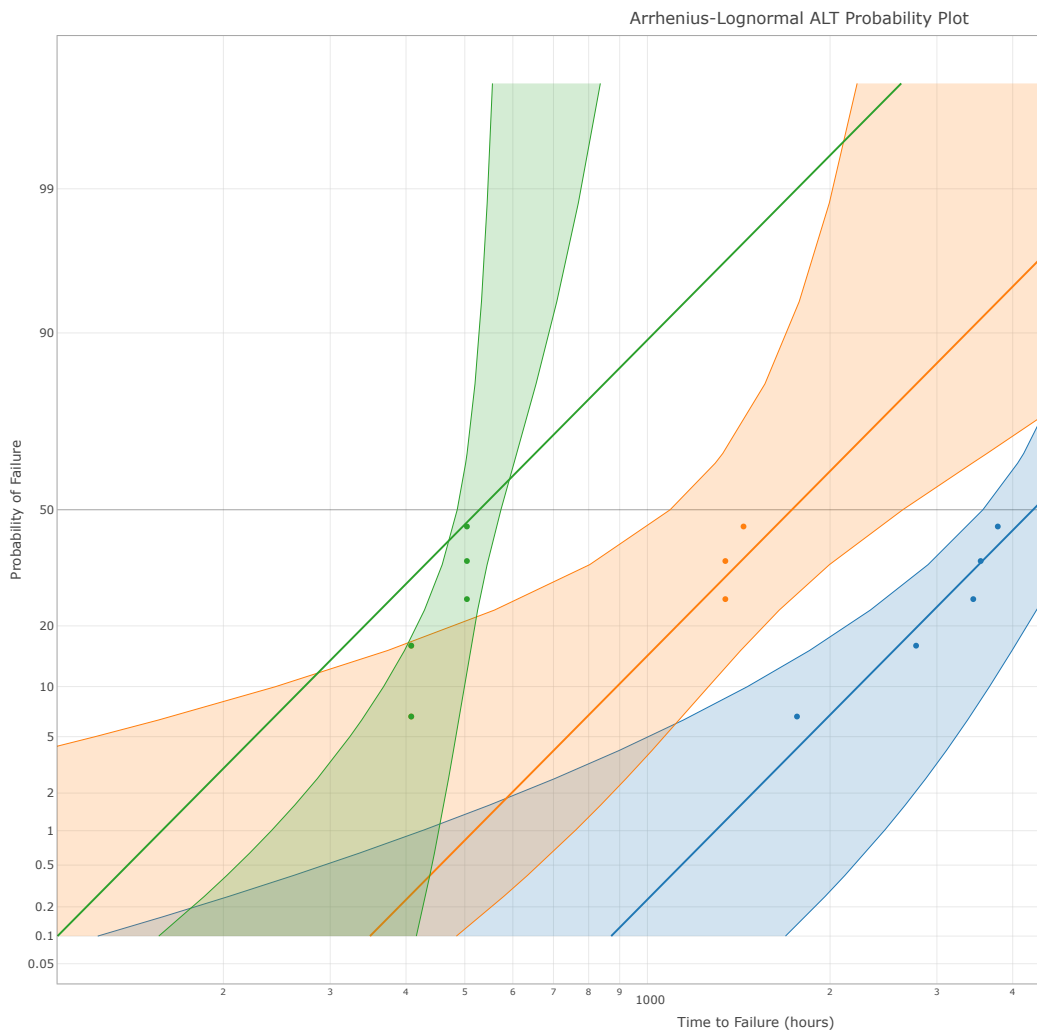


Figure 7.8: Interactive ALT probability plot: Arrhenius-Lognormal model (Class B insulation)

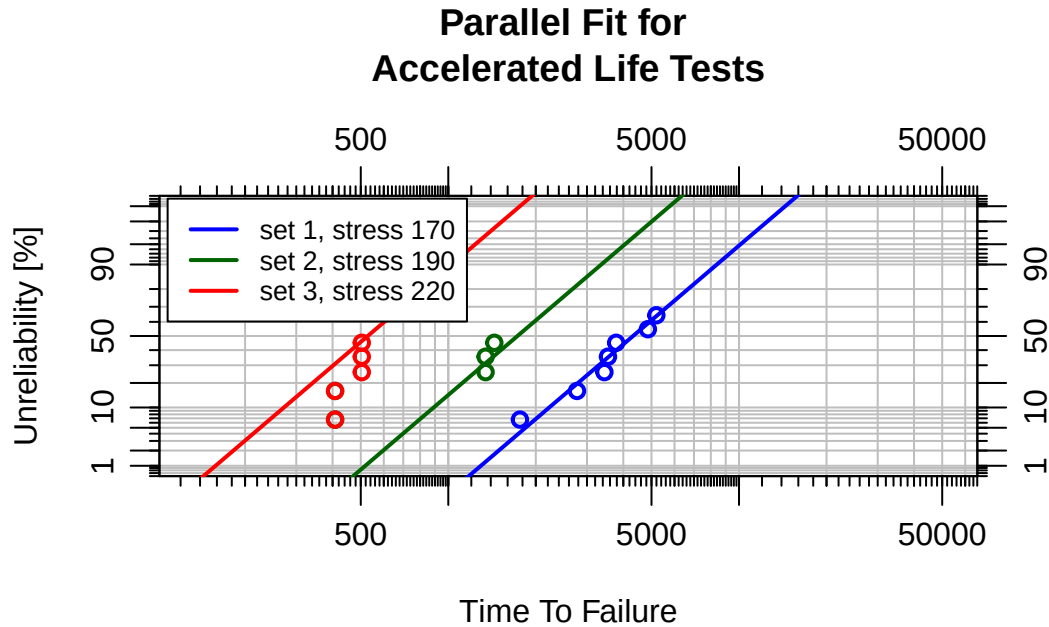


Figure 7.9: Interactive ALT probability plot: Arrhenius-Lognormal model (Class B insulation)

Each color represents a different temperature level. Hover to inspect unreliability at any time. Note: `plotly_alt()` requires a parallelized object from `alt.parallel()`.

7.5.2 Life-Stress Relationship Plot — `plotly_rel()`

```
lnrobject <- alt.fit(prlobject)
plotly_rel(lnrobject,
  main = "Life-Stress Relationship: Arrhenius Model",
  xlab = "Temperature (°C)",
  ylab = "Time to Failure")
```

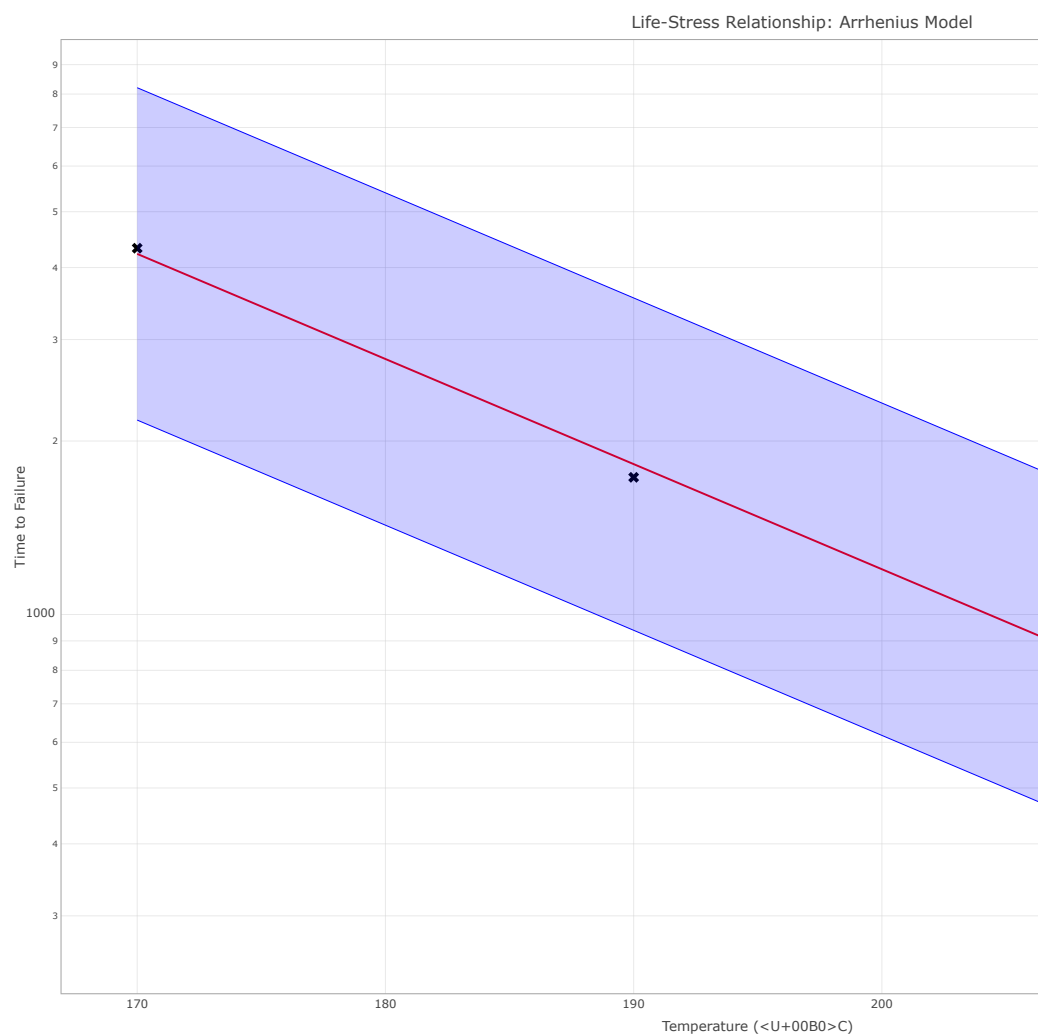


Figure 7.10: Interactive life-stress relationship plot

Each line represents a percentile of the failure distribution (P10, P63.2, P90, etc.). Hover over any point to read the expected life at that stress level and percentile.

7.6 Repairable Systems Plots

These functions accept the fitted objects from `ReliaGrowR` — the same objects produced in Chapter 6.

7.6.1 NHPP Plot — `plotly_nhpp()`

```
times_pump    <- c(500, 1200, 2100, 3300, 4800, 6500, 8400, 10500, 12800, 15000)
failures_pump <- c(3, 4, 4, 5, 4, 3, 3, 3, 2, 2)

fit_nhpp <- nhpp(time = times_pump, event = failures_pump)
plotly_nhpp(fit_nhpp,
             main = "Crow-AMSAA: Industrial Pump Fleet",
             xlab = "Cumulative Operating Hours",
             ylab = "Cumulative Failures")
```

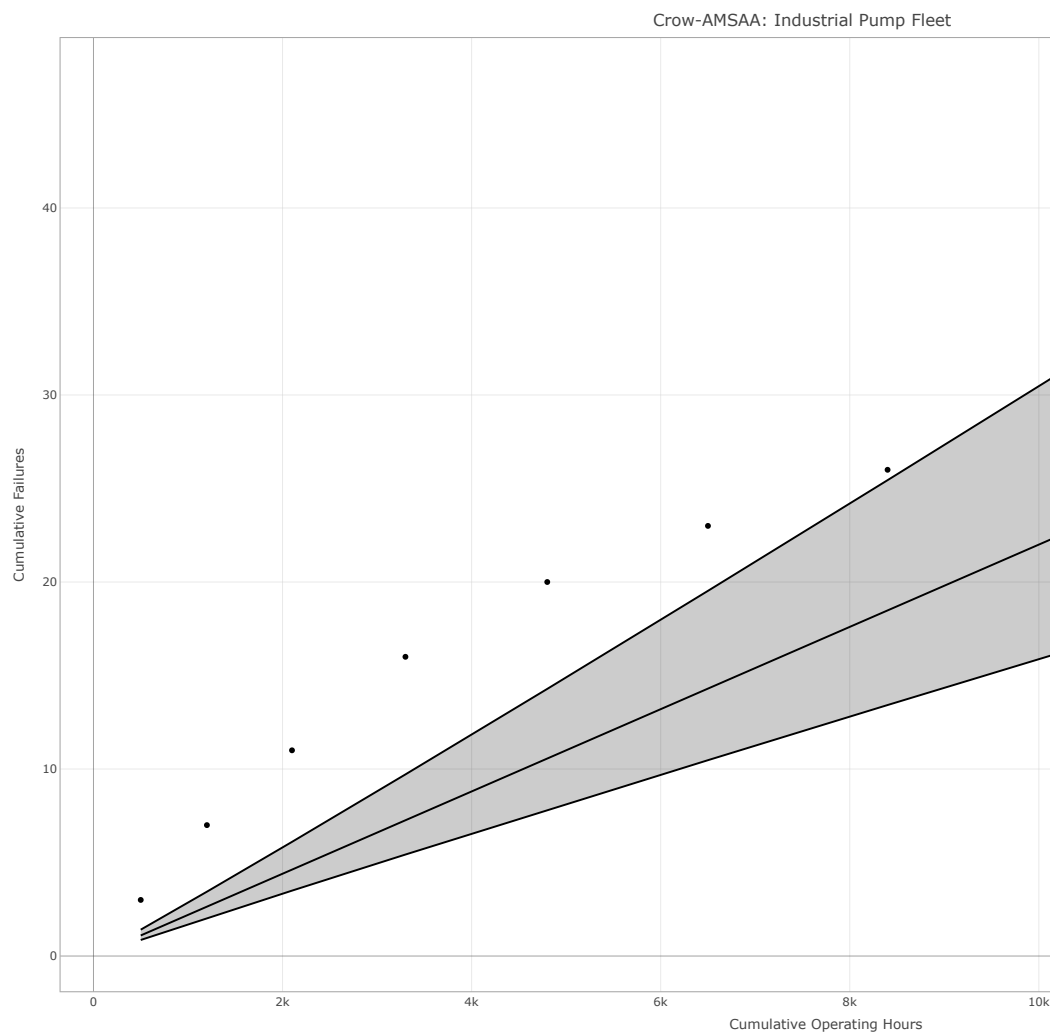


Figure 7.11: Interactive Crow-AMSAA NHPP plot: industrial pump fleet

7.6.2 MCF Plot — `plotly_mcf()`

```
pump_data <- data.frame(  
  id    = c(1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 5),  
  time  = c(310, 850, 1620, 420, 1050, 2100, 580, 1890,  
            240, 710, 1380, 2400, 530, 1740),  
  event = rep(1, 14)  
)  
end_times <- c("1" = 3000, "2" = 3000, "3" = 3000, "4" = 3000, "5" = 3000)  
  
mcf_result <- mcf(id = pump_data$id, time = pump_data$time,  
                  event = pump_data$event, end_time = end_times)  
plotly_mcf(mcf_result,  
  main = "MCF: Industrial Pump Fleet",  
  xlab = "Operating Hours",  
  ylab = "Expected Cumulative Failures per System")
```

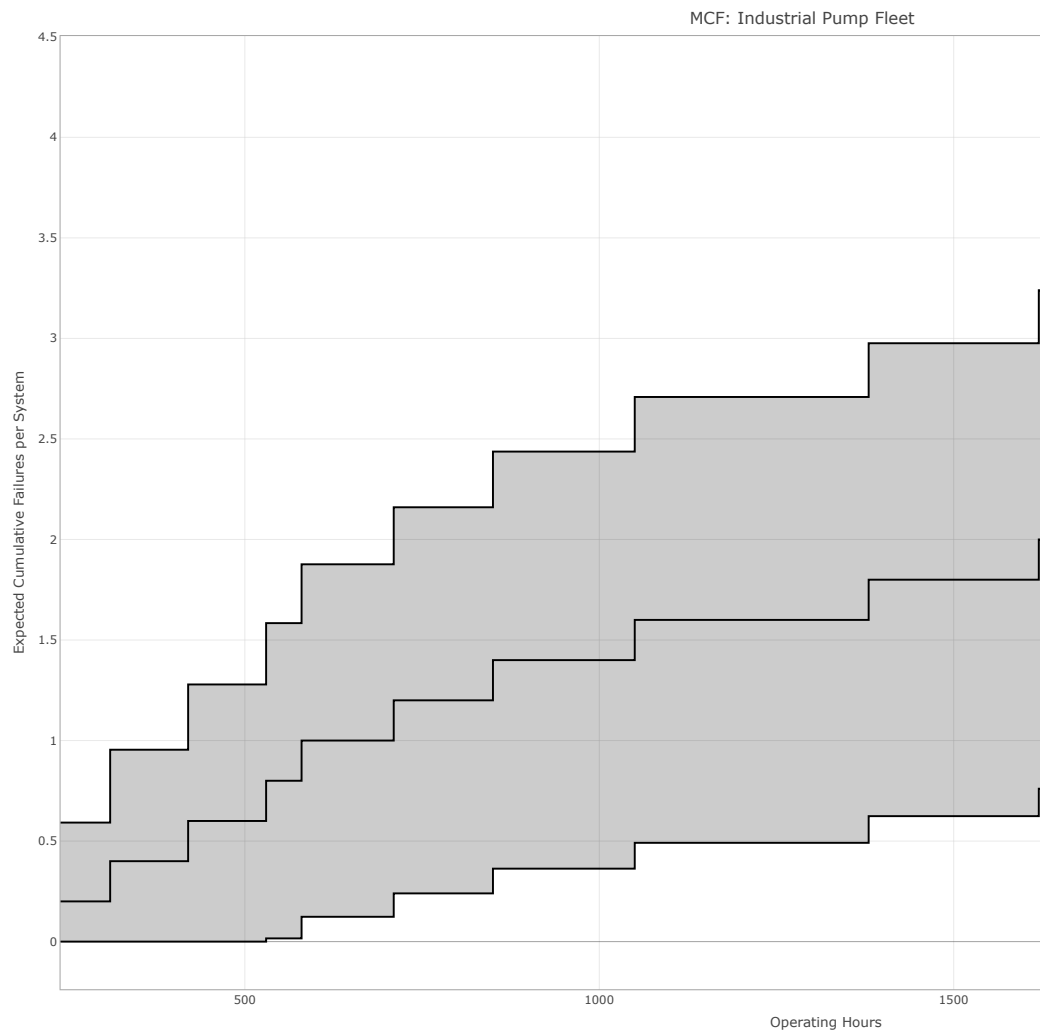


Figure 7.12: Interactive Mean Cumulative Function plot with confidence bounds

7.6.3 Exposure Plot — `plotly_exposure()`

```
exp_result <- exposure(id = pump_data$id, time = pump_data$time,  
                       event = pump_data$event)  
plotly_exposure(exp_result,  
                 main = "Fleet Exposure Plot",  
                 xlab = "Operating Hours",  
                 ylab = "Event Rate (failures / hour)")
```

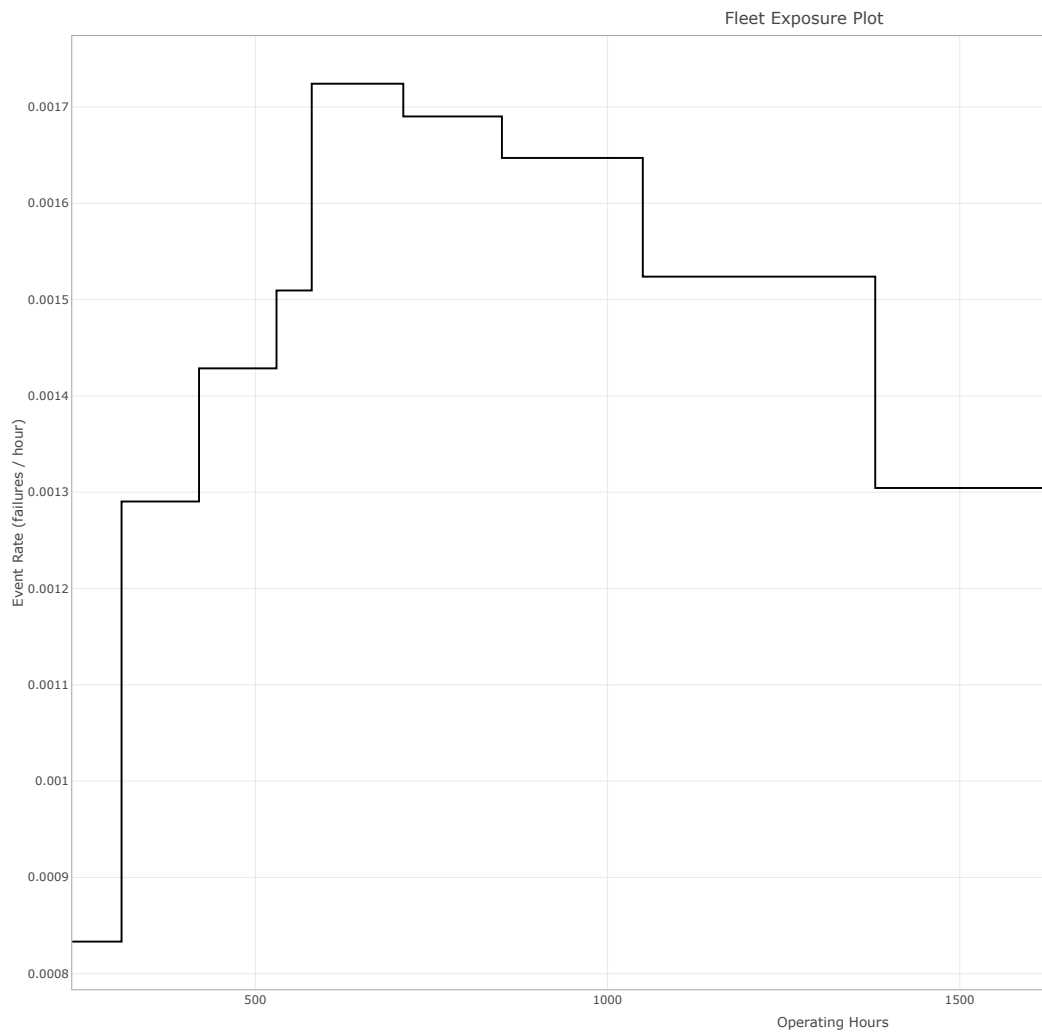


Figure 7.13: Interactive exposure plot showing fleet-level event rate

7.7 Embedding in Reports and Dashboards

ReliaPlotR outputs are standard `plotly` objects and embed natively in:

Quarto / R Markdown — interactive widgets in HTML output, static fallback in PDF:

```
```{r}
library(ReliaPlotR)
library(WeibullR)
failures <- c(30, 49, 82, 90, 96)
obj <- wblr.conf(wblr.fit(wblr(failures)))
plotly_wblr(obj)
```
```

Shiny — use `plotlyOutput()` / `renderPlotly()`:

```
library(shiny)
library(plotly)
library(ReliaPlotR)
library(WeibullR)

ui <- fluidPage(plotlyOutput("wblr_plot"))

server <- function(input, output) {
  output$wblr_plot <- renderPlotly({
    obj <- wblr.conf(wblr.fit(wblr(c(30, 49, 82, 90, 96))))
    plotly_wblr(obj)
  })
}

shinyApp(ui, server)
```

7.8 When to Use ReliaPlotR

| Use ReliaPlotR when... | Use base WeibullR / ReliaGrowR plots when... |
|---|--|
| Delivering results to non-R stakeholders | Running quick exploratory analysis in a script |
| Embedding in an HTML report or dashboard | Generating PDF / print output directly |
| Stakeholders need to hover and inspect points | Batch-processing many analyses |
| Presenting in a web browser or Shiny app | Working in a non-HTML environment |

7.9 Getting Help

```
help(package = "ReliaPlotR")
```

- CRAN page: <https://CRAN.R-project.org/package=ReliaPlotR>

7.10 Summary

ReliaPlotR wraps every major fitted object in the ReliaLearnR ecosystem — `wblr`, `duane`, `rga`, `nhpp`, `mcf`, `exposure`, `alt.make`, `alt.fit` — in an interactive plotly chart. The pattern is consistent: fit your model with WeibullR, ReliaGrowR, or WeibullR.ALT as usual, then pass the result to the corresponding `plotly_*` function. All nine functions support the same customization arguments (titles, axis labels, colors, grid control) and return standard plotly objects compatible with Quarto, R Markdown, and Shiny.

8 Web-Based Analysis with ReliaShiny

8.1 Introduction

All of the analyses in this book require writing R code. **ReliaShiny** (Govan 2023b) removes that requirement — it is a point-and-click Shiny (Chang et al. 2022) web application that exposes the same reliability analysis workflows (RAM, life data analysis, reliability testing, repairable systems, reliability block diagrams) through a graphical interface. No R programming knowledge is needed to run an analysis.

This chapter introduces ReliaShiny, tours its modules, and explains when to reach for it versus a scripted R workflow.

8.2 Installation and Launch

ReliaShiny is available on CRAN:

```
install.packages("ReliaShiny")
```

Once installed, launch the app from the R console:

```
library(ReliaShiny)  
ReliaShiny()
```

This opens the application in your default web browser. The app runs locally — no internet connection or account is required.

8.3 App Overview

When ReliaShiny opens, the left sidebar displays the available analysis modules, each corresponding to a chapter in this book:

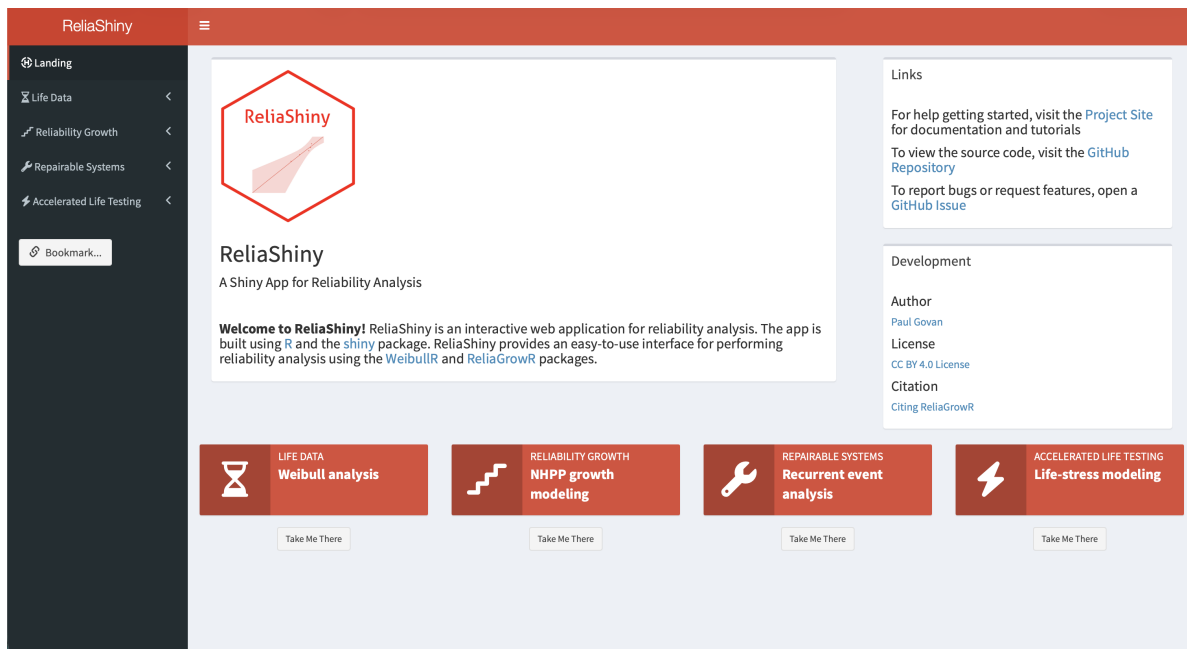


Figure 8.1: ReliaShiny landing page showing the sidebar module list.

| Module | Corresponding chapter |
|-----------------------------------|---------------------------|
| RAM | Chapter 1 |
| Life Data Analysis | Chapter 3 |
| Reliability Growth | Chapter 4 |
| Accelerated Life Testing | Chapter 5 |
| Repairable Systems | Chapter 6 |
| Reliability Block Diagrams | Chapter 2 |

Each module follows the same three-step workflow:

1. **Input** — enter or upload your data using form fields, tables, or file uploads.
2. **Analyze** — click **Run** to fit the model.
3. **Output** — view plots, parameter estimates, and summary statistics; download results.

8.4 Module Walkthroughs

8.4.1 RAM Module

The RAM module computes reliability, availability, MTTR, MTTF, MTBF, and failure rate from summary inputs.

Workflow:

1. Select the *RAM* tab in the sidebar.
2. Enter the total time, failed time, and (for availability) scheduled maintenance time.
3. Click **Calculate**.
4. The panel displays all five metrics: Reliability, Availability, MTTR, MTTF, MTBF, and Failure Rate.

This mirrors the `rel()`, `avail()`, `mttf()`, `mtbf()`, and `fr()` helper functions from Chapter 1.

8.4.2 Life Data Analysis Module

The LDA module fits Weibull models and generates probability plots.

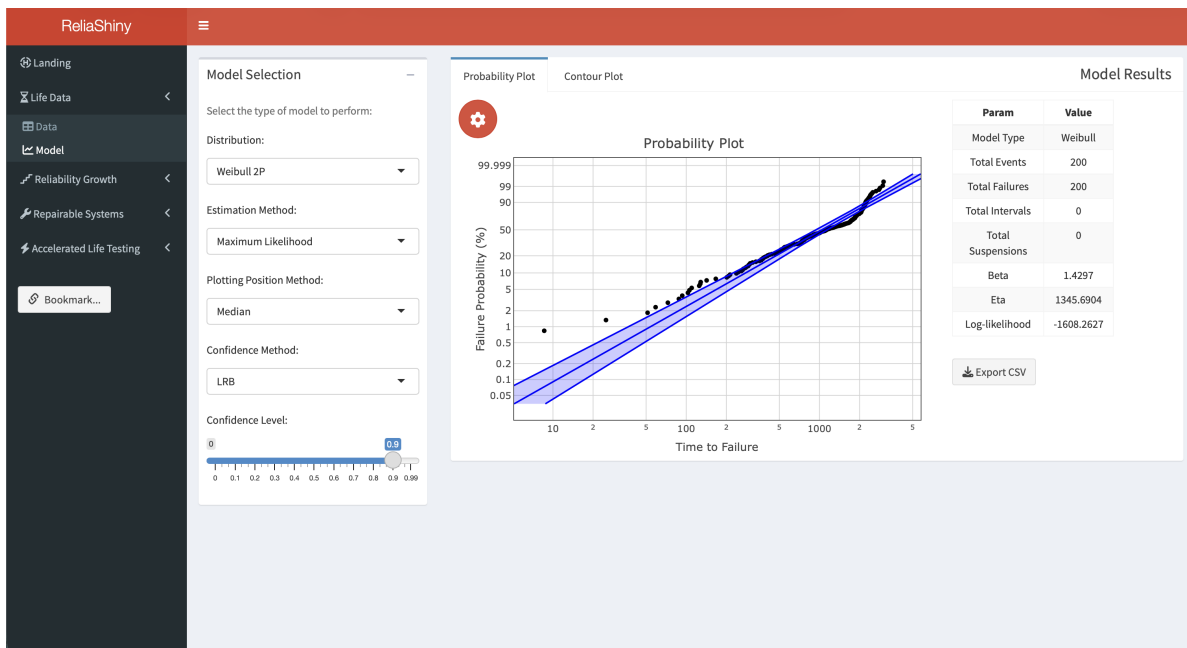


Figure 8.2: LDA module showing a fitted Weibull probability plot.

Workflow:

1. Select the *Life Data Analysis* tab.
2. **Upload data** — paste failure times into the input table, or upload a CSV with columns `time` and `event` (1 = failure, 0 = suspension).
3. **Choose model** — 2-parameter Weibull (default), 3-parameter Weibull, or Weibayes.
4. **Choose estimation method** — MRR or MLE.
5. Click **Fit Model**.
6. The probability plot appears in the main panel, with $\hat{\beta}$, $\hat{\eta}$, and the Anderson-Darling statistic in the sidebar.
7. Click **Download Plot** to save the chart as a PNG.

CSV format for upload:

```
time,event
30,1
49,1
82,1
90,1
96,1
100,0
45,0
10,0
```

A 0 in the `event` column marks a suspension (right-censored observation).

8.4.3 Reliability Growth Module

The Reliability Growth module fits Duane and Crow-AMSAA reliability growth models.

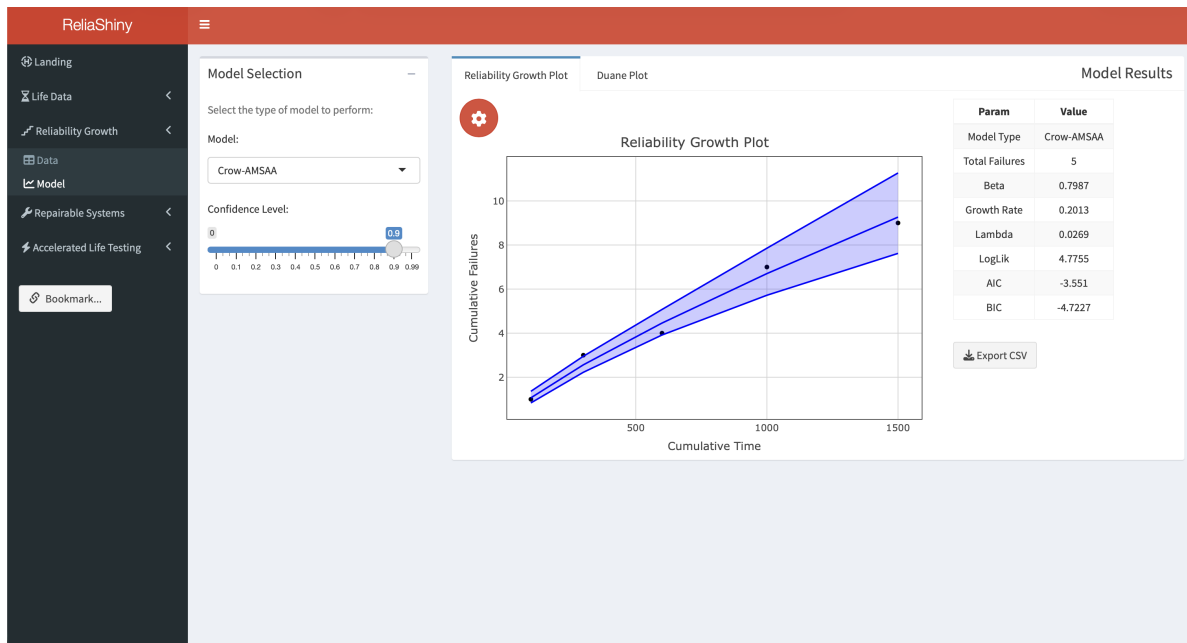


Figure 8.3: Reliability Growth module showing a fitted Crow-AMSAA cumulative failures plot.

Workflow:

1. Select *Reliability Growth*.
2. Enter cumulative test times and failure counts in the data table.
3. Choose **Duane** or **Crow-AMSAA** model.
4. Click **Fit**.
5. The plot shows the fitted model; $\hat{\beta}$ and model parameters appear in the results panel.

8.4.4 Accelerated Life Testing Module

The ALT module fits Arrhenius and Power Law acceleration models using the `WeibullR.ALT` package (see Chapter 5).

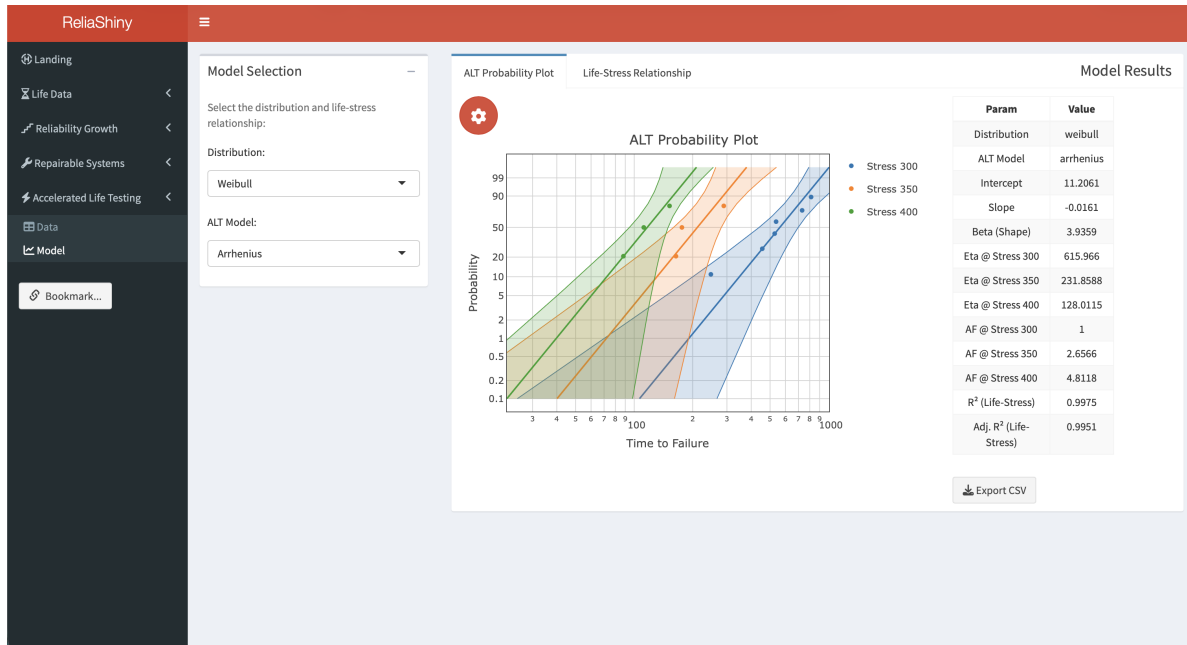


Figure 8.4: ALT module showing a probability plot and life-stress relationship.

Workflow:

1. Select *Accelerated Life Testing*.
2. Upload a dataset with columns **time**, **event**, **qty**, and a stress column (e.g., **TempC** or **Voltage**).
3. Choose the acceleration model: **Arrhenius** (temperature) or **Power Law** (mechanical/voltage stress).
4. Click **Fit ALT Model**.
5. The probability plot and relationship plot are displayed, along with fitted parameters and the computed Acceleration Factor.

8.4.5 Repairable Systems Module

The Repairable Systems module fits the Power Law Process (Crow-AMSAA) and computes the MCF for fleet data.

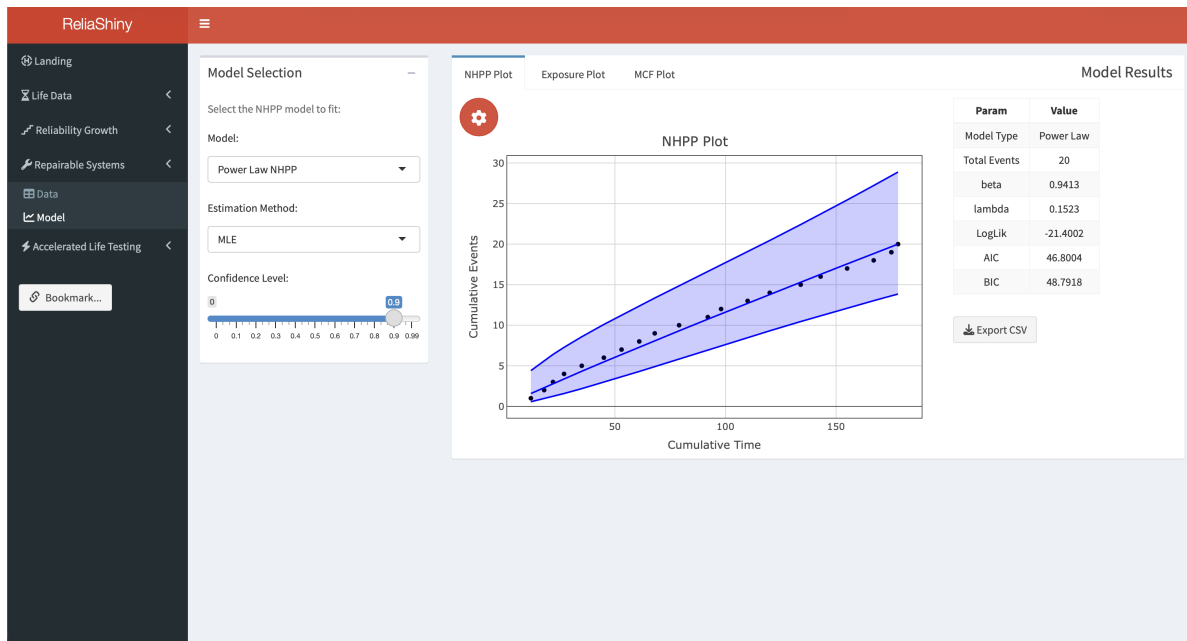


Figure 8.5: Repairable Systems module showing a Mean Cumulative Function plot.

Workflow:

1. Select *Repairable Systems*.
2. Upload fleet data with columns `id`, `time`, and `event`.
3. Choose analysis type: **NHPP (Power Law)** or **MCF**.
4. Optionally enter a breakpoint for piecewise NHPP fitting.
5. Click **Analyze**.
6. The cumulative failures plot (NHPP) or MCF curve is displayed, with estimated parameters.

Example CSV format:

```
id,time,event
1,310,1
1,850,1
1,1620,1
2,420,1
2,1050,1
2,3000,0
```

The last row for system 2 (`event = 0`) indicates that system 2 was observed until hour 3,000 without a third failure.

8.4.6 Reliability Block Diagrams Module

The RBD module computes system reliability from component reliabilities for series, parallel, mixed, and k-out-of-n configurations.

Workflow:

1. Select *Reliability Block Diagrams*.
2. Choose the system **topology** from the dropdown: Series, Parallel, Mixed, or k-out-of-n.
3. Enter the number of components and their individual reliabilities.
4. For Mixed systems, define subsystem groupings.
5. For k-out-of-n systems, enter k (minimum required) and n (total).
6. Click **Calculate**.
7. System reliability and, for exponential components, system MTTF are displayed.
8. A block diagram graphic is shown for Series and Parallel configurations.

8.5 Data Upload Tips

- **CSV files:** column names are case-sensitive. Use lowercase (`time`, `event`, `id`).
- **Missing values:** rows with NA in the `time` column are automatically excluded.
- **Large datasets:** the app handles datasets up to ~10,000 rows without performance issues.
- **Units:** ReliaShiny is unit-agnostic — use hours, years, cycles, or any consistent unit. Label your outputs accordingly.

8.6 Downloading Results

Every module provides download buttons for:

- **Plot PNG** — the current chart at screen resolution.
- **Results CSV** — parameter estimates and summary statistics in tabular form.
- **Report HTML** — a self-contained report combining the inputs, plot, and results (available in LDA and RT modules).

8.7 When to Use ReliaShiny vs Scripted R

| Situation | ReliaShiny | Scripted R |
|--------------------------|------------|------------|
| Quick one-off analysis | | |
| Teaching / demonstration | | |

| Situation | ReliaShiny | Scripted R |
|---|------------|------------|
| Sharing with non-R users | | |
| Automating repeated analyses | | |
| Custom plots and formatting | | |
| Reproducible research / version control | | |
| Embedding results in a Quarto book | | |
| Large batch processing | | |

ReliaShiny excels at rapid exploration, prototyping, and sharing results with stakeholders who do not use R. For production workflows, automated pipelines, and fully reproducible analyses, scripted R with the underlying packages (WeibullR, ReliaGrowR, WeibullR.ALT) gives more control.

8.8 Getting Help

Within the app, click the ? icon on any panel for a tooltip explaining the inputs and outputs for that module.

From R:

```
help(package = "ReliaShiny")
```

Additional resources:

- CRAN page: <https://CRAN.R-project.org/package=ReliaShiny>

8.9 Summary

ReliaShiny provides a point-and-click interface to the full ReliaLearnR analysis ecosystem — RAM, life data analysis, reliability testing, repairable systems, and reliability block diagrams — with no R coding required. Install it with `install.packages("ReliaShiny")` and launch it with `ReliaShiny()`. Use it for rapid exploration, stakeholder demonstrations, and sharing results; switch to scripted R for reproducibility, automation, and custom outputs.

Appendix: Helper Functions Reference

The `ReliaLearnR` package exports five RAM calculation helper functions. They accept scalar or vector inputs, validate their arguments, and return a single numeric scalar.

```
library(ReliaLearnR)
```

`rel()` — Reliability

Computes reliability as $1 - \text{outage}/\text{total}$.

```
rel(outageTime, totalTime)
```

| Argument | Type | Description |
|-------------------------|--------------------------|------------------------------|
| <code>outageTime</code> | numeric scalar or vector | Forced outage (failed) time |
| <code>totalTime</code> | numeric scalar or vector | Total time in the same units |

Returns: Numeric scalar in $[0, 1]$. Sums vectors before dividing.

Validation: Errors on NA, non-numeric, negative values, zero `totalTime`, or mismatched vector lengths.

```
# Motor: failed 10 days out of 5 years (1825 days)
rel(outageTime = 10, totalTime = 5 * 365)
```

```
[1] 0.9945205
```

```
# Vector input: two machines pooled together
rel(outageTime = c(10, 5), totalTime = c(365, 730))
```

```
[1] 0.9863014
```

avail() — Availability

Computes **availability** as $1 - \text{unavailable}/\text{total}$, where unavailable time includes both failures *and* scheduled maintenance.

```
avail(unavailTime, totalTime)
```

| Argument | Type | Description |
|-------------|--------------------------|---|
| unavailTime | numeric scalar or vector | Total unavailable time (failed + maintenance) |
| totalTime | numeric scalar or vector | Total time in the same units |

Returns: Numeric scalar in $[0, 1]$.

```
# Motor: 10 days failed + 15 days maintenance over 5 years
avail(unavailTime = 10 + 15, totalTime = 5 * 365)
```

```
[1] 0.9863014
```

```
# Vector input: two assets
avail(unavailTime = c(5, 14), totalTime = c(3 * 365, 3 * 365))
```

```
[1] 0.9913242
```

mttf() — Mean Time to Failure

Computes **MTTF** as total time/failures, for **non-repairable** items.

```
mttf(failures, totalTime)
```

| Argument | Type | Description |
|-----------|--------------------------|----------------------|
| failures | numeric scalar or vector | Failure counts |
| totalTime | numeric scalar or vector | Total operating time |

Returns: Numeric scalar (same units as `totalTime`). Returns `Inf` with a warning if `failures = 0`.

```
# 100 motors, 5 years each, 5 failures
mttf(failures = 5, totalTime = 100 * 5)
```

```
[1] 100
```

```
# Pooled fleet data
mttf(failures = c(2, 3), totalTime = c(500, 500))
```

```
[1] 200
```

mtbf() — Mean Time Between Failures

Computes MTBF as total time/failures, for **repairable** systems.

The formula is identical to `mttf()` but the semantic interpretation differs — MTBF is the expected time *between* successive failures on the same repairable system.

```
mtbf(failures, totalTime)
```

| Argument | Type | Description |
|------------------------|--------------------------|----------------------|
| <code>failures</code> | numeric scalar or vector | Failure counts |
| <code>totalTime</code> | numeric scalar or vector | Total operating time |

Returns: Numeric scalar. Returns `Inf` with a warning if `failures = 0`.

```
# Machine failed 5 times in 45,000 operating hours
mtbf(failures = 5, totalTime = 45000)
```

```
[1] 9000
```

```
# Fleet of 50 generators, 8 years, 20 failures
mtbf(failures = 20, totalTime = 50 * 8)
```

```
[1] 20
```

fr() — Failure Rate

Computes failure rate () as failures/total time.

```
fr(failures, totalTime)
```

| Argument | Type | Description |
|------------------------|--------------------------|----------------------|
| <code>failures</code> | numeric scalar or vector | Failure counts |
| <code>totalTime</code> | numeric scalar or vector | Total operating time |

Returns: Numeric scalar (failures per unit time). Note: $\lambda = 1/\text{MTBF}$.

```
# 100 motors, 5000 hours each, 75 failures
fr(failures = 75, totalTime = 100 * 5000)
```

```
[1] 0.00015
```

```
# Pooled data
fr(failures = c(10, 5), totalTime = c(1000, 2000))
```

```
[1] 0.005
```

Validation Behaviour

All five functions share the same input checks via an internal `.validate_inputs()` helper:

| Condition | Error message |
|---------------------------|--|
| NA in either argument | "NA values not allowed in inputs" |
| Non-numeric argument | "argument" must be numeric' |
| Negative values | "inputs must be non-negative" |
| Empty vector | "inputs must not be empty vectors" |
| Mismatched vector lengths | "inputs must have the same length or one must be a scalar" |

| Condition | Error message |
|-----------------|--|
| Zero total time | "sum(totalTime) is zero (cannot divide by zero)" |

```
# These all produce informative errors:
rel(NA, 100)
```

```
Error in `rel()` :
! NA values not allowed in inputs
```

```
avail(-5, 100)
```

```
Error in `avail()` :
! times must be non-negative
```

```
mttf(5, 0)
```

```
Error in `mttf()` :
! sum(totalTime) is zero (cannot compute MTTF)
```

Glossary and Formula Reference

A quick-reference guide to the key symbols and formulas used throughout this book. Each entry links back to the chapter where it is first introduced.

Key Symbols

| Symbol | Name | Definition | First used |
|-----------|---------------------------------------|---|---------------------------|
| β | Shape parameter | Controls whether the failure rate increases, decreases, or stays constant over time | Chapter 1 |
| η | Scale parameter (characteristic life) | Time at which $F(t) = 63.2\%$ for any β | Chapter 3 |
| λ | Failure rate | Failures per unit time (constant rate — exponential model) | Chapter 1 |
| $\rho(t)$ | Intensity / recurrence rate | Time-varying failure rate for repairable systems | Chapter 6 |
| $R(t)$ | Reliability function | Probability of survival to time t | Chapter 1 |
| $F(t)$ | Cumulative distribution function | Probability of failure by time t ; $F(t) = 1 - R(t)$ | Chapter 3 |

| Symbol | Name | Definition | First used |
|--------|----------------------------|--|------------|
| MTTF | Mean Time To Failure | Expected life for non-repairable items | Chapter 1 |
| MTBF | Mean Time Between Failures | Expected time between failures for repairable systems | Chapter 1 |
| MTTR | Mean Time To Repair | Expected time to restore a failed system | Chapter 1 |
| AF | Acceleration Factor | Ratio of life at use conditions to life at elevated stress | Chapter 5 |
| E_a | Activation energy | Energy barrier for thermally activated failure (eV) | Chapter 5 |
| k | Boltzmann constant | 8.617×10^{-5} eV/K | Chapter 5 |
| n | Stress exponent | Sensitivity of life to non-thermal stress (Power Law) | Chapter 5 |
| t_0 | Failure-free period | Minimum time before any failure can occur (3-parameter Weibull) | Chapter 3 |
| AD | Anderson-Darling statistic | Goodness-of-fit measure; lower values indicate a better fit | Chapter 3 |
| MCF | Mean Cumulative Function | Expected cumulative failures per system over time (non-parametric) | Chapter 6 |

| Symbol | Name | Definition | First used |
|----------------------|--------------------|--|------------|
| β_{CCF} | CCF
beta-factor | Fraction of total failure rate attributable to common cause failures | Chapter 2 |

Key Formulas

RAM — Chapter 1

$$R = 1 - \frac{\text{failed time}}{\text{total time}}$$

$$A = 1 - \frac{\text{unavailable time}}{\text{total time}}$$

$$\lambda = \frac{\text{failures}}{\text{total time}}, \quad \text{MTTF} = \frac{1}{\lambda}$$

$$R(t) = e^{-\lambda t} \quad (\text{exponential model})$$

$$B_n \text{ life : time } t \text{ such that } F(t) = n/100$$

Weibull Distribution — Chapter 3

$$R(t) = \exp\left[-\left(\frac{t}{\eta}\right)^\beta\right], \quad F(t) = 1 - R(t)$$

$$F(\eta) = 1 - e^{-1} \approx 63.2\% \quad \text{for any } \beta$$

3-parameter Weibull with failure-free period t_0 :

$$R(t) = \exp\left[-\left(\frac{t-t_0}{\eta}\right)^\beta\right]$$

Reliability Block Diagrams — Chapter 2

$$R_{\text{series}} = \prod_{i=1}^n R_i$$

$$R_{\text{parallel}} = 1 - \prod_{i=1}^n (1 - R_i)$$

$$R_{\text{k-out-of-n}} = \sum_{i=k}^n \binom{n}{i} R^i (1 - R)^{n-i}$$

Beta-factor CCF model:

$$\lambda_{\text{CCF}} = \beta_{\text{CCF}} \cdot \lambda_{\text{total}}, \quad \lambda_{\text{ind}} = (1 - \beta_{\text{CCF}}) \cdot \lambda_{\text{total}}$$

Accelerated Life Testing — Chapter 5

Arrhenius (temperature):

$$AF = \exp \left[\frac{E_a}{k} \left(\frac{1}{T_{\text{use}}} - \frac{1}{T_{\text{stress}}} \right) \right]$$

Power Law (non-thermal stress):

$$AF = \left(\frac{S_{\text{stress}}}{S_{\text{use}}} \right)^n$$

$$\text{Life}_{\text{use}} = AF \times \text{Life}_{\text{stress}}$$

Reliability Growth Analysis — Chapter 4

Duane cumulative MTBF:

$$\text{CMTBF}(t) = K \cdot t^{\beta-1}$$

Crow-AMSAA cumulative failures:

$$N(t) = \lambda_0 \cdot t^\beta$$

| β (Crow-AMSAA) | Meaning |
|----------------------|---|
| < 1 | Failures decreasing — reliability improving |
| $= 1$ | Constant rate — stable |
| > 1 | Failures increasing — reliability worsening |

Repairable Systems — Chapter 6

Power Law intensity (recurrence rate):

$$\rho(t) = \lambda \beta t^{\beta-1}$$

Instantaneous MTBF at time t :

$$\text{MTBF}(t) = \frac{1}{\rho(t)}$$

Abernethy, Robert B. 2004. *The New Weibull Handbook*. 5th ed. R.B. Abernethy.

Ascher, Harold, and Harry Feingold. 1984. *Repairable Systems Reliability*. Marcel Dekker.

Billinton, Roy, and Ronald N. Allan. 1992. *Reliability Evaluation of Engineering Systems*. 2nd ed. Plenum Press.

Chang, Winston, Joe Cheng, JJ Allaire, et al. 2022. *Shiny: Web Application Framework for r*. <https://doi.org/10.32614/CRAN.package.shiny>.

- Crow, Larry H. 1975. *Reliability Analysis for Complex, Repairable Systems*. Army Material Systems Analysis Activity. <https://apps.dtic.mil/sti/citations/ADA020296>.
- Duane, J. T. 1964. “Learning Curve Approach to Reliability Monitoring.” *IEEE Transactions on Aerospace* 2: 563–66. <https://doi.org/10.1109/TA.1964.4319640>.
- Govan, Paul. 2023a. *ReliaPlotR: Interactive Reliability Analysis Plots*. <https://doi.org/10.32614/CRAN.package.ReliaPlotR>.
- Govan, Paul. 2023b. *ReliaShiny: A ‘Shiny’ App for Reliability Analysis*. <https://doi.org/10.32614/CRAN.package.ReliaShiny>.
- Govan, Paul. 2024. *ReliaGrowR: Reliability Growth Analysis*. <https://doi.org/10.32614/CRAN.package.ReliaGrowR>.
- Guo, H., A. Mettas, G. Sarakakis, and P. Niu. 2010. “Piecewise NHPP Models with Maximum Likelihood Estimation for Repairable Systems.” *2010 Proceedings – Annual Reliability and Maintainability Symposium (RAMS)*, 1–7. <https://doi.org/10.1109/RAMS.2010.5448029>.
- Meeker, William Q., and Luis A. Escobar. 1998. *Statistical Methods for Reliability Data*. Wiley-Interscience.
- Nelson, Wayne. 1990. *Accelerated Testing*. Wiley-Interscience.
- Silkworth, David. 2020. “WeibullR: An r Package for Weibull Analysis for Reliability Engineers.” *10th International Symposium on Data Science and Its Applications (ISDSA2019)*, 43–53. <https://doi.org/10.35566/isdsa2019c3>.
- Silkworth, David. 2022. *WeibullR.ALT: Accelerated Life Testing Using ‘WeibullR’*. <https://doi.org/10.32614/CRAN.package.WeibullR.ALT>.
- Silkworth, David, and Jurgen Symynck. 2022. *WeibullR: Weibull Analysis for Reliability Engineering*. <https://doi.org/10.32614/CRAN.package.WeibullR>.
- Vesely, W., F. Goldberg, N. Roberts, and D. Haasl. 1981. *Fault Tree Handbook*. NUREG-0492. US Nuclear Regulatory Commission.